MindVideo AI

TensorFlow

من الأساس إلى الإنتاج

الأساسيات › بناء النماذج › التدريب والتقييم › النشر والإنتاج



م/ أحمد صالح المغز

2026

مرجع عربي شامل

TensorFlow الاحترافي

من الأساس إلى الإنتاج

مرجع متكامل للتعلّم العميق والشبكات العصبية وتطبيقاتها الواقعية، من أول تنسور إلى النشر والمراقبة في الإنتاج.

م/أحمد صالح المغز

TensorFlow 2.21 · Keras 3.15.1 · Python 3.13 (الرسمي TensorFlow توافق)

الإصدار الأول · 2026

بيانات الكتاب

العنوان	TensorFlow الاحترافي – من الأساس إلى الإنتاج
المؤلف	م/أحمد صالح المغز
الإصدار	الإصدار الأول · 2026
المراجع التقنية	Python 3.13 · Keras 3.15.1 · TensorFlow 2.21 (توافق TensorFlow الرسمي)
اللغة	العربية الفصحى المعاصرة
التصنيف	تعلّم الآلة · الشبكات العصبية · هندسة البرمجيات

© 2026 م/أحمد صالح المغز. جميع الحقوق محفوظة. لا يُسمح بإعادة إنتاج هذا الكتاب أو توزيعه أو تخزينه في نظام استرجاع، كليًّا أو جزئيًّا، بأي وسيلة إلكترونية أو ميكانيكية، دون إذن كتابي مسبق من المؤلف، باستثناء الاقتباسات القصيرة لأغراض المراجعة والتدريس مع الإشارة إلى المصدر.

أسماء TensorFlow Kerasg Pythong علامات تجارية مسجّلة لأصحابها. استُخدمت في هذا الكتاب لأغراض تعليمية وصفية فقط، ولا يعني ذلك أي رعاية أو ارتباط رسمي.

أُعدّت الأمثلة البرمجية لتعمل على الإصدارات المذكورة. قد تتغيّر الواجهات في الإصدارات اللاحقة؛ راجع التوثيق الرسمي عند الترقية. المعلومات الواردة في هذا الكتاب تُقدَّم لأغراض تعليمية، ولا تُعدّ استشارة مهنية أو قانونية.

> «الفكرة الجيدة لا تكفي؛ يجب أن تصل.» – من مقدمة الكتاب

للتواصل والملاحظات: عبر صفحة الكتاب الرسمية · الإصدار الأول 2026

بسم الله الرحمن الرحيم

إهداء

إلى زوجتي **أم عماد**، رفيقة دربي وشريكة أيامي؛ إلى من منحت البيت سكينته، والحلم صبره، والخطوات المتعثرة سببًا جديدًا للمضي. ما كان لهذا الجهد أن يكتمل لولا دعاؤك، ومساندتك، وإيمانك بأن وراء كل فكرة صادقة عملًا يستحق أن يُبذل.

إلى ابنيَّ **عماد** و**م/ عمار**، وهما ما يزالان على مقاعد الجامعة؛ أرى في أسئلتكما فضول العلم، وفي سعيكما وعدًا بمستقبل تصنعانه بأيديكما. أسأل الله أن يفتح لكما أبواب المعرفة، وأن يجعل هذا الكتاب رفيقًا في طريقكما، لا نهايةً له.

إلى ابنتي **الملكة**، آخر العنقود وبهجة البيت؛ لكِ من القلب محبة لا يحدّها وصف، وأمنية بأن تظلي مقبلة على الحياة، واثقةً بأن لكل حلم جميل بداية تستحق المحاولة.

إلى **عمرو**، الذي يرى في التفاصيل عوالم تستحق الاكتشاف؛ وإلى **عصماء**، صاحبة الصبر والعزيمة؛ وإلى **علا**، التي تمنح الحكايات دفأها؛ وإلى **عزام**، الذي يليق باسمه الإصرار: أنتم فرح العمر وسنده.

وإلى كل طالب وطالبة وباحث وباحثة، بدأوا من سؤال صغير، ثم واصلوا التعلم كلما استعصى عليهم الطريق: لعل هذه الصفحات تقرّب فكرة، أو تفتح باب تجربة، أو تذكّركم بأن المحاولة التالية قد تكون بداية الفهم.

لكم جميعًا أهدي هذا الكتاب، محبةً وامتنانًا وأملًا.

— م/أحمد صالح المغز

مقدمة المؤلف

في صيفٍ بعيد، جلستُ أمام شاشةٍ تُضيءُ غرفةً صغيرة، أراقبُ رقمًا واحدًا يتغيّر ببطء: قيمة الخسارة. كان النموذجُ الأولُ الذي كتبتُه لا يعرف شيئًا عن الصور ولا عن اللغة؛ كان يحاول أن يرسم خطًا مستقيمًا يمرُّ بين نقاطٍ مبعثرة. وحين نجح، بعد ساعاتٍ من المحاولة، شعرتُ بأنّي لم أكتبْ أسطرًا من بايثون، بل فتحتُ نافذةً على طريقةٍ جديدةٍ في التفكير: أن تُعطي الآلةَ أمثلةً، فتبحثَ هي عن القانون.

من ذلك اليوم إلى اليوم، تغيّر الكثير. كبرتْ المكتبةُ، وتغيّرتْ واجهاتُها، وتبدّلتْ أسماؤها، ودخلتْ الذكاءاتُ الاصطناعيةُ كلَّ بيتٍ وكلَّ هاتف. لكنّ السؤالَ الذي دفعني إلى الكتابة ظلَّ كما هو: **كيف نفهمُ ما نفعله، بدلًا من أن نُشغّلَ ما لا نفهمه؟**

هذا الكتابُ محاولةٌ صادقةٌ للإجابة. ليس قاموسًا للأوامر، ولا مجموعةً من الوصفات الجاهزة، بل رحلةٌ متدرّجةٌ تبدأ من السؤال «ما هو التنسور؟»، وتمرّ بالمعادلة والكود والتجربة والخطأ، وتنتهي عند نموذجٍ يعملُ في العالم الحقيقي، على هاتفٍ أو خادمٍ أو جهازٍ طبي. بين البداية والنهاية، ستجدُ قصصًا، وأمثلةً محلولة، وتمارينَ تُشبهُ ما ستواجهه فعلًا، وأخطاءً شائعةً سقطتُ فيها أنا نفسي، وأخرى رأيتُها تتكرّر أمامي في قاعات الدرس وفي غرفِ الخوادم.

لماذا TensorFlow في عام 2026؟

قد يسألُ سائل: «أليس هناك ما هو أسهل؟». نعم، هناك أدواتٌ أسهل، وأخرى أسرع في البحث، وثالثةٌ يحبّها مجتمعُ الأبحاث. لكنّ TensorFlow، بعد أكثر من عقدٍ من النضوج، صار **نظامًا بيئيًا كاملًا** لا مكتبةً واحدة: لغةٌ للتنسورات، وواجهةٌ عاليةٌ لبناء النماذج، وخطوطُ بياناتٍ عالية الأداء، وأدواتُ تدريبٍ موزّع، ومنصّةُ نشرٍ على الأجهزة الطرفية، وخوادمُ استدلال، وأدواتُ مراقبةٍ وحوكمة. من يتعلّمُ TensorFlow يتعلّمُ، في الحقيقة، **كيف تنتقل الفكرة من دفترٍ إلى منتج**.

ولا يعني ذلك أنّه الخيارُ الوحيد. الفصلُ الأول يقارنُ بين TensorFlow وPyTorch وJAX بلا تحيّز، ويشرحُ متى يكون كلٌّ منها الخيارَ الصحيح. فالهدفُ ليس أن تُحبّ أداة، بل أن تعرفَ **لماذا** تختارها.

عن الإصدارات: كلمة صريحة

أُكتبُ هذه الأسطرَ في عام 2026، والنسخةُ المرجعيةُ للكتاب هي **TensorFlow 2.21** و**Keras 3.15.1**، وأمثلةُ اللغة مكتوبةٌ بمعايير **Python 3.14**. لكنّ الأمانةَ العلميةَ تفرضُ عليّ قولَ الحقيقة: عجلاتُ TensorFlow الرسميةَ عند لحظة الكتابة تدعمُ Python حتى 3.13، ولم تصدرْ بعد عجلةٌ رسميةٌ مستقرّةٌ لـ 3.14. لذلك خصّصتُ الفصلَ الثاني لثلاثة مساراتٍ عملية: تشغيلُ TensorFlow داخل بيئة Python 3.13، أو داخل الحاوية الرسمية، أو تجربةُ `tf-nightly` عند توفّر عجلةٍ متوافقة. أمّا كودُ الكتاب نفسُه فقد كُتبَ بأسلوبٍ حديثٍ يعملُ على 3.13 و3.14 معًا، مع الإشارة إلى ما يعتمدُ على ميزاتٍ جديدة.

لن أُخفيَ عليك أنّك ستواجهُ أخطاءً. سأُخفّفُ عنك، لكن لن أُخدعك: من يُعلّمُك أنّ الطريقَ مفروشٌ بالورود يبيعُك وهمًا. الطريقُ مليءٌ بـNaN، وبأشكالٍ لا تتوافق، وبذاكرةٍ تنفد، وبمقاييسَ ترفعُ يدها كأنّها تقول: «لستُ واثقًا». ولهذا خُصّصتْ صناديقُ «أخطاء شائعة» في كل فصل، لأنّ الخطأَ ليس عيبًا في الرحلة؛ إنّه جزءٌ من المنهج.

لمن هذا الكتاب؟

- لطالبِ الهندسةِ أو علومِ الحاسبِ الذي يريدُ أن يبني أساسًا لا يُنسى.
- للمهندسِ الذي يعرفُ البرمجةَ ويقفُ على بابِ التعلّمِ العميقِ يبحثُ عن خريطة.
- للباحثِ التطبيقيِّ الذي يريدُ تحويلَ ورقةٍ علميةٍ إلى كودٍ يعمل.
- للمدرّبِ الذي يبحثُ عن مادةٍ عربيةٍ متسلسلةٍ، واضحةٍ، وقابلةٍ للتدريس.
- ولكلّ من قرأَ كثيرًا، وطبّقَ قليلًا، ثم قرّرَ أن يُكملَ الطريق.

لا يُشترطُ أن تكونَ خبيرًا في الرياضيات. الملحقُ (أ) يُعيدُ ترتيبَ ما تحتاجه من جبرٍ خطيٍّ وتفاضلٍ واحتمالات. ولا يُشترطُ أن تملكَ GPU؛ كلُّ أمثلةِ الكتاب تعملُ على CPU، وإن كانتْ بعضُ التجاربِ ستكونُ أسرعَ كثيرًا على كرتٍ رسومي.

كيف تقرأ هذا الكتاب؟

ثلاثُ طرقٍ، اخترْ منها ما يناسبُك:

1. **الطريقُ المتدرّج:** اقرأْ بالترتيب، ونفّذِ الكودَ في كل فصل قبل الانتقال. لا تقرأْ ثلاثةَ فصولٍ ثم تقول: «فهمتُ». الفهمُ الحقيقيُّ يظهرُ عند أول خطأٍ تُصلحُه بيدك.

2. **الطريقُ العملي:** ابدأْ من الفصل الخامس، ثم عُدْ إلى الأساسيات كلّما احتجت. هذا مسارٌ للمهندسِ الذي يريدُ نتيجةً سريعة، لكنّه لا يُغني عن فهمِ التنسورات والاشتقاق.
3. **الطريقُ المرجعي:** استخدمْ فهرسَ الموضوعات والملاحقَ عند الحاجة. الكتابُ صُمّمَ ليعودَ إليه المحترفُ بعد أن يُنهيَه، لا ليُوضعَ على الرف.

شكر وتقدير

إلى زوجتي «الملكة»، التي تعلّمتُ منها أنّ الإنجازَ ليس لحظةَ وصول، بل عادةُ سيرٍ يوميّة. إلى أبنائي: د/ عماد، د/ عمار، عمرو، عصماء، علا، عزام. إلى والديّ، الذين زرعوا فيّ فضولَ السؤال. إلى أساتذتي، وإلى كلِّ من كتبَ سطرًا تعلّمتُ منه. وإلى مجتمعِ TensorFlow مفتوحِ المصدر، الذي جعلَ المعرفةَ مشاعًا بين الناس.

وأخيرًا إليك، أيّها القارئ. إنّك تحملُ بين يديك دعوةً لا كتابًا: افتحْ محرّرَ الأكواد، وشغّلْ أول سطر، ودعِ الفضولَ يفعلُ الباقي.

م/أحمد صالح المغز

2026

كيف تستخدم هذا الكتاب؟

خريطة الرموز

الرمز	المعنى
💡	**نصيحة عملية** جرّبتُها ونجحت
⚠️	**تحذير:** خطأ شائع قد يُضيّع وقتك
🚫	**خطر:** قد يُفسد النموذج أو البيانات
🧪	**تجربة:** كود قابل للتنفيذ والتعديل
📌	**ملاحظة نظرية** تربط الفكرة بجذورها
🏭	**حالة واقعية** من الصناعة
🎯	**هدف الفصل**
✍️	**تمرين** مع الناتج المتوقع

عادات ثلاث تُغيّر نتيجتك

1. اكتبْ بيدك، لا تنسخْ فقط

قراءةُ الكودِ تُشبهُ مشاهدةَ السباحة من المدرّجات. لن تتعلّمَ السباحةَ حتى تنزلَ الماء. اكتبْ المثالَ سطرًا سطرًا، ثم أغلقِ المصدرَ وأعدْ كتابتَه من الذاكرة. حين تخطئ، ستعرفُ بالضبط أين يسكنُ سوءُ الفهم.

2. شغّلِ التجربةَ الصغيرة أولًا

قبل أن تُدرّبَ على 60 ألف صورة، درّبْ على 600. قبل أن تبني Transformer، ابنِ طبقةَ انتباهٍ واحدة. التجربةُ الصغيرةُ تُعطي إشارةً سريعة، والتجربةُ الكبيرةُ تُعطي إنذارًا متأخرًا.

3. اقرأِ الأخطاءَ كما تقرأُ الدرس

رسالةُ الخطأ ليست عدوًّا؛ إنّها أوّلُ من يُخبرك بالحقيقة. تعلّمْ أن تقرأَ: - نوعُ الخطأ (Shape/Dtype/ResourceExhausted/InvalidArgument). - الشكلُ المتوقّع مقابل الشكلِ الفعلي. - آخرُ طبقةٍ نجحتْ قبل الفشل.

بنية الفصل

كلُّ فصلٍ يتبعُ إيقاعًا ثابتًا:

1. **قصة افتتاحية:** مشكلةٌ واقعية تُشعلُ السؤال.
2. **أهداف الفصل:** ما ستُتقنهُ بنهايته.
3. **الشرح النظري:** الفكرة، ثم المعادلة، ثم الرموز.
4. **التنفيذ:** كودٌ كاملٌ مع تعليقاتٍ عربية.
5. **المقارنة والحالة الواقعية:** متى تختار؟ وماذا يحدث في الصناعة؟
6. **أخطاء شائعة:** العرض، السبب، الحل.
7. **تمارين:** 6-8 تمارين متنوعة.
8. **ملخص:** خلاصةٌ في نقاط.

البيئة والملفات

- كلُّ الأكواد في الكتاب مكتوبةٌ لتعملَ في **دفتر Jupyter** أو سكربت بايثون.
- المسارُ الموصى به: `book/` للمصادر، و`notebooks/` لتجاربك.
- لا تُشغّلْ كلَّ الأمثلةِ في ملفٍ واحد. افصلْ كلَّ تجربةٍ في دفترٍ مستقل، حتى تعرفَ مصدرَ الخطأ.
- احفظْ نسخةً من كلِّ تجربةٍ ناجحة، مع رقم الإصدار والنتيجة. ستحتاجُ إليها لاحقًا.

اصطلاحات الكود

```python
# تعليق عربي يشرح الفكرة، لا يكرّر السطر
import tensorflow as tf
import keras

keras.utils.set_random_seed(42)   # تكرار النتائج قدر الإمكان

# النتيجة المتوقعة تُكتب في تعليق مستقل
print(tf.__version__)             # 2.21.0
```

- **الأسماءُ الإنجليزية** للمتغيّرات والدوال، لأنّها لغةُ المكتبة والمجتمع.
- **التعليقاتُ عربية**، لأنّها مساحةُ الفهم.
- **الأرقامُ صغيرة** في الأمثلة، لتُشغَّلَ في ثوانٍ.
- **لا تُخفِ الخطأ:** إن كان المثالُ يحتاجُ GPU، سيُقالُ ذلك صراحةً.

ماذا تفعل إذا لم يعمل الكود؟

1. تحقّقْ من الإصدارات: `tf.__version__` و`keras.__version__`.
2. أعدْ تشغيلَ الجلسة من الصفر؛ كثيرٌ من الأخطاءِ سببُها حالةٌ قديمة.
3. ابحثْ عن رسالة الخطأ كما هي، ثم اقرأْ أوّلَ إجابةٍ رسمية، لا أوّلَ إجابةٍ عشوائية.
4. بسّطِ المثال: احذفْ كلَّ شيءٍ حتى يبقى الخطأ، ثم أعدْ بناءَه خطوةً خطوة.
5. إن استمرّ الخطأ، فتحقّقْ من الفصل الثاني (دليل البيئة) والملحق (ب).

ما لا يفعله هذا الكتاب

- لن يُعطيكَ «مشروعًا جاهزًا» تنسخُه وتنسى. المشاريعُ في الفصل الأخير هياكلُ كاملة، لكنّك أنت من يملأُ بياناتها ويواجهُ أخطاءها.
- لن يُحوّلكَ إلى باحثٍ في ثلاثة أسابيع. البحثُ يحتاجُ رياضياتٍ وأوراقًا وتجربة. لكنّه سيُعطيكَ أرضيةً صلبةً تنطلقُ منها.
- لن يُخفيَ عنك التعقيد. التعقيدُ جزءٌ من الحقيقة، ومن يُبسّطُ أكثر من اللازم يُنتجُ ثقةً زائفة.

خطتك الأولى: 30 يومًا

الأسبوع	الهدف	المخرَج
1	الفصول 1-5	بيئة تعمل + أول نموذج Keras
2	الفصول 6-8	شبكة من الصفر + تقييم صحيح
3	الفصول 9-10	خط بيانات + CNN على صور حقيقية
4	الفصل 11 + مشروع صغير	نقل تعلّم + تقرير تجربة

بعد الشهر الأول، ستكونُ قد كسرتَ الحاجزَ النفسي. الباقي رحلةُ عمقٍ وتخصص.

خريطة الطريق: من المبتدئ إلى المحترف

المستويات الخمسة

المستوى 0 — الاستكشاف (أسبوع)

- ما الذكاء الاصطناعي؟ ما الفرق بينه وبين تعلّم الآلة والتعلّم العميق؟
- تثبيت البيئة، وتشغيل أول نموذج.
- قراءة: الفصلان 1-2.

المستوى 1 — الأساسيات (3–4 أسابيع)

- التنسورات والأشكال والبث.
- الاشتقاق التلقائي والرسوم البيانية.
- Keras ودورة التدريب.
- قراءة: الفصول 3-5، مع تنفيذ كل تمرين.

المستوى 2 — بناء النماذج (6–8 أسابيع)

- الشبكات العصبية والانتشار الخلفي.
- المحسّنات والتنظيم والتقييم.
- خطوط البيانات.
- الرؤية الحاسوبية.
- قراءة: الفصول 6-10.

المستوى 3 — التخصص (8–10 أسابيع)

اختَرْ مسارًا واحدًا أولًا، ثم عُدْ للثاني: - **الرؤية:** 10-11، 19، 23. - **اللغة والتسلسلات:** 12-14، 19. - **التوليد:** 15-16، 19. - **الإنتاج:** 17-20، 24.

المستوى 4 — الاحتراف (مستمر)

- الأداء والتدريب الموزّع (18).
- MLOps والحوكمة (20، 23).
- التعلم المعزّز (21).
- البيانات الجدولية والرسوم (22).
- مشاريع كاملة ومعرض أعمال (24).

مسارات جاهزة

الهدف	الفصول بالترتيب
مهندس رؤية حاسوبية	1-11، 17-19، 23-24
مهندس NLP	1-9، 12-14، 17-19، 24
مهندس نماذج توليدية	1-9، 13-16، 18-19، 24
مهندس MLOps	1-9، 17-20، 23-24
باحث تطبيقي	1-18، 21-24
مدرّس/مدرب	الكتاب كاملًا + الملاحق + التمارين

قواعد التقدّم

1. **لا تنتقلْ من فصلٍ قبل أن تُنجزَ 70% من تمارينه.** التمرينُ هو المكانُ الذي يتحوّلُ فيه الشرحُ إلى مهارة.
2. **مشروعٌ صغيرٌ بعد كلِّ جزء.** بعد الجزء الأول: نموذجُ تصنيفٍ بسيط. بعد الثاني: CNN على بياناتك. بعد الثالث: تطبيقٌ لغويٌّ أو توليدي. بعد الرابع: خدمةٌ منشورة.
3. **اكتبْ ما تعلّمتَه.** مدوّنةٌ أو دفترُ ملاحظاتٍ أو شرحٌ لصديق. التعليمُ أفضلُ مراجعة.
4. **قِسْ، لا تُخمّنْ.** احتفظْ بجدولٍ للتجارب: التاريخ، الفكرة، التغيير، النتيجة، الوقت.
5. **اعتنِ بصحتك.** الجلوسُ الطويلُ أمام الشاشة ليس بطولة. النومُ والحركةُ جزءٌ من جودة التفكير.

ماذا بعد الكتاب؟

- اقرأْ ورقةً بحثيةً واحدةً أسبوعيًا، وأعدْ تنفيذَ فكرتها الأساسية.
- شاركْ في مسابقةٍ أو مشروعٍ مفتوح المصدر.
- اكتبْ نموذجًا صغيرًا بلغةٍ أخرى (PyTorch/JAX) لفهمِ الحدودِ بين الأدوات.
- تعلّمِ الرياضياتَ التي تحتاجها عند الحاجة، لا كلَّ الرياضيات قبل البدء.
- ابنِ معرضًا من 3 مشاريع عميقة، لا 20 مشروعًا سطحيًا.

اصطلاحات رياضية

الرمز	المعنى	ملاحظة
x	مُدخل (عدد أو متجه أو تنسور)	قد يكون صورة أو كلمة مُرمَّزة
y	الهدف/التسمية الحقيقية	في التصنيف: فئة
$\hat{y}$	التنبؤ	ناتج النموذج
W	مصفوفة الأوزان	قابلة للتعلّم
b	التحييز (bias)	قابل للتعلّم
z	المُدخل الخطّي للطبقة	$z = Wx + b$
a	التنشيط	$a = f(z)$
f	دالة التنشيط	ReLU، sigmoid، …
L أو $\mathcal{L}$	دالة الخسارة	كلّما صغرتْ كان أفضل
θ	وسائط النموذج	الأوزان والتحييزات
η	معدّل التعلّم	خطوة التحديث
$\nabla_\theta L$	تدرّج الخسارة	اتجاه أسرع زيادة
$\odot$	ضرب عنصري	Hadamard product
$\lVert W \rVert_2^2$	مربع معيار L2	مجموع مربعات العناصر
$\mathbb{E}$	التوقّع الرياضي	المتوسط النظري
σ	دالة sigmoid	$\frac{1}{1+e^{-z}}$
$\mathrm{softmax}$	توزيع احتمالي	مجموع المخرجات = 1
N	حجم الدفعة/العيّنة	batch size
D	بُعد الميزات	features
T	طول التسلسل	خطوات زمنية

الرمز	المعنى	ملاحظة
α	معدّل/وزن	حسب السياق
λ	قوة التنظيم	في L2
ϵ	قيمة صغيرة	لتفادي القسمة على صفر
μ, σ^2	المتوسط والتباين	في التطبيع

اصطلاحات الأشكال

الترميز	المعنى
`(N,)`	متجه بطول N
`(N, D)`	مصفوفة: N عيّنة، D ميزة
`(N, H, W, C)`	دفعة صور: ارتفاع، عرض، قنوات
`(N, T, D)`	دفعة تسلسلات: طول، بُعد
`None`	بُعد حرّ (غالبًا حجم الدفعة)
`...`	أبعاد محذوفة للاختصار

ملاحظة على الترقيم

- الفصول مرقّمة 1-24، والأقسام $N.1, N.2, \ldots$
- الأشكال مرقّمة $N.M$ حيث N رقم الفصل.
- المعادلات مرقّمة عند الحاجة فقط، حفاظًا على سلاسة القراءة.
- التمارين مرقّمة داخل كل فصل، والحلول المختارة في الملحق (هـ).

خريطة الكتاب

فهرس المحتويات

أربعة أجزاء · أربعة وعشرون فصلًا · سبعة ملاحق

مقدّمات الكتاب

تمهيد

إهداء iv
مقدمة المؤلف iv
كيف تستخدم هذا الكتاب؟ vii
خريطة الطريق xi
اصطلاحات رياضية xiv

الجزء الأول · الأساسات: كيف تفكّر في TensorFlow؟

1 TensorFlow: الحكاية والمنظومة والمكانة 2
1.1 أهداف الفصل 2
1.2 ما الذكاء الاصطناعي؟ ثلاث دوائر متداخلة 2
1.3 من DistBelief إلى TensorFlow 2.21: لماذا كل هذه الأسماء؟ 3
1.4 المنظومة: سبعة أجزاء تعمل معًا 4
1.5 لماذا TensorFlow؟ خمسة أسباب بلا مبالغة 5
1.6 TensorFlow أم PyTorch أم JAX؟ 6
1.7 أول نموذج: خطٌّ مستقيم يتعلّم 7
1.8 متى لا تختار TensorFlow؟ 9
1.9 حالة واقعية: عندما أنقذت LiteRT المشروع 9
1.10 أخطاء شائعة 10
1.11 تمارين 10
1.12 ملخص 11
2 بيئة 2026: Python 3.14 وTensorFlow 2.21 12
2.1 أهداف الفصل 12

2.2 التوافق يسبق أمر التثبيت 12
2.3 أنشئ بيئة واعرف ما ثبّتّ 13
2.4 CPU وGPU: افصل بين التعريف والمكتبة 14
2.5 تشخيص الأعطال 15
2.6 حالة واقعية: تجهيز فريق قبل بدء التدريب 15
2.7 مصفوفة التثبيت العملية 15
2.8 ملف اعتماد قابل لإعادة الإنتاج 16
2.9 فحص GPU بطريقة منصفة 17
2.10 التخطيط بين مبتدئ وباحث وفريق إنتاج 17
2.11 إدارة البيئة عبر عدة مشاريع 17
2.12 التثبيت ليس نهاية التحقق 18
2.13 أمثلة تنفيذية إضافية 18
2.14 أخطاء شائعة 19
2.15 تمارين 19
2.16 ملخص 20
3 التنسورات: التفكير بالأبعاد 21
3.1 أهداف الفصل 21
3.2 أربعة أوصاف للتنسور 21
3.3 اقرأ شكل الصورة قبل أن تدخل النموذج 23
3.4 إعادة تشكيل دون تغيير الترتيب 25
3.5 البث: محاذاة المحاور من اليمين 25
3.6 قيم ثابتة وأوزان قابلة للتعلم 26
3.7 حالة واقعية: دفعة صور لا تدخل النموذج 26
3.8 فحص الأشكال كعقد بين مكونات النظام 26
3.9 النوع العددي والدقة والذاكرة 27
3.10 قناع الحشو في التسلسلات 27
3.11 أدوات تشخيص سريعة 28
3.12 البث بالتفصيل: محور مفقود لا يساوي قيمة مفقودة 28
3.13 المتغيرات وسجل الحالة 28
3.14 الجهاز والنقل 29

3.15 اختبار خصائص الأشكال 29
3.16 مثال حساب يدوي للبث 29
3.17 أمثلة تنفيذية إضافية 30
3.18 توثيق عقد البيانات 30
3.19 فهرسة آمنة للأمثلة 30
3.20 سجلات المتغيرات 31
3.21 توحيد أنواع البيانات 31
3.22 أخطاء شائعة 31
3.23 تمارين 31
3.24 ملخص 32
4 الاشتقاق التلقائي والرسوم البيانية 33
4.1 أهداف الفصل 33
4.2 المشتقة: لغة التغيير 33
4.3 الشريط: كيف يتذكّر TensorFlow؟ 35
4.4 من الاشتقاق إلى التدريب 37
4.5 tf.function: الرسم البياني الذي يُبنى مرة واحدة 40
4.6 XLA: حين يُعاد تشكيل الرسم 42
4.7 حالة واقعية: ثماني دقائق بدل دقيقة 43
4.8 أخطاء شائعة 44
4.9 تمارين 44
4.10 ملخص 45
5 Keras: من الفكرة إلى النموذج 46
5.1 أهداف الفصل 46
5.2 أي بنية تناسب الحساب؟ 46
5.3 الخسارة والمقياس يجيبان عن سؤالين مختلفين 48
5.4 دورة الحفظ والنشر 49
5.5 حالة واقعية: الطلبات بفرعين 49
5.6 خط أساس كامل من البيانات إلى القياس 49
5.7 مدخلان ومخرجان في Functional API 50
5.8 الحفظ الآمن ودورة النشر 50

5.9 عقود التدريب: خسارة، مقياس، ووزن عينة 51
5.10 بناء input pipeline مع Keras 51
5.11 اكشف انهيار التعلم بالتشخيص 51
5.12 قابلية إعادة الإنتاج 52
5.13 Model subclassing ومتى تحتاجه 52
5.14 فصل البحث عن التقييم النهائي 52
5.15 Keras callbacks تعاقد وليست سحرًا 52
5.16 فحص قبل الإنتاج 53
5.17 دورة تعلم بحسب المستوى 53
5.18 أمثلة تنفيذية إضافية 53
5.19 مواصفات التشغيل والتسليم 53
5.20 تحقق من التسمية والتوزيع 54
5.21 استدلال بدفعات صغيرة 54
5.22 اختيار metric مرتبط بالتكلفة 54
5.23 احتفظ بعقد الخدمة مع النموذج 54
5.24 أخطاء شائعة 54
5.25 تمارين 55
5.26 ملخص 55

الجزء الثاني · بناء الشبكات وتدريبها

6 الشبكات العصبية من الصفر 57
6.1 أهداف الفصل 57
6.2 من جمع موزون إلى قرار 57
6.3 الانتشار الخلفي قاعدة سلسلة منظمة 59
6.4 التهيئة والاستقرار 60
6.5 حالة واقعية: مصنف القطع 60
6.6 اشتقاق رجوع الخطأ من طبقة واحدة 60
6.7 التنشيط مشتقةً وسلوكًا 61
6.8 التهيئة وحجم الطبقة 61
6.9 نموذج رقمي كامل للمبتدئ 62

6.10 المقارنة بين التنشيطات على رسم واحد 62
6.11 من المشتقة إلى تحديث الدفعة 63
6.12 كيف تقرأ model.summary؟ 63
6.13 حدود نظرية التقريب الشامل 63
6.14 تشخيص كل طبقة على حدة 63
6.15 منطق اختيار عدد الطبقات 64
6.16 مثال انحدار متعدد الميزات 64
6.17 ماذا تعرض في التقرير؟ 64
6.18 أمثلة تنفيذية إضافية 64
6.19 أخطاء شائعة 65
6.20 تمارين 65
6.21 ملخص 66
7 المحسّنات والتنظيم: فنّ التدريب 67
7.1 أهداف الفصل 67
7.2 المحسّن يحدد طريقة الحركة 67
7.3 التنظيم يحدّ من اعتماد النموذج على تفاصيل العينة 69
7.4 معدل التعلم والقص والدقة المختلطة 69
7.5 حالة واقعية: تعميم نموذج المستشفى 70
7.6 مسارات ضبط منهجية 70
7.7 تطبيع داخل الشبكة لا يطبع البيانات الأصلية 70
7.8 وصفة تشخيص عملية لفرط التعلّم 71
7.9 الدقة المختلطة والاستقرار العددي 71
7.10 جداول معدل التعلم 71
7.11 BatchNorm وDropout في inference 72
7.12 تنظيم يعكس فرضية 72
7.13 خطة اختبار لعدم الاستقرار 72
7.14 سجل المقارنة 72
7.15 فرط التعلّم تحت المجهر 73
7.16 وزن الفئات والـ focal loss 73
7.17 Early stopping ومشكلة epoch المختارة 73

7.18 تنظيم البيانات أهم من تنظيم الوزن 73
7.19 جدول قرار للتقنيات 74
7.20 أمثلة تنفيذية إضافية 74
7.21 أخطاء شائعة 74
7.22 تمارين 75
7.23 ملخص 75
8 التقييم والضبط: كيف تعرف أن نموذجك جيد؟ 76
8.1 أهداف الفصل 76
8.2 ثلاثة صناديق: لماذا لا تكفي مجموعة واحدة؟ 76
8.3 التحقق المتقاطع: أن تختبر أكثر من مرة 77
8.4 مصفوفة الالتباس: تشريح الخطأ 77
8.5 المعايرة: هل الثقة تعني الاحتمال؟ 79
8.6 عندما تكون الأقلية هي المهمة 80
8.7 الضبط والتتبع وتحليل الأخطاء 81
8.8 حالة واقعية: حين صار قرار المراجعة أهم من AUC 82
8.9 أمثلة تنفيذية إضافية 82
8.10 أخطاء شائعة 83
8.11 تمارين 84
8.12 ملخص 84
9 tf.data: هندسة خطوط البيانات 85
9.1 أهداف الفصل 85
9.2 Dataset سلسلة تحويلات 85
9.3 ترتيب التحويلات قرار ذاكرة وزمن 87
9.4 الملفات الكبيرة وTFRecord 88
9.5 حالة واقعية: صور خط إنتاج 88
9.6 من ملف صورة إلى دفعة مدربة 88
9.7 متى تختار TFRecord؟ 89
9.8 الحتمية والتوزيع 89
9.9 تشخيص اختناق البيانات 89
9.10 الثبات وإعادة التشغيل 90

9.11 schemag TFRecord 90
9.12 زيادة البيانات على مستوى batch 90
9.13 مراجعة قبل التدريب 90
9.14 نمط Dataset J reproducible 91
9.15 مسألة cache عمليًا 91
9.16 معايير قبول خط البيانات 91
9.17 هندسة CSV والنصوص 91
9.18 البيانات التي تتغير عند القراءة 92
9.19 أمثلة تنفيذية إضافية 92
9.20 فحوص قبول قبل حساب مكلف 92
9.21 حجم الملفات والمشاركة 93
9.22 فحص label mapping 93
9.23 أخطاء شائعة 93
9.24 تمارين 93
9.25 ملخص 94
10 الرؤية الحاسوبية: CNN من الالتفاف إلى الإنتاج 95
10.1 أهداف الفصل 95
10.2 مرشح محلي يعاد استخدامه 95
10.3 من LeNet إلى ConvNeXt 97
10.4 تفسير بصري لا يثبت السببية 98
10.5 حالة واقعية: كشف خدش 98
10.6 مجال الرؤية وحساب الكلفة 98
10.7 زيادة البيانات يحافظ على معنى الملصق 98
10.8 النقل من نموذج مدرب مسبقًا 98
10.9 تفسير Grad-CAM ومسؤولية التحقق 99
10.10 من التدريب إلى القياس التشغيلي 99
10.11 وصفة تدريب قابلة للتكرار 99
10.12 لماذا يختلف أداء CNN بين المجالين؟ 99
10.13 ضوابط BatchNorm ودفعة صغيرة 100
10.14 تهيئة صور الخدمة 100

10.15 Grad-CAM: تطبيق وتفسير حذر 100
10.16 تصميم مشروع تصنيف صور كامل 100
10.17 الموازنة بين الدقة والكمون 101
10.18 أخطاء من الواقع 101
10.19 المراجعة العلمية 101
10.20 استكشاف feature maps 101
10.21 التهيئة المسبقة والملصقات الصاخبة 101
10.22 متطلبات اكتساب البيانات 102
10.23 أمثلة تنفيذية إضافية 102
10.24 أخطاء شائعة 102
10.25 تمارين 103
10.26 ملخص 103

الجزء الثالث · الرؤية والتسلسلات واللغة والتوليد

11 الرؤية المتقدمة: النقل والكشف والتجزئة 105
11.1 أهداف الفصل 105
11.2 التعلم بالنقل مرحلتان 105
11.3 الكشف يحدد أين وماذا 107
11.4 U-Net للتجزئة 108
11.5 حالة واقعية: صور من جهاز جديد 108
11.6 الكشف: من الصندوق إلى قرار التشغيل 108
11.7 التجزئة: قناع صحيح مكانيًا 108
11.8 التنبؤ بوجود هدف نادر 109
11.9 TTA وموازنة الكمون 109
11.10 تقرير مشروع قابل للمراجعة 109
11.11 Transfer learning بميزانية بيانات صغيرة 109
11.12 مكونات كاشف الأجسام 110
11.13 أقنعة segmentation ومعالجة الحدود 110
11.14 تقييم مقاوم لتغير المجال 110
11.15 annotator agreement وإدارة الأقنعة 110

11.16 كشف الأجسام الصغيرة وتعدد المقاييس 111
11.17 تكلفة البيانات وتحديد الأولوية 111
11.18 عتبات تشغيل حسب المخاطر 111
11.19 وثائق المشروع والقيود 111
11.20 حين لا توجد صناديق لكل البيانات 111
11.21 القياس الميداني وإدارة الإصدارات 112
11.22 إمكانية إعادة إنتاج المشروع 112
11.23 أمثلة تنفيذية إضافية 112
11.24 أخطاء شائعة 113
11.25 تمارين 113
11.26 ملخص 113
12 التسلسلات: RNN/LSTM/GRU والسلاسل الزمنية 114
12.1 أهداف الفصل 114
12.2 ترتيب الزمن جزء من البيانات 114
12.3 لماذا LSTM وGRU؟ 116
12.4 حالة واقعية: تخطيط مخزون 117
12.5 النافذة والهدف وخط الأساس 117
12.6 LSTM بواباتٌ وحالةٌ 117
12.7 تقييم زمني واقعي 117
12.8 بيانات متغيرة وحالات مفقودة 118
12.9 Stationarity والاتجاه والموسمية 118
12.10 التسرب الزمني وfeature availability 118
12.11 اختيار نافذة الإدخال 118
12.12 teacher forcing والتنبؤ المتكرر 119
12.13 تفسير نموذج زمني 119
12.14 تدريب LSTM عمليًا 119
12.15 سلاسل متعددة ومتغيرات خارجية 119
12.16 مقاييس الانحدار الزمني 120
12.17 اختبار انزياح الزمن 120
12.18 عدم اليقين والتخطيط 120

12.19 اختيار RNN أو Conv1D أو Transformer 120
12.20 تقرير متانة زمني 121
12.21 أمثلة تنفيذية إضافية 121
12.22 توقع احتمالي واحتياط المخزون 121
12.23 أخطاء شائعة 121
12.24 تمارين 122
12.25 ملخص 122
13 الانتباه والمحوّلات 123
13.1 أهداف الفصل 123
13.2 من السياق المضغوط إلى أوزان الانتباه 123
13.3 لماذا رؤوس متعددة ومعلومة موضع؟ 125
13.4 الكلفة والبدائل 125
13.5 حالة واقعية: تصنيف تذاكر الدعم 126
13.6 بناء الكتلة خطوةً خطوة 126
13.7 موضعية مطلقة ونسبية 126
13.8 طول السياق والذاكرة 127
13.9 اختيار بنية المحول 127
13.10 أساليب تدريب ومعايرة 127
13.11 ضبط شكل الاستعلام والمفتاح والقيمة 127
13.12 قناع padding في دفعة لغوية 128
13.13 لماذا انتشر Flash Attention؟ 128
13.14 التحليل الحسابي والذاكرة 128
13.15 فحوص قبل fine-tuning 128
13.16 تصميم pretraining وfine-tuning 129
13.17 التعامل مع طول السياق 129
13.18 التحسين والتدرج 129
13.19 المعايرة والتقييم 129
13.20 إدارة checkpoints 130
13.21 تجربة استئصال (ablation) سليمة 130
13.22 Transformer للتصنيف على بيانات صغيرة 130

13.23 حدود الانتباه كشرح 130
13.24 أمثلة تنفيذية إضافية 130
13.25 أخطاء شائعة 131
13.26 تمارين 131
13.27 ملخص 132
14 معالجة اللغة الطبيعية والعربية 133
14.1 أهداف الفصل 133
14.2 من الحرف إلى الرمز: لماذا لا يرى النموذج نصك؟ 133
14.3 التضمين: حين تصبح الكلمة متجهًا 136
14.4 BERT: قراءة الجملة في اتجاهين 138
14.5 العربية: لغةٌ تتحدى المعجم 140
14.6 من النص إلى نموذج: تصنيف المشاعر خطوة بخطوة 141
14.7 التقييم: ما الذي نقيسه فعلًا؟ 144
14.8 أخطاء شائعة 145
14.9 تمارين 146
14.10 ملخص 147
15 التوليد: Autoencoder وVAE وGAN 148
15.1 أهداف الفصل 148
15.2 Autoencoder يتعلم إعادة البناء 148
15.3 VAE يعلّم فضاءً احتماليًا 148
15.4 GAN لعبة خصومية لا مقياس جودة وحيد 149
15.5 تقييم العينات 150
15.6 حالة واقعية: عيوب صناعية نادرة 151
15.7 أمثلة تنفيذية إضافية 151
15.8 ورشة عملية: ما الذي نراقبه أثناء التدريب؟ 151
15.9 بناء تجربة توليد يمكن للآخرين مراجعتها 152
15.10 بروتوكول عملي على صور صغيرة 153
15.11 اختيار latent dimension وتفسيره 153
15.12 الاستخدام البحثي والأخلاقي 153
15.13 دراسات ablation ومقارنة عادلة 153

15.14 التوليد المشروط بالمعلومات 154
15.15 اختيار مقياس الصورة 154
15.16 صلاحية استخدام البيانات 154
15.17 تصميم دراسة ومقارنة عادلة 154
15.18 سياسة الاستخدام 155
15.19 نشر النتائج 155
15.20 أخطاء شائعة 155
15.21 تمارين 156
15.22 ملخص 156
16 نماذج الانتشار: التوليد الحديث 157
16.1 أهداف الفصل 157
16.2 إضافة ضوضاء وفق جدول 157
16.3 إزالة الضوضاء مشروطة بالزمن 157
16.4 التوجيه والتكلفة 158
16.5 حالة واقعية: صور قطع تحت إضاءة مختلفة 159
16.6 أمثلة تنفيذية إضافية 159
16.7 جدول الضوضاء والتدريب 159
16.8 U-Net مشروطة بالزمن 160
16.9 أخذ عينات DDPM وDDIM 160
16.10 تقييم ومخاطر بيانات التوليد 160
16.11 اختبار forward process قبل التدريب 160
16.12 أخذ عينات مرئي ومقاييس هندسية 161
16.13 اختيار دقة التدريب 161
16.14 تلخيص دراسة نموذج 161
16.15 اشتقاق معاملات الجدول 161
16.16 تشخيص artifacts 162
16.17 التقطير والتسريع 162
16.18 تسجيل نموذج بحثي 162
16.19 حدود التطبيق 162
16.20 مقارنة sampler وتقليل الكلفة 163

16.21 الأثر البحثي والبيئي 163
16.22 اختيار تطبيق مسؤول 163
16.23 التحقق من شرط التوليد 163
16.24 استقرار التحديث 164
16.25 أخطاء شائعة 164
16.26 تمارين 164
16.27 ملخص 165

الجزء الرابع · الاحتراف والإنتاج

17 الطبقات والنماذج المخصصة والتدريب المتقدم 167
17.1 أهداف الفصل 167
17.2 وزن ينشأ مرة واحدة 167
17.3 خسائر ومقاييس وسلوك تدريب خاص 168
17.4 حالة واقعية: خسارة موزونة لعدة مخرجات 169
17.5 أمثلة تنفيذية إضافية 169
17.6 تعديل trainstep دون كسر Keras 170
17.7 الأقنعة والحالة في call 171
17.8 التهيئة والتسلسل والتحميل 171
17.9 عزل أخطاء التدرج 172
17.10 مقاييس مخصصة ذات حالة 172
17.11 addloss وقيود الطبقة 172
17.12 دعم أكثر من مخرج وsample weights 172
17.13 tf.function والتوافق مع التصدير 172
17.14 خطة اختبار طبقة احترافية 173
17.15 طبقة Attention مصغرة 173
17.16 حساب الخسارة لعدة أهداف 173
17.17 توزيع الطبقة المخصصة 173
17.18 التوافق بين backends 174
17.19 فشل مختبر التدريب المخصص 174
17.20 دورة تدريب مخصصة عند الضرورة 174

17.21 اختبار الطبقة عبر الأشكال والأنواع 174
17.22 دعم backend ومراجعة الشفرة 174
17.23 واجهة عامة وصيانة طويلة الأجل 175
17.24 قراءة سجل التدرجات 175
17.25 دورة مراجعة على دفعة واحدة 175
17.26 أخطاء شائعة 175
17.27 تمارين 175
17.28 ملخص 176
18 الأداء والتدريب الموزّع 177
18.1 أهداف الفصل 177
18.2 قِس خط أساس 177
18.3 XLA والدقة المختلطة 179
18.4 التوزيع وتراكم التدرجات 179
18.5 حالة واقعية: قياس خدمة تدريب 180
18.6 أمثلة تنفيذية إضافية 180
18.7 استراتيجيات التوزيع والتكلفة 181
18.8 تراكم التدرج 181
18.9 اقرأ trace لا رقم واحد 181
18.10 الدقة المختلطة وحدود الأعداد 181
18.11 تقرير أداء قابل للتكرار 182
18.12 مختبر profiler خطوة بخطوة 182
18.13 تقدير الذاكرة 182
18.14 استراتيجية اختيار الدقة 182
18.15 الاتصالات في التدريب الموزع 183
18.16 حوكمة القياس 183
18.17 Mixed precision حسب backend 183
18.18 توازي البيانات والنموذج 183
18.19 مراجعة الأشكال الديناميكية 183
18.20 جمع الدرجات والتحقق من scale 184
18.21 لوحة أداء للفريق 184

18.22 XLA وكلفة التجميع 184
18.23 الذاكرة وحجم الدفعة 184
18.24 اختبار تراكم التدرج 184
18.25 استراتيجية نمو 185
18.26 أخطاء شائعة 185
18.27 تمارين 185
18.28 ملخص 186
19 النشر: TF Servingg TF.jsg LiteRT 187
19.1 أهداف الفصل 187
19.2 صيغة الاستدلال وعقده 187
19.3 LiteRT على الأجهزة الطرفية 188
19.4 المتصفح والخدمات 189
19.5 حالة واقعية: فحص عيب دون إنترنت 189
19.6 أمثلة تنفيذية إضافية 189
19.7 عقد الإدخال وإصدارات النموذج 190
19.8 معايرة التكميم وقرار القبول 190
19.9 تشغيل الخدمة وطرح تدريجي 191
19.10 أمان سلسلة النشر 191
19.11 تشغيل TensorFlow Serving 191
19.12 تطبيق طرفي: مسار اختبار كامل 191
19.13 المراقبة بعد النشر 192
19.14 خطة rollback جاهزة 192
19.15 مثال استدلال يطابق العقد 192
19.16 TFLite/LiteRT signatures 193
19.17 المتصفح وخصوصية المستخدم 193
19.18 اختبار الطرف إلى الطرف 193
19.19 خدمة API قابلة للتوسع 194
19.20 اختبار الحمل والفشل 194
19.21 تقييم الطرف إلى الطرف 194
19.22 مصفوفة توافق الإصدار 194

19.23 التوثيق للمستخدم النهائي 194
19.24 أخطاء شائعة 195
19.25 تمارين 195
19.26 ملخص 195
20 MLOps: TFX والمراقبة 196
20.1 أهداف الفصل 196
20.2 خط إنتاج له نسب بيانات وإصدارات 196
20.3 المراقبة ترى التغير لا الحقيقة وحدها 197
20.4 دورة إصدار قابلة للرجوع 198
20.5 حالة واقعية: تغير سلوك العملاء 198
20.6 أمثلة تنفيذية إضافية 199
20.7 خطوط التشغيل ليست مجرد رسم مربعات 199
20.8 مراقبة متعددة الطبقات 199
20.9 دورة إصدار تحمي من التحسن الزائف 200
20.10 إعادة التدريب والاستدامة 200
20.11 نموذج أولي وتدرج الاحتراف 200
20.12 قائمة قبول لنموذج جديد 201
20.13 تتبع التجارب والآثار 201
20.14 تصميم مراقبة الانجراف 201
20.15 مسؤوليات الفريق والاستدامة 202
20.16 مؤشرات تشغيلية وبوابات قرار 202
20.17 فشل pipeline والتعافي 202
20.18 اتفاقية تشغيل بين الفرق 202
20.19 أمان البيانات والنماذج 202
20.20 مستوى نضج تدريجي 203
20.21 حادث انجراف: سيناريو استجابة 203
20.22 تخزين النسب والاحتفاظ 203
20.23 بوابة قبول مرشحة 203
20.24 أخطاء شائعة 204
20.25 تمارين 204

20.26 ملخص 204
21 التعلم المعزز 205
21.1 أهداف الفصل 205
21.2 عملية قرار ماركوف 205
21.3 من Q-learning إلى DQN 206
21.4 تجربة آمنة في CartPole 207
21.5 حالة واقعية: روبوت المستودع 207
21.6 أمثلة تنفيذية إضافية 207
21.7 صياغة Bellmang MDP 208
21.8 Q-learning وDQN عمليًا 208
21.9 REINFORCE وPPO 209
21.10 تقييم قبل الواقع 209
21.11 متى لا تستخدم RL؟ 209
21.12 درس CartPole كتجربة علمية 209
21.13 ضبط المكافأة دون خلق تحايل 209
21.14 Replay وحالات نهاية الحلقة 210
21.15 قياس المتانة وقرار الانتقال للواقع 210
21.16 بيانات بشرية كخط أساس 210
21.17 انحياز وتغطية replay 210
21.18 وثائق بيئة قابلة للتكرار 211
21.19 فحص سياسة قبل نشرها 211
21.20 Offline RL والحذر من البيانات 211
21.21 خريطة تنقل للمبتدئ والباحث 211
21.22 مؤشرات فشل التدريب 211
21.23 التعلم من بيانات ثابتة 212
21.24 أخطاء شائعة 212
21.25 تمارين 212
21.26 ملخص 213
22 البيانات الجدولية والتوصيات والرسوم البيانية 214
22.1 أهداف الفصل 214

22.2 اعرف العمود قبل ترميزه 214
22.3 توصية على مرحلتين 215
22.4 الرسوم ومرور الرسائل 216
22.5 حالة واقعية: اقتراح المنتج التالي 216
22.6 أمثلة تنفيذية إضافية 216
22.7 معالجة الجدول ضمن النموذج 217
22.8 ترتيب التوصية من الطرف إلى الطرف 217
22.9 الرسم البياني وتمرير الرسائل 217
22.10 التعميم على فئات وعناصر جديدة 218
22.11 شجرة قرار مقابل شبكة عميقة 218
22.12 منع تسرب السمات 218
22.13 تجربة توصية من التفاعل إلى العرض 218
22.14 بناء GNN قابلة للتوسع 219
22.15 تغطية العناصر الجديدة والنتائج 219
22.16 خدمة الميزات والفهارس 219
22.17 مراقبة عرض التوصية 219
22.18 واجهة fallback واضحة 219
22.19 Feature store ومزامنة الزمن 220
22.20 حذف المستخدم والخصوصية 220
22.21 سجلات الدلالة وجودة الوسم 220
22.22 تحديث نظام التوصية 220
22.23 الاختبار بحسب السيناريو 220
22.24 أخطاء شائعة 221
22.25 تمارين 221
22.26 ملخص 222
23 الأخلاق والخصوصية والتفسير والأمان 223
23.1 أهداف الفصل 223
23.2 العدالة سؤال عن القرار والسياق 223
23.3 التفسير أداة فحص 224
23.4 أمان الخصومة والحوكمة 225

23.5 حالة واقعية: نموذج توظيف 225
23.6 أمثلة تنفيذية إضافية 225
23.7 تقييم عدالة القرار خطوةً خطوة 226
23.8 الخصوصية التفاضلية والتعلم الاتحادي 226
23.9 التفسير والتحقق من متانته 226
23.10 التهديدات: FGSM والتسميم وإساءة الاستعمال 227
23.11 بطاقة نموذج ومراجعة إطلاق 227
23.12 دراسة تدقيق: نموذج توظيف 227
23.13 الخصوصية في جمع البيانات والتسجيل 227
23.14 شرح النموذج وتدخل المشغل 228
23.15 نموذج تهديد مختصر 228
23.16 دورة الحوكمة بعد الإطلاق 228
23.17 مراجعة إطلاق موثقة 228
23.18 إدارة حادث أو اعتراض 229
23.19 مراجعة مستقلة للتغييرات الكبيرة 229
23.20 مقاييس متانة وخصوصية مفيدة 229
23.21 التوثيق بلغة المستخدم 229
23.22 مصفوفة مخاطر قبل الجمع والنشر 229
23.23 تصميم تفسير يتجنب الإفراط في الثقة 230
23.24 موازنة الخصوصية والفائدة 230
23.25 أخطاء شائعة 230
23.26 تمارين 231
23.27 ملخص 231
24 مشاريع ختامية وخارطة الطريق 232
24.1 أهداف الفصل 232
24.2 ما الذي يفرّق المشروع الختامي عن دفتر تجارب؟ 232
24.3 تصنيف الصور: حين تكون الدقة أسهل ما في المشروع 234
24.4 تنبؤ الطلب: الزمن لا يُخلط 236
24.5 تحليل المشاعر العربي: اللغة أصعب من النموذج 238
24.6 كشف الاحتيال: حين يكون الخطأ مكلفًا 240

24.7 ست حالات واقعية تكمل الأمثلة الأربعة 242
24.8 خارطة الطريق ومعرض الأعمال 247
24.9 أخطاء شائعة 249
24.10 تمارين 250
24.11 ملخص 251

الملاحق

مراجع وأدوات

A الرياضيات العملية 252
A.1 المتجهات والمصفوفات 252
A.2 التفاضل وقاعدة السلسلة 252
A.3 الاحتمال والتوقع 252
A.4 حساب يدوي صغير 253
A.5 أحجام الأشكال الشائعة 253
A.6 قائمة مراجعة 253
B دليل البيئة والأعطال 255
B.1 إعداد نظيف 255
B.2 التحقق 255
B.3 جدول الأعطال 256
B.4 GPU وWSL2 والحاويات 256
B.5 تقرير مشكلة مفيد 256
C المصطلحات: عربي ↔ إنجليزي 258
D مرجع TensorFlow/Keras السريع 269
D.1 التنسورات والتدرج 269
D.2 نموذج Keras 269
D.3 خسائر ومقاييس 270
D.4 خطوط البيانات والنشر 270
D.5 تذكير 271
E حلول التمارين المختارة 272
E.1 حل تمرين الأبعاد 272

E.2 حل تمرين الانحدار 272
E.3 حل تمرين مصفوفة الالتباس 272
E.4 حل تمرين عدم توازن الفئات 272
E.5 حل تمرين تقسيم السلسلة الزمنية 272
E.6 حل تمرين نقل التعلم 273
E.7 حل تمرين النشر الطرفي 273
E.8 حل تمرين المراجعة الأخلاقية 273
F المراجع والمصادر 274
F.1 الوثائق الرسمية 274
F.2 كتب تأسيسية 275
F.3 أوراق محورية 275
F.4 استخدام مجموعات البيانات 276
G فهرس الموضوعات 277

الجزء الأول

الأساسات: كيف تفكّر في TensorFlow؟

من التنسور الأول إلى أول نموذج كامل

01 TensorFlow: الحكاية والمنظومة والمكانة

02 بيئة 2026: Python 3.14 وTensorFlow 2.21

03 التنسورات: التفكير بالأبعاد

04 الاشتقاق التلقائي والرسوم البيانية

05 Keras: من الفكرة إلى النموذج

الفصول 1-5 · 5 فصول

الجزء الأول · الأساسات: كيف تفكّر في TensorFlow؟

الفصل الأول

01

TensorFlow: الحكاية والمنظومة والمكانة

حكاية الفصل

في مستشفى جامعي، أمضى فريقٌ من المهندسين ثلاثة أشهر في بناء نموذجٍ يكتشف الالتهاب الرئوي من صور الأشعة. على دفتر Jupyter، كانت الدقة 94%، وكانت الرسوم البيانية جميلة. ثم جاء يوم التركيب: جهازُ الأشعة لا يملك GPU، وشبكةُ المستشفى معزولة، وحجمُ النموذج أكبر من أن يُحمَّل على الجهاز الطرفي. سألني رئيسُ الفريق بهدوء: «نحن نعرف كيف نُدرّب. لكن هل نعرف كيف نُنشر؟». في تلك اللحظة أدركتُ أنّ TensorFlow ليس مكتبةً لتدريب النماذج فقط؛ إنّه **سلسلةُ أدواتٍ تُوصل الفكرةَ من المختبر إلى سرير المريض**. فما الحكاية؟ وكيف وصلنا إلى هنا؟

1.1 أهداف الفصل

- [ ] التمييز بين الذكاء الاصطناعي وتعلّم الآلة والتعلّم العميق.
- [ ] فهم مكان TensorFlow في المنظومة، وما الذي يميّزه عن PyTorch وJAX.
- [ ] معرفة مكوّنات المنظومة: Keras، tf.data، TFX، LiteRT، TF Serving، TF.js.
- [ ] بناء أول نموذج كامل بـ Keras وفهم دورة حياته.
- [ ] اتخاذ قرارٍ واعٍ: متى تختار TensorFlow، ومتى تختار غيره؟

1.2 ما الذكاء الاصطناعي؟ ثلاث دوائر متداخلة

تخيّل ثلاث دوائر متداخلة. الكبرى اسمها **الذكاء الاصطناعي (AI)**: كلُّ محاولةٍ لجعل الآلة تؤدّي مهمةً نعدّها «ذكية»، من برنامج الشطرنج القديم إلى روبوت المحادثة. داخل هذه الدائرة تأتي **دائرة تعلّم الآلة (ML)**: بدلًا من كتابة القواعد يدويًا، نُعطي الخوارزميةَ أمثلةً تستخرج منها القواعد. ثم تأتي الدائرة الصغرى، **التعلّم العميق (DL)**: تعلّمُ الآلة بشبكاتٍ عصبيةٍ متعدّدة الطبقات، قادرةٍ على اكتشاف أنماطٍ هرمية: الحافة، ثم الشكل، ثم الوجه، ثم الشخص.

لماذا هذا التمييز مهم؟ لأنّ كثيرًا من الأسئلة تختلط: «هل نحتاج TensorFlow؟» الجواب يعتمد على الدائرة التي تقف فيها. إن كانت مشكلتك جدولَ مبيعاتٍ بسيطًا، فقد يكفيك الانحدار الخطي أو شجرة قرار. وإن كانت صورًا أو نصوصًا أو إشاراتٍ معقّدة، فالتعلّم العميق يصبح خيارًا قويًا. **الأداةُ تتبعُ المشكلة، لا العكس.**

> ◆ **ملاحظة**
>
> التعلّم العميق ليس سحرًا. إنّه تقريبُ دوالٍ معقّدة عبر تحسين وسائط كثيرة. كلّما زادت البيانات والوسائط، زادت قدرة التقريب – وزاد خطر الحفظ الأعمى (overfitting).

1.3 من DistBelief إلى TensorFlow 2.21: لماذا كل هذه الأسماء؟

في بداية العقد الثاني من الألفية، كان باحثو Google يبنون شبكاتٍ عصبيةً ضخمة على أنظمةٍ داخلية. كان المشروعُ يُسمّى **DistBelief**، وكانت فكرتُه الأساسية: وزّعِ الحسابَ على آلاف الآلات. لكنّه كان صلبًا، مرتبطًا ببنيةٍ داخلية، ويصعبُ على الخارجيين استخدامُه.

ثم جاءت لحظةٌ فارقة: أراد الفريقُ لغةً واحدةً تصفُ الحسابَ الرياضيَّ كرسمٍ بيانيّ، ثم تُنفّذه على CPU أو GPU أو هاتف. في عام 2015 أُعلن **TensorFlow** مفتوحَ المصدر. كان الاسمُ مستوحى من «تدفّق التنسورات» عبر الرسم البياني. في الإصدار الأول، كانت الكتابةُ أشبهَ ببناء معبد: `graph`، `placeholder`، `Session`. قويةٌ جدًا، لكنّها قاسيةٌ على المبتدئ.

في 2019 جاء **TensorFlow 2.0**، فقلبَ المعادلة: التنفيذُ الفوري (eager execution) صار الافتراضي، وKeras صارت الواجهةَ الرسميةَ العليا، واختفتْ جلساتُ `Session` من الحياة اليومية. صار بإمكانك أن تكتب:

```python
import tensorflow as tf
x = tf.constant([1.0, 2.0, 3.0])
print(x * 2)   # tf.Tensor([2. 4. 6.], shape=(3,), dtype=float32)
```

هذه البساطةُ الظاهرة تخفي تحتها نظامًا عميقًا: يمكنك أن تُغلّفَ الدالةَ بـ `tf.function` فتُحوَّلَ إلى رسمٍ بيانيّ عالي الأداء، وأن تُصدّرَ النموذجَ إلى SavedModel، ثم إلى LiteRT ليعملَ على هاتف. منذ 2020 وحتى **2.21**، تراكمتْ طبقاتُ النضوج: دعمٌ أفضل للمحوّلات، وتكاملٌ أعمق مع Keras 3، وتحسيناتٌ في الأداء الموزّع، وأدواتُ LiteRT وTF Servingg.

الشكل التالي يضع المنظومةَ في صورةٍ واحدة:



شكل 1-1 خريطة منظومة TensorFlow: من البيانات إلى النشر

1.4 المنظومة: سبعة أجزاء تعمل معًا

حين تقول «TensorFlow»، فأنت لا تقصد ملفًا واحدًا. أنت تقصد عائلةً من المشاريع، لكلٍّ منها دور:

المكوّن	الدور	متى تستخدمه؟
TensorFlow Core	التنسورات، العمليات، التدرّج التلقائي، `tf.function`	حين تحتاج تحكّمًا منخفض المستوى
Keras 3	واجهة بناء النماذج والتدريب	في 90% من المشاريع
tf.data	خطوط بيانات عالية الأداء	عند التعامل مع صور/نصوص/ملفات كبيرة
TensorBoard	مراقبة التدريب والرسوم	لرؤية ما يحدث داخل النموذج
TFX	خطوط إنتاج النماذج وMLOps	عند الانتقال إلى الإنتاج
LiteRT (TFLite)	تشغيل النماذج على الأجهزة الطرفية	هاتف، Raspberry Pi، متحكّم دقيق
TF Serving / TF.js	خدمة النموذج على الخادم أو المتصفح	APIs، تطبيقات ويب، نشر واسع

هذه المنظومةُ هي ميزة TensorFlow الكبرى: **يمكنك أن تبدأ صغيرًا وتكبرَ دون أن تغيّر أدواتك**. النموذجُ الذي درّبتَه في دفترٍ يمكن أن يُصدَّرَ ويُخدمَ على خادم، ثم يُصغَّرَ ليعملَ على هاتف. لكنّها أيضًا مصدرُ إرباكٍ للمبتدئ: أسماءٌ كثيرة، وإصداراتٌ متداخلة، ودروسٌ قديمة. لذلك سنُبني الفهمَ طبقةً طبقة، ولن نستخدم أداةً قبل أن نعرفَ لماذا.

> ✦ **نصيحة**
>
> لا تُحاولْ تعلّمَ المنظومة كلها في أسبوع. ابدأ بـ `tf.constant` و `keras.Sequential` و `model.fit`، ثم أضفْ أداةً واحدةً كلّما احتجت. الأدواتُ تُتعلَّمُ بالألم، لا بالحفظ.

1.5 لماذا TensorFlow؟ خمسة أسباب بلا مبالغة

1. **النشرُ من اليوم الأول.** ليست الفكرةُ أن تُدرّبَ فقط، بل أن تُخرجَ نموذجًا يعملُ في بيئةٍ حقيقية. SavedModel وLiteRT وTF Serving يغطّون سلسلةً كاملة.
2. **نظامٌ بيئيٌّ ناضج.** توثيقٌ ضخم، أدواتُ مراقبة، تكاملٌ مع السحابات، ومجتمعٌ واسع.
3. **الأداءُ على المدى الطويل.** `tf.data` و `tf.function` وXLA والتدريبُ الموزّع مصمّمةٌ للمشاريع الكبيرة.
4. **الاستقرارُ في الإنتاج.** واجهاتُ الحفظ والخدمة مُختبرةٌ في أنظمةٍ ضخمة، مع دعمٍ للإصدارات.

5. **الجسرُ إلى الأجهزة الطرفية.** من الخادم إلى الهاتف إلى المتحكّم الدقيق، بلا إعادة كتابة النموذج من الصفر.

لكنّ هذه الأسبابَ نفسَها تحملُ ثمنًا: سطحٌ واسعٌ قد يُشتّت، وتغييراتٌ بين الإصدارات، وكودٌ منخفضُ المستوى أحيانًا. ولا عيبَ في الاعتراف بذلك.

1.6 TensorFlow أم PyTorch أم JAX؟

لا يوجد «أفضل» مطلق. يوجد **سياق**. هذا جدولٌ يختصرُ الفروق العملية:

المعيار	TensorFlow 2.21	PyTorch 2.x	JAX
سهولة التعلّم الأولى	متوسطة (Keras تُسهّلها)	عالية	منخفضة (نمط وظيفي)
مرونة البحث	جيدة	ممتازة	ممتازة جدًا
النشر على الأجهزة الطرفية	ممتاز (LiteRT)	جيد (ExecuTorch)	محدود
التدريب الموزّع	ناضج (Strategy)	ناضج	قوي جدًا (pmap/shard_map)
النظام البيئي للإنتاج	الأوسع	واسع	أصغر
التوثيق العربي	نادر	نادر	نادر جدًا

قاعدة القرار: إن كان هدفك نشرَ نموذجٍ على هاتفٍ أو خدمةٍ إنتاجيةٍ واسعة، فTensorFlow خيارٌ طبيعي. إن كنتَ في مختبر أبحاثٍ تُجرّب أفكارًا جديدة كلَّ يوم، فقد يكون PyTorch أو JAX أسرع. وإن كنتَ تحتاجُ تفاضلًا تلقائيًا متقدّمًا وتوازيًا هائلًا، فJAX يستحقُّ التجربة. والأهم: **المفاهيمُ تنتقل**. من يُتقنُ الانتشارَ الخلفيَّ والالتفافَ والانتباهَ يستطيعُ الانتقالَ بين الأدوات في أسابيع، لا سنوات.



شكل 1-2 المسار الزمني لتطوّر TensorFlow من 2015 إلى 2026

1.7 أول نموذج: خطٌّ مستقيم يتعلّم

لنبدأ بأصغر مثالٍ ممكن: نريدُ نموذجًا يتعلّم العلاقة بين سعرٍ بالدولار وسعرٍ بالريال. العلاقةُ خطية:

$$y = wx + b$$

حيث x السعرُ بالدولار، وy السعرُ بالريال، وw معاملُ التحويل (سعر الصرف)، وb التحييز. لا نعرفُ w وb؛ نريدُ أن يتعلّمهما النموذجُ من أمثلة. نُعرّفُ الخسارةَ بمتوسط مربع الخطأ:

$$L(w, b) = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2$$

حيث N عددُ الأمثلة، و$\hat{y}_i$ التنبؤ، وy_i القيمةُ الحقيقية. كلّما صغرتْ L، اقتربَ الخطُّ من النقاط. يحدّثُ المحسّنُ الوسائطَ في اتجاه عكس التدرّج:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \qquad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

وη هو معدّلُ التعلّم: حجمُ الخطوة. صغيرٌ جدًا = تعلّمٌ بطيء، كبيرٌ جدًا = قفزٌ فوق الحل.

والآن الكود الكامل:

```python
import numpy as np
import tensorflow as tf
import keras

keras.utils.set_random_seed(42)          # نتائج قابلة للتكرار قدر الإمكان

# 1) بيانات صغيرة: سعر بالدولار → سعر بالريال (علاقة خطية تقريبية)
x = np.array([[1.0], [2.0], [3.0], [4.0], [5.0]], dtype="float32")
y = np.array([[3.75], [7.50], [11.25], [15.00], [18.75]], dtype="float32")

# 2) النموذج: طبقة خطية واحدة (Dense) بلا دالة تنشيط
model = keras.Sequential([
    keras.Input(shape=(1,)),
    keras.layers.Dense(units=1),
])

# 3) التجميع: خسارة MSE، محسّن SGD بمعدل تعلّم صغير
model.compile(
    optimizer=keras.optimizers.SGD(learning_rate=0.05),
    loss=keras.losses.MeanSquaredError(),
    metrics=["mae"],
)

# 4) التدريب: 200 دورة على خمس نقاط
history = model.fit(x, y, epochs=200, verbose=0)

# 5) الفحص
print("W =", model.layers[0].get_weights()[0].ravel()[0])
print("b =", model.layers[0].get_weights()[1][0])
print("التنبؤ بسعر 7$ =", model.predict(np.array([[7.0]]), verbose=0)[0][0])
# النتيجة المتوقعة: W قريب من 3.75، b قريب من 26.25، والتنبؤ قريب من 0
```

قفْ عند كل سطر. `Input(shape=(1,))` تعني: كلُّ عيّنةٍ رقمٌ واحد. `Dense(1)` تعني: تحويلٌ خطيٌّ واحد. `compile` لا يُدرّب، بل يُجهّز. `fit` هو الذي يُشغّلُ الحلقة: تمريرٌ أمامي، حسابُ خسارة، تمريرٌ خلفي، تحديثُ أوزان. `predict` يُعيدُ القيمةَ المتوقّعة.

هذا المثالُ الصغيرُ يحتوي، في جوهره، كلَّ ما ستفعله في الشبكات العملاقة: بيانات، نموذج، خسارة، محسّن، تدريب، تقييم. الفرقُ أن الشبكةَ العميقةَ تضيفُ طبقاتٍ وتنشيطاتٍ وتنظيمًا، لا مبادئَ جديدة.

> ⚠ **تحذير**
>
> **عرض:** يبدأ المبتدئ بـ `model.fit` على بياناتٍ لا يفهمها، ثم يتفاجأ بأنّ الخسارة لا تنخفض. **السبب:** شكلُ البيانات أو نوعُها أو تطبيعُها غير مناسب. **الحل:** اطبعْ `y.shape` , `x.dtype` , `x.shape` , وأولَ خمس قيم قبل أي تدريب.

راقب مجموعة تحقق واحفظ أفضل نسخة

بعد أن يعمل النموذج، اجعل قرار التوقف والحفظ جزءًا من الدورة نفسها بدل اختيار آخر حقبة يدويًا:

```python
callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=3,
                                  restore_best_weights=True),
    keras.callbacks.ModelCheckpoint("best.keras", save_best_only=True),
]
history = model.fit(x_train, y_train, validation_split=0.2,
                    epochs=30, callbacks=callbacks)
```

1.8 متى لا تختار TensorFlow؟

هذا سؤالٌ لا يطرحُه المتحمّسون عادةً، لكنّه علامةُ النضوج:

- **بياناتٌ جدوليةٌ صغيرة:** غالبًا شجرةُ قرارٍ أو XGBoost/LightGBM أسرعُ وأدقُّ من شبكةٍ عميقة.
- **نموذجٌ كلاسيكي:** الانحدارُ اللوجستي أو SVM قد يكفي، وبلا تعقيد.
- **فريقٌ يعتمدُ PyTorch بالأساس:** لا تُغيّرِ الأداةَ لمجرد الذوق؛ التكلفةُ البشريةُ أكبرُ من الفرق التقني.
- **مهمةٌ تحتاجُ مكتبةً بحثيةً حديثةً جدًا:** بعض الأوراق تُنشرُ أولًا بـPyTorch/JAX، وقد يستغرقُ نقلُها وقتًا.
- **جهازٌ محدودٌ جدًا بلا دعمٍ للنشر:** راجعْ بدائلَ LiteRT وONNX أولًا.

1.9 حالة واقعية: عندما أنقذت LiteRT المشروع

عاد فريقُ المستشفى إلى النموذج. كان حجمُه 90 ميغابايت، ودقتُه 94%. جرّبوا تشغيلَه على جهاز الأشعة: بطيءٌ جدًا. الحلُّ لم يكن شبكةً أفضل، بل **طريقةَ نشرٍ أفضل**:

1. تصديرُ النموذج إلى SavedModel.
2. تحويلُه إلى LiteRT.
3. ضغطُ الأوزان إلى `float16`.
4. اختبارُ الدقة بعد الضغط: 93.6% – فرقٌ مقبول.
5. قياسُ الزمن: من 4.2 ثانية إلى 0.6 ثانية للصورة.

لم يتغيّر النموذجُ علميًا، لكنّه صار قابلًا للاستخدام. هذه هي قيمةُ المنظومة: **الفكرةُ الجيدةُ لا تكفي؛ يجب أن تصل**. في الفصل التاسع عشر ستُعيدُ هذه الرحلةَ بيدك، من `model.export` إلى خدمة FastAPI.

1.10 أخطاء شائعة

خطر

الخطأ: تعلّمُ TensorFlow بشراء كتابٍ وحفظِ الأوامر دون تشغيل. **النتيجة:** معرفةٌ هشّة تنهار عند أول خطأ شكل. **البديل:** اكتبْ كلَّ مثال، وأفسدْه عمدًا، ثم أصلحْه. الخطأُ المُفتعَلُ في المختبر أرحمُ من الخطأ الحقيقي في الإنتاج.

تحذير

الخطأ: استخدامُ `tf.keras` و `keras` بالتبادل دون معرفة الفرق. **التوضيح:** في TensorFlow 2.16+ أصبح Keras 3 هو الافتراضي، و `tf.keras` واجهةَ توافق. اكتبْ `import keras` في الكتاب، واعرفْ أن `tf.keras` قد يختلف في الإصدارات.

تحذير

الخطأ: توقّعُ أن GPU سيجعل كلَّ شيء أسرع. **الحقيقة:** إن كان خطُّ البيانات هو عنقَ الزجاجة، فـGPU سينتظر. حسّنْ `tf.data` أولًا.

1.11 تمارين

1. **تصنيف الدوائر:** صنّفْ خمسة تطبيقات (كشف spam، قيادة ذاتية، توصية أفلام، تشخيص طبي، ترجمة) إلى AI/ML/DL، مع تبرير. (المتوقع: جدولٌ يوضّح أن كلَّها ML، وبعضها DL، مع تفسير.)
2. **النسخة:** شغّلْ `print(tf.__version__)` و `print(keras.__version__)`. (المتوقع: 2.21.0 و3.15.1 أو أحدث.)
3. **تعديل المثال:** غيّرْ معدّلَ التعلّم إلى 0.5 ثم 0.001. ماذا يحدث للخسارة؟ (المتوقع: 0.5 قد يتذبذب/ينفجر، و0.001 يتعلّم ببطء.)
4. **بلا تحييز:** احذفِ التحييز من النموذج باستخدام `use_bias=False`. هل يناسب البيانات؟ (المتوقع: نعم هنا، لأن الخط يمرّ بالأصل تقريبًا، لكن ليس دائمًا.)
5. **تحليل:** لماذا لا تُستخدم دقة التصنيف وحدها في بياناتٍ غير متوازنة؟ (المتوقع: لأن نموذجًا يتنبأ بالأغلبية قد يحقق دقة عالية دون أن يكتشف الأقلية.)
6. **بحث:** ابحثْ عن مكوّنٍ من منظومة TensorFlow لم يُذكر في الفصل، واكتبْ عنه ثلاثة أسطر. (المتوقع: مثل TF-GNN، TF Decision Forests، TF Privacy، TF Hub.)
7. **قرار:** فريقٌ يريدُ نموذجًا للهاتف خلال شهر. اكتبْ توصيتك في خمسة أسطر. (المتوقع: TensorFlow + LiteRT، مع خطة بيانات وتقييم ونشر.)

1.12 ملخص

- الذكاء الاصطناعي ⊃ تعلّم الآلة ⊃ التعلّم العميق، والأداةُ تتبعُ المشكلة.
- TensorFlow منظومةٌ كاملة: Core، Keras، tf.data، TensorBoard، TFX، LiteRT، Serving، TF.js.
- أقوى ما فيه هو **الجسر من التدريب إلى النشر**، لا مجرّد بناء النماذج.
- لا يوجد إطارٌ «أفضل» مطلقًا؛ يوجد سياقٌ ومعايير.
- أول نموذجٍ يكشف الحلقةَ الأساسية: بيانات ← نموذج ← خسارة ← محسّن ← تقييم.
- النجاحُ في الإنتاج يبدأ من فهم البيانات وطريقة النشر، لا من تعقيد المعمارية.

الجزء الأول · الأساسات: كيف تفكّر في TensorFlow؟

الفصل الثاني

02

بيئة 2026: Python 3.14 وTensorFlow 2.21

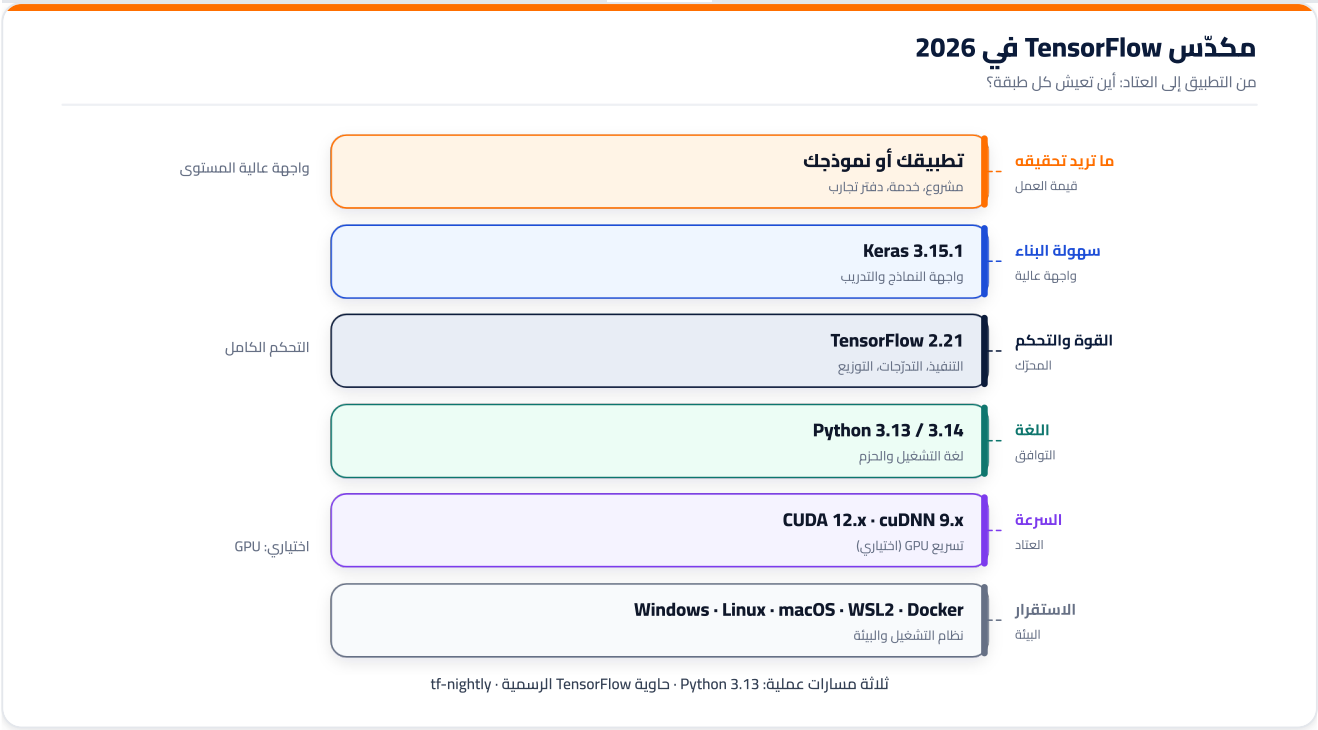
> **حكاية الفصل**
>
> وصل مهندس جديد إلى فريق الرؤية، وثبّت Python الأحدث على حاسوبه قبل يوم العرض. نجح تثبيت المحرّر، لكن حزمة TensorFlow لم تجد عجلة متوافقة. لم تكن المشكلة في بطاقة الرسوم ولا في الأمر الذي كتبه: كان إصدار Python نفسه خارج مصفوفة الدعم. كيف نختار بيئة قابلة لإعادة الإنتاج بدل مطاردة أحدث رقم؟

2.1 أهداف الفصل

- ☐ قراءة توافق Python وTensorFlow قبل التثبيت.
- ☐ إنشاء بيئة معزولة وتثبيت الحزم والتحقق منها.
- ☐ اختيار مسار CPU أو GPU أو حاوية أو بيئة سحابية.
- ☐ تشخيص أخطاء المكتبات والتعريفات الشائعة.

2.2 التوافق يسبق أمر التثبيت

لقطة توافق الطبعة — 24 أكتوبر 2026: الإصدار المستقر المرجعي هنا TensorFlow 2.21.0، وتدعم عجلاته الرسمية Python 3.10-3.13. أُعلن أيضًا عن TensorFlow 2.22.0-rc0 بوصفه إصدارًا مرشحًا، لكن الإصدار المرشح ليس اختيارًا افتراضيًا لبيئة إنتاج. مرجع Keras المستقر في أمثلة الكتاب 3.15.1؛ توافق Keras مع Python 3.14 لا يعني أن TensorFlow يوفّر عجلة رسمية لهذا المفسر. راجع تثبيت TensorFlow ومصفوفة Python، وإصدار TensorFlow 2.22.0-rc0، وإصدار Keras 3.15.1 قبل تثبيت بيئة جديدة.



شكل 2-1 طبقات بيئة TensorFlow من مفسر Python إلى المسرّع

حزمة TensorFlow ليست ملف Python خالصًا؛ تحتوي أجزاء مترجمة ترتبط بإصدار المفسر ونظام التشغيل والمعمارية. بحسب مرجع هذا الكتاب، TensorFlow 2.21.0 يوفر عجلات رسمية لـ Python 3.10-3.13. أما Python 3.14.6 فلا تعني تلقائيًا توفر عجلة متوافقة. افحص مصفوفة الإصدار على الموقع الرسمي وقت إعداد الجهاز، لأن التوافق يتغير أسرع من الكتب.

المسار	متى تختاره؟	ما الذي تتحقق منه؟
Python 3.13 داخل venv	تطوير محلي اعتيادي	عجلة TensorFlow ونسخة pip
حاوية رسمية	فريق يحتاج بيئة موحّدة	وسم الصورة ونسخة CUDA
WSL2 على Windows	عمل GPU ضمن بيئة Linux	تعريف NVIDIA وتوافق CUDA
Kaggle أو Colab	تجربة سريعة أو موارد مؤقتة	إصدار الحزمة ونوع المسرّع
tf-nightly	اختبار ميزة أو توافق مبكر	أنه إصدار ليلي متغيّر لا نشر إنتاج

المعادلة العملية هي أن البيئة القابلة للتكرار = إصدار مفسر معلوم + إصدارات حزم مثبتة + بيانات وإعدادات موثّقة. لا يكفي أن يعمل الاستيراد على جهاز شخص واحد.

2.3 أنشئ بيئة واعرف ما ثبّتّ

يمكن تلخيص شرط صلاحية التثبيت بأن يتقاطع دعم المفسّر والمنصة مع العجلات المنشورة للحزمة:

$$Environment_valid = Python_supported \wedge Platform_supported \wedge Dependencies_compatible$$

كل حد شرط مستقل: نجاح اثنين لا يعوض غياب الثالث.

```powershell
# على Windows: استخدم مفسرًا مدعومًا بدل الاعتماد على Python الافتراضي.
py -3.13 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install tensorflow==2.21.0 keras==3.15.1
python -m pip freeze > requirements-lock.txt
```

على Linux وmacOS يُفعّل المفسّر عبر `source .venv/bin/activate`. ويمكن استخدام `uv` لإنشاء البيئة وتثبيت الاعتمادات، بشرط تثبيت إصدارات صريحة للمشروع. لا تخلط بين `pip` الذي يعود إلى مفسّر آخر وبين البيئة النشطة؛ نفّذ `python -m pip` كي يكون الارتباط واضحًا.

```python
import sys
import tensorflow as tf

print(sys.version)
print(tf.__version__)
print(tf.config.list_physical_devices("GPU"))
# إصدار Python المتوقع: 2.21.0، ثم قائمة الأجهزة التي اكتشفها TensorFlow.
```

2.4 CPU وGPU: افصل بين التعريف والمكتبة

إن لم تظهر GPU، افحص أولًا أن نظام التشغيل يرى البطاقة وأن تعريف NVIDIA حديث ومدعوم. بعد ذلك طابق إصدار CUDA وcuDNN الذي تتطلبه نسخة TensorFlow؛ تثبيت CUDA عشوائيًا قد يجعل مكتبات النظام تتعارض. على Windows، المسار الشائع للتدريب المحلي ببطاقات NVIDIA هو WSL2 مع بيئة Linux، أو استخدام

حاوية موثّقة. لا تثبّت ملفات DLL من مصادر مجهولة لمعالجة رسالة تحميل مكتبة.

قبل الاعتماد على المسرّع، نفّذ عملية صغيرة عليه وقِس الزمن بعد الإحماء. ظهور اسم GPU يثبت الاكتشاف فقط، ولا يثبت أن النموذج أسرع: نقل البيانات، وصغر الدفعة، والعمليات غير المدعومة قد تجعل CPU أفضل.

2.5 تشخيص الأعطال

العرض	السبب المرجّح	خطوة التشخيص
`No matching distribution`	Python أو المعمارية غير مدعومة	افحص `python --version` ومصفوفة العجلات
`DLL load failed`	مكتبة أصلية أو runtime مفقود/متعارض	أنشئ venv نظيفة وافحص متطلبات النظام
استيراد NumPy يفشل	إصدار ABI غير متوافق	ثبّت نسخًا متوافقة من ملف القفل
GPU غير ظاهرة	تعريف أو بيئة تشغيل غير مكتملة	افحص الجهاز خارج TensorFlow ثم داخل الحاوية/WSL
نفاد VRAM	دفعة أو نموذج أكبر من الذاكرة	خفّض batch size وراقب الذاكرة

لا تعالج كل خطأ بإعادة تثبيت Python على النظام كله. سجّل نص الخطأ، والمفسّر النشط، ونسخ الحزم، ثم أعد التجربة في بيئة نظيفة يمكن حذفها دون إفساد مشاريع أخرى.

2.6 حالة واقعية: تجهيز فريق قبل بدء التدريب

يحتاج فريق جامعي إلى تدريب نموذج صور على حاسوبين مختلفين. يثبت الفريق Python 3.13 وTensorFlow 2.21g في بيئتين افتراضيتين، ويحفظ `requirements-lock.txt` ونسخة بيانات صغيرة وبذرة العشوائية. يخصص أحد الجهازين لـ CPU والآخر لـ GPU، ويقارن عدد الصور في الثانية بعد الإحماء. يصبح اختلاف الأداء معلومًا وقابلًا للتفسير، بدل أن يختلط بفرق إصدارات غير مسجل.

2.7 مصفوفة التثبيت العملية

لا توجد وصفة GPU واحدة صحيحة لكل أنظمة التشغيل. ثبّت نظام الأساس أولًا، ثم اتبع جدول الدعم الخاص بالإصدار الدقيق من TensorFlow. مرجع هذا الفصل TensorFlow 2.21.0 وKeras 3.15.1. يدعم TensorFlow 2.21 Python من 3.10 إلى 3.13 وفق مصفوفة الإنشاء؛ وجود دعم Python 3.14 في Keras لا يعني أن TensorFlow نفسه يملك عجلة رسمية لهذا المفسر. هذه نقطة يسهل أن تختلط على المبتدئ لأن `import keras` قد يعمل بينما `import tensorflow` لا يملك حزمة متوافقة. ملاحظات إصدار TensorFlow 2.21 وإعدادات البناء المدعومة.

النظام	CPU	GPU	قرار عملي
Windows أصلي	حزمة CPU على نسخ حديثة	دعم GPU الأصلي توقف بعد TensorFlow 2.10	استخدم WSL2 أو جهازًا سحابيًا لـ GPU
Linux x86-64	`tensorflow`	إعداد CUDA المعتمد لذلك الإصدار	طابق التعريف والمكتبات مع الدليل الرسمي
macOS	CPU	لا يوجد دعم CUDA رسمي لبطاقات Apple	استخدم CPU أو backend مناسبًا للعتاد
Colab/Kaggle	بيئة جاهزة	موارد سحابية بحسب الجلسة	لا تعتمد على بقاء القرص المؤقت

هذه مصفوفة قرار وليست قائمة كل الاستثناءات. الحزم المرتبطة بالعتاد تتغير؛ لا تنسخ أرقام CUDA من مقال قديم إلى إصدار جديد. على Windows، يجب التفريق بين تشغيل CPU على Windows وبين تسريع CUDA عبر Linux/WSL2. كما أن ظهور GPU في النظام لا يضمن أن كل العمليات أو كل أنواع البيانات تستخدمها.

2.8 ملف اعتماد قابل لإعادة الإنتاج

ملف `pip freeze` مفيد لتسجيل حل بيئة موجودة، لكنه ليس دائمًا أفضل ملف اعتماد بشري لمكتبة عامة؛ فقد يسجل حزمًا انتقالية خاصة بجهازك. للمشروع الصغير، اكتب الاعتمادات المباشرة بإصدارات مثبتة ثم أنشئ ملف قفل بأداة المشروع. سجّل نسخة Python ونظام التشغيل في README، لأن سطر الحزمة وحده لا يقفل العجلات الثنائية أو تعريف GPU.

```text
# requirements.in: الاعتمادات التي اختارها المشروع مباشرة
tensorflow==2.21.0
keras==3.15.1
numpy
```

في بيئة بحث، أضف تاريخ بناء البيئة، الجهاز، إعداد البذور، وطريقة الوصول إلى البيانات. لا تضع رموز الوصول أو كلمات المرور في ملف القفل. خزّن نسخة آمنة من ملف الاعتماد مع نتيجة التجربة، وأعد إنشاءها في بيئة نظيفة قبل مشاركة الورقة أو النموذج.

2.9 فحص GPU بطريقة منصفة

يتطلب تقرير الاستخدام أكثر من `list_physical_devices` . اجعل العملية كبيرة بما يكفي لإظهار عمل المسرّع، نفذها عدة مرات بعد الإحماء، واجمع زمنًا متزامنًا. بعض التنفيذات غير متزامنة؛ قياس وقت إرسال الأمر فقط يعطي زمنًا مضللًا. افحص كذلك ذاكرة الجهاز، وتأكد أن البيانات لا تُنسخ بين CPU وGPU في كل خطوة. ابدأ بـ CPU لاختبار صحة الشفرة، ثم قارن السرعة على حجم عمل واقعي.

2.10 التخطيط بين مبتدئ وباحث وفريق إنتاج

للمبتدئ، ابدأ ببيئة CPU نظيفة ومثال Keras صغير؛ لا تجعل إعداد CUDA شرطًا لفهم loss أو gradient. للباحث، ثبّت إصدارًا دقيقًا واحتفظ بملف البيئة وإعداد الجهاز مع الورقة أو الدفتر. لفريق الإنتاج، ابنِ صورة حاوية محددة بالـ digest، وافصل build environment عن runtime، واختبر تحميل النموذج من الصفر في بيئة معزولة. لكل مرحلة احتياج مختلف؛ لا تبدأ بخدمة Kubernetes عند حل واجب تصنيف.

إذا استخدمت Docker، مرر GPU من المضيف إلى الحاوية عبر إعداد runtime موثّق، وثبت tag للصورة لا `latest` . لا تخلط صورة TensorFlow التي تتضمن CUDA بإصدار تعريف غير متوافق على المضيف. سجّل `nvidia-smi` , نسخة الحاوية، `tf.__version__` , ومخرجات اختبار GPU؛ عند إعادة الإنتاج، لا تكفي عبارة «لدي بطاقة RTX». إذا كانت بياناتك خاصة، افحص mount والصلاحيات ومسار cache وملفات checkpoints قبل نشر الحاوية.

2.11 إدارة البيئة عبر عدة مشاريع

لا تضع TensorFlow في Python النظام. استخدم venv أو uv أو Conda وفق سياسة الفريق، ولا تخلط حزمًا ثنائية من قنوات مختلفة. أنشئ اسم بيئة صريحًا لكل مشروع، ونفذ التثبيت في CI على مفسر مدعوم. يمكن أن يثبت `pip check` تناقض metadata، لكنه لا يضمن أن كل kernel GPU يعمل. أضف smoke test قصيرًا في CI يستورد الحزم، ويبني نموذجًا صغيرًا، ويحفظه ثم يعيد تحميله.

عند ترقية TensorFlow، شغّل مقارنة على مجموعة تحقق ثابتة وسجل تغييرات API والحزم الملحقة.
TensorFlow 2.21 أزال الاعتماد الافتراضي على TensorBoard؛ إذا كان المشروع يستخدمه في callbacks أو تحليل ملفات، أضفه اعتمادًا مباشرًا وبنسخة معلومة بدل الاعتماد العرضي. وتراجع دعم Python 3.9 في هذا الإصدار. ملاحظات الإصدار الرسمية.

2.12 التثبيت ليس نهاية التحقق

بعد `pip install` , تحقق من أربع طبقات: مسار المفسر، إصدار TensorFlow/Keras، عملية حساب على CPU، ثم اكتشاف/استخدام GPU إن كان مطلوبًا. جرب تحميل ملف نموذج وحفظه في مسار يمكن للبرنامج الوصول إليه. نفذ تجربة صغيرة من ملف CSV أو صورة؛ مشكلات codecs ومكتبات الصور لا تظهر في `import` `tensorflow`. وإذا كان العمل متعدد الأجهزة، اختبر `strategy.num_replicas_in_sync` على الجهاز الحقيقي قبل تنفيذ تدريب طويل.

2.13 أمثلة تنفيذية إضافية

مفسّر Python

```python
import sys
print(sys.executable)
print(sys.version.split()[0])
```

عملية على CPU

```python
import tensorflow as tf
with tf.device("/CPU:0"):
    result = tf.reduce_sum(tf.ones((128, 128)))
print(float(result))  # المتوقع: 16384.0
```

2.14 أخطاء شائعة

⚠ تحذير

أحدث Python لا يعني أنه مدعوم من كل حزمة ذات امتدادات أصلية. تحقّق من العجلات قبل إنشاء المشروع. [DANGER!] لا تخلط CUDA وcuDNN من مصادر وإصدارات مختلفة اعتمادًا على رسالة خطأ منفردة؛ قد يبدو الاستيراد ناجحًا ثم يفشل التدريب.

2.15 تمارين

1. اختر إصدار Python مدعومًا في المصفوفة الحالية وثبّت البيئة. (المتوقع: تقرير نسخ يمكن إعادة بنائه.)
2. قارن `python -m pip --version` قبل تفعيل البيئة وبعده. (المتوقع: مساران مختلفان واضحان.)
3. افحص الأجهزة المرئية لـ TensorFlow. (المتوقع: CPU على الأقل، وGPU إن كان الإعداد مكتملًا.)
4. شخّص رسالة `No matching distribution` دون حذف Python العام. (المتوقع: تحديد عدم توافق المفسّر أو المنصة.)
5. قارن CPU وGPU بعد إحماء عمليتين متطابقتين. (المتوقع: قياس منصف لا يعتمد على أول تشغيل.)
6. اكتب ملف قفل وشارك خطوات إعادة إنشاء البيئة. (المتوقع: إعداد موثّق لفريق آخر.)

2.16 ملخص

- افحص توافق TensorFlow والمفسّر قبل التثبيت؛ مرجع هذا الكتاب لا يَعُد Python 3.14 مدعومًا بعجلة رسمية لـ 2.21.
- اعزل الاعتمادات وسجّلها، ثم تحقق من الأجهزة والإصدارات من داخل البيئة النشطة.
- GPU المكتشفة لا تضمن تدريبًا أسرع، وبيئة نظيفة أفضل من ترقيع تثبيت النظام.

الجزء الأول · الأساسات: كيف تفكّر في TensorFlow؟

الفصل الثالث

03

التنسورات: التفكير بالأبعاد

حكاية الفصل

أرسل فريق متجرٍ صور المنتجات على هيئة دفعة، لكن النموذج رفض الإدخال. كل صورة منفردة كانت مقبولة، والبيانات لم تتغير. اختلف ترتيب الأبعاد فقط: توقّع النموذج دفعة ثم ارتفاعًا وعرضًا وقنوات، بينما وصلته القنوات في الموضع الثاني. كيف نقرأ شكل التنسور قبل أن نتهم النموذج؟

3.1 أهداف الفصل

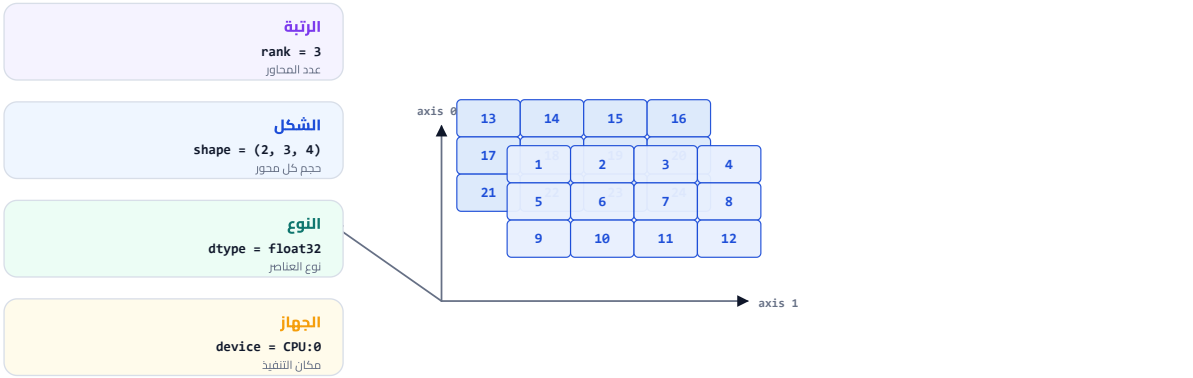
- ☐ التمييز بين الرتبة والشكل والنوع والجهاز.
- ☐ إنشاء التنسورات وفهرستها وإعادة تشكيلها ودمجها.
- ☐ فهم البث ومتغيرات التدريب والبيانات المتناثرة.
- ☐ تشخيص عدم توافق الأبعاد قبل التدريب.

3.2 أربعة أوصاف للتنسور

التنسور مصفوفة متعددة الأبعاد. الرتبة عدد محاوره، والشكل حجم كل محور، والنوع يحدد دقة التخزين والعمليات، والجهاز يحدد مكان التنفيذ. المتجه `[2, 5, 7]` رتبته 1 وشكله `(3,)`؛ ومصفوفة ثلاث صور رمادية قد يكون شكلها `(3, 28, 28, 1)`. البعد الأول هنا عدد الأمثلة، لا جزءًا من كل صورة.

التنسور: رقم، متجه، مصفوفة، ومكعب

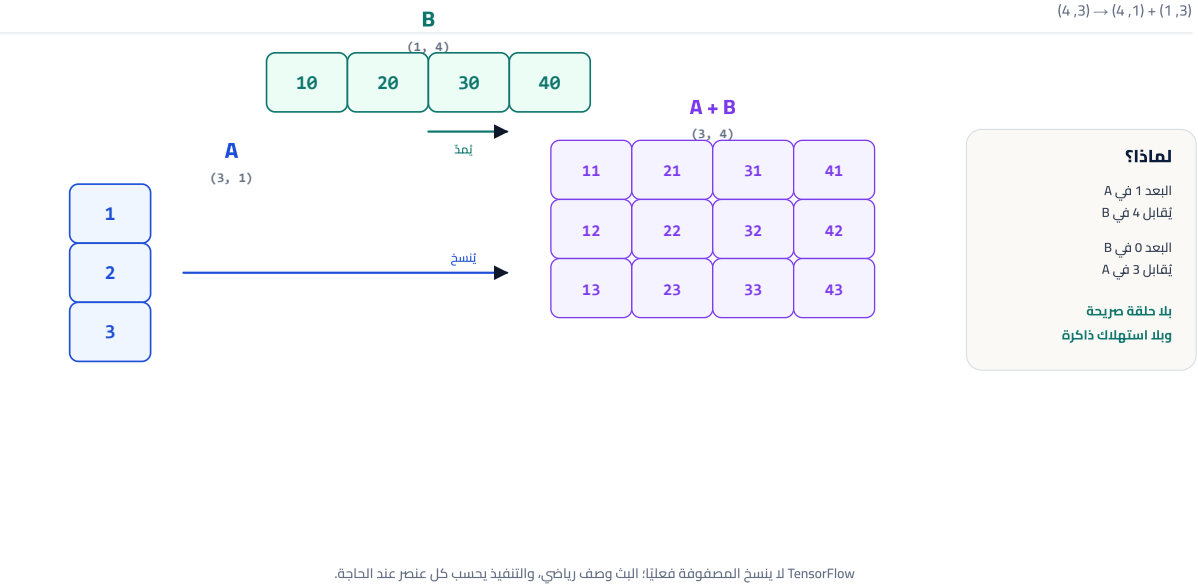
الرتبة تصف عدد الأبعاد، والشكل يصف حجم كل بُعد



شكل 3-1 الرتبة والشكل والنوع في التنسور

البث (Broadcasting): توسيع تلقائي بلا نسخ

(3, 1) + (1, 4) → (3, 4)



TensorFlow لا ينسخ المصفوفة فعليًا؛ البث وصف رياضي، والتنفيذ يحسب كل عنصر عند الحاجة.

شكل 3-2 أمثلة توافق الأبعاد في البث

$$N_{elements} = \prod_{i=1}^{r} d_i$$

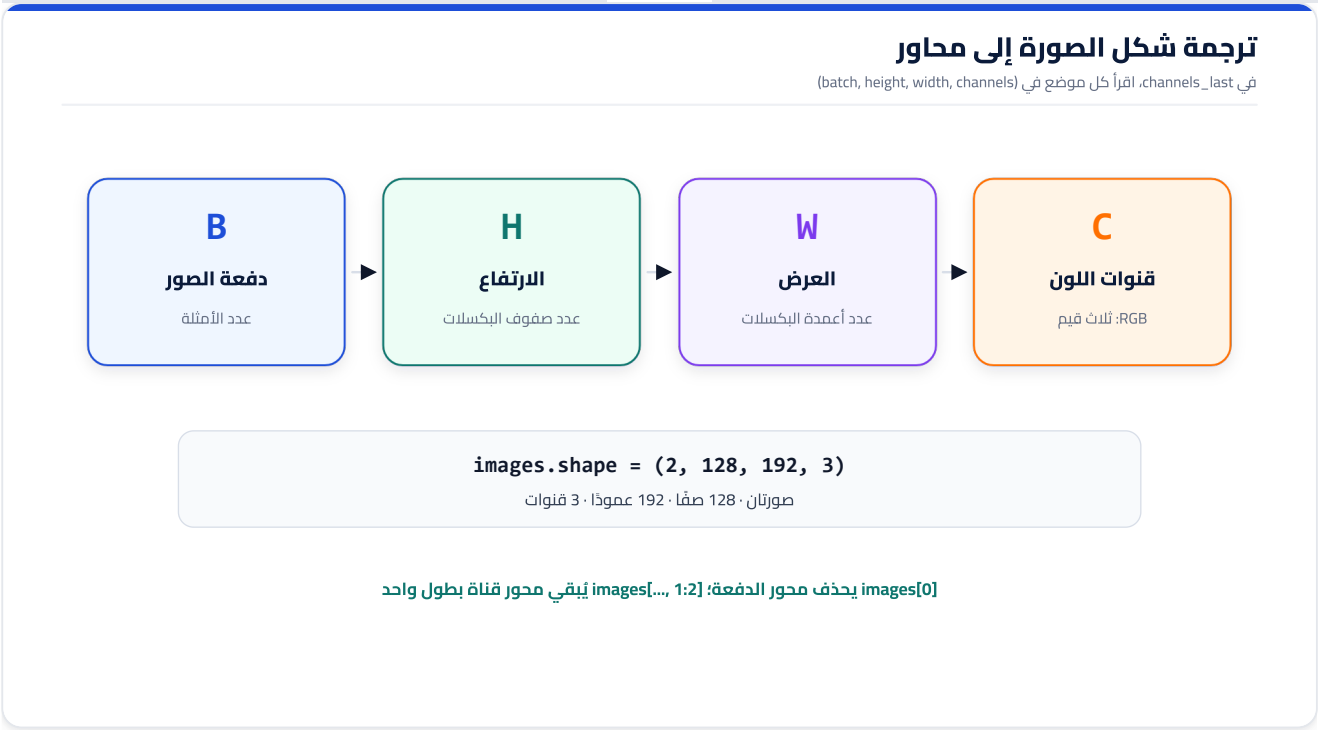
حيث `r` الرتبة و`d_i` طول المحور رقم `i`. يساعد عدد العناصر على اكتشاف إعادة تشكيل لا تغيّر البيانات: يجب أن يبقى حاصل الضرب نفسه.

```python
import tensorflow as tf

x = tf.constant([[1, 2, 3], [4, 5, 6]], dtype=tf.float32)
print(x.shape, x.dtype, x.ndim)
print(x[:, 1])
# المتوقع: (2, 3) و float32 ثم [2. 5.]، و2
```

3.3 اقرأ شكل الصورة قبل أن تدخل النموذج

ما معنى `(2, 128, 192, 3)`؟ لا تحفظه كسلسلة أرقام؛ اسأل عن كل محور: هل هذا عدد الأمثلة، أم ارتفاع الصورة، أم عرضها، أم قنوات اللون؟ وفق ترتيب `channels_last` الشائع في Keras، لدينا صورتان، ارتفاع كل منهما 128 بكسل، وعرضها 192، وفي كل بكسل ثلاث قيم للأحمر والأخضر والأزرق. إذا اخترنا صورة واحدة بقيت أبعادها الثلاثة، وإذا أخذنا القناة الخضراء وحدها تغيّر الشكل مرة أخرى.



شكل 3-3 ترجمة شكل الصورة إلى محاور مفهومة قبل تمريرها إلى النموذج

جرّب القراءة على بيانات صغيرة قبل تحميل مجموعة صور كبيرة. يطبع المثال شكل الدفعة والصورة المفردة والقناة الخضراء؛ وانظر كيف يحافظ التقطيع `[..., 1:2]` على محور قناة بطول واحد، بينما `[..., 1]` يحذفه.

```python
import tensorflow as tf

# محاكاة دفعة صور RGB بأعداد صحيحة بين 0 و255.
raw = tf.random.uniform((2, 128, 192, 3), maxval=256, dtype=tf.int32)
raw = tf.cast(raw, tf.uint8)
images = tf.image.convert_image_dtype(raw, tf.float32)

one_image = images[0]
green_without_axis = images[..., 1]
green_with_axis = images[..., 1:2]
print(images.shape, one_image.shape)
print(green_without_axis.shape, green_with_axis.shape)
# المتوقع: (3 ,192 ,128 ,2) و(3 ,192 ,128)
# ثم (192 ,128 ,2) و(1 ,192 ,128 ,2).
```

لماذا نهتم بمحور واحد؟ لأن طبقة `Conv2D` تحتاج أن تعرف أي محور يمثل القنوات. وإذا وصلها `(batch, channels, height, width)` بدل `(batch, height, width, channels)` فالأعداد نفسها موجودة، لكن معانيها تبدلت. افحص `element_spec` للدفعة، واطبع شكل عينة، واختبر ناتج التحويل قبل بدء التدريب. هذه الخطوات الصغيرة تمنع ساعات من البحث في خطأ سببه ترتيب الأبعاد.

3.4 إعادة تشكيل دون تغيير الترتيب

`reshape` يغيّر طريقة عرض العناصر في ترتيبها الحالي. أما `transpose` فيبدّل محاورها. إذا كان `images` بالشكل `(batch, height, width, channels)`، فإن التحويل إلى `(batch, channels, height, width)` هو تبديل محاور، لا إعادة تشكيل عمياء. استخدم الأسماء أو اطبع الأشكال عند كل حدّ بين خط البيانات والنموذج.

```python
import tensorflow as tf

x = tf.reshape(tf.range(12), (3, 4))
column = tf.expand_dims(x[:, 0], axis=-1)
joined = tf.concat([column, column], axis=1)
stacked = tf.stack([column, column], axis=0)
print(x.shape, column.shape, joined.shape, stacked.shape)
# المتوقع: (3, 4)، (3, 1)، (2, 3)، (1, 3, 2)
```

الدمج `concat` يزيد محورًا موجودًا، أما `stack` فينشئ محورًا جديدًا. اختيار المحور يحدد معنى الناتج: هل أضفنا ميزات إلى كل مثال أم أضفنا مجموعة أمثلة جديدة؟

3.5 البث: محاذاة المحاور من اليمين

يجوز تنفيذ عملية عنصر بعنصر عندما تتطابق الأبعاد أو يكون أحدها 1، بدءًا من المحور الأيمن. لذلك يمكن جمع متجه انحياز شكله `(features,)` إلى كل صف في دفعة شكلها `(batch, features)`. لكن الشكل `(batch,)` لا يطابق تلقائيًا `(batch, features)` كما قد يتوقع المرء؛ أضف محورًا صريحًا عند الحاجة.

```python
import tensorflow as tf

images = tf.zeros((8, 32, 32, 3))
mean = tf.constant([0.5, 0.4, 0.3])
normalized = images - mean
print(normalized.shape)
# المتوقع: (8, 32, 32, 3)، إذ بُثّ متوسط القنوات على المحاور السابقة.
```

3.6 قيم ثابتة وأوزان قابلة للتعلم

`tf.constant` مناسب للبيانات التي لا تتغير أثناء التدريب. `tf.Variable` يحمل حالة متغيرة مثل وزن طبقة، ويحدّثه المحسّن. لا تُنشئ وزنًا جديدًا داخل كل استدعاء للدالة؛ عند بناء طبقة Keras استخدم `add_weight` كي تتعقّب المنظومة الحالة وتحفظها.

البنية	تمثيلها	مثال الاستخدام
Tensor كثيف	تخزين كل الخانات	صورة أو دفعة سمات
RaggedTensor	أطوال صفوف متفاوتة	جمل بعدد كلمات مختلف
SparseTensor	قيم غير صفرية مع فهارس	مصفوفة تفاعلات قليلة الكثافة
Variable	Tensor ذو حالة قابلة للتحديث	أوزان النموذج

البيانات المتناثرة لا تصبح أصغر دائمًا في كل عملية؛ بعض العمليات تحوّلها ضمنيًا إلى تمثيل كثيف. اختبر الذاكرة والدعم الفعلي قبل اختيارها.

3.7 حالة واقعية: دفعة صور لا تدخل النموذج

نموذج `Conv2D` الشائع في Keras يتوقع `(batch, height, width, channels)` افتراضيًا. إذا كان الملف يأتي من نظام آخر بترتيب `(batch, channels, height, width)`، افحص `shape` أولًا ثم بدّل المحاور بـ `tf.transpose`. لا تستخدم `reshape` لإخفاء اختلاف ترتيب البيانات؛ ستتغير دلالة البكسلات حتى لو بقي عددها ثابتًا.

3.8 فحص الأشكال كعقد بين مكونات النظام

أخطاء الأبعاد ليست مجرد رسائل مزعجة؛ إنها إخفاق عقد بين المحمّل والتحويل والنموذج. وثّق لكل مدخل اسمه ونوعه وشكله ووحداته ومداه. على سبيل المثال، الصورة ليست `(224,224,3)` فقط: هل القنوات RGB أم BGR؟ هل القيم `[0,255]` أم `[0,1]`؟ هل يعني `None` حجم دفعة متغيرًا؟ كتابة هذه التفاصيل تمنع أن تقبل الخدمة Tensor صحيح الشكل لكنه يمثل بيانات مختلفة.

المحور المجهول مفيد للدُفعات المتغيرة، لكن كل الأبعاد ليست ديناميكية تلقائيًا. داخل `tf.function` قد يتخصص الرسم لشكل أول استدعاء، ثم يعيد التتبع عند شكل جديد. استخدم `TensorSpec` أو `input_signature` عندما تعرف عقدًا مرنًا، وراقب عدد عمليات التتبع إذا كان الأداء يتذبذب. لا تضع عمليات تحويل إلى NumPy داخل المسار المطلوب تتبعه؛ ذلك قد يكسر التدرج أو يمنع الرسم.

```python
import tensorflow as tf

signature = tf.TensorSpec(shape=(None, 4), dtype=tf.float32)
@tf.function(input_signature=[signature])
def normalize_batch(values):
    tf.debugging.assert_all_finite(values, "المدخل يحتوي NaN أو Inf")
    return tf.math.l2_normalize(values, axis=-1)

print(normalize_batch(tf.ones((3, 4)))).shape)
print(normalize_batch(tf.ones((12, 4)))).shape)
# كلا الحجمين يعيدان رسمًا واحدًا يقبل دفعة متغيرة.
```

3.9 النوع العددي والدقة والذاكرة

`float32` مناسب لمعظم أمثلة التدريب، لكن `float16` و`bfloat16` قد يسرعان العمليات على عتاد يدعمهما. الأعداد الصحيحة مناسبة للفئات والفهارس، لا للضرب الحسابي مع أوزان عائمة دون تحويل صريح. عند الانتقال بين الأنواع، افحص مجال القيم: تحويل float إلى int يقتطع الجزء الكسري، والتحويل إلى دقة أقل قد يقرب الرقم أو يفيض. لا تعتمد على تحويل ضمني عند حدود الخدمة.

يتناسب تخزين Tensor كثيف تقريبًا مع عدد العناصر مضروبًا في بايتات العنصر، لكن الذاكرة أثناء التدريب تشمل التفعيلات والتدرجات وحالة المحسن. لهذا قد ينفد VRAM مع Tensor دخل يبدو صغيرًا. عند التخطيط، احسب حدًا أوليًا ثم قس الذروة باستخدام Sparseg profiler. Ragged يقللان تمثيل الخانات غير المستخدمة في حالات مناسبة، لكن بعض العمليات قد تنشئ تمثيلًا كثيفًا داخليًا.

3.10 قناع الحشو في التسلسلات

حين تجمع جملًا متفاوتة الطول في Tensor مستطيل، تملأ الذيل برمز padding. هذا الرمز ليس كلمة حقيقية؛ إذا دخل في المتوسط أو الانتباه دون قناع، يتعلم النموذج طول الدفعة أو محتوى الحشو بدل النص.

`Embedding(mask_zero=True)` يمكن أن يولد قناعًا في بنى Keras التي تمرره، لكن عليك التحقق من أن الطبقات اللاحقة تحافظ عليه. اختبر جملة قصيرة وحدها ومع حشو إضافي، وقارن التمثيل قبل النشر.

3.11 أدوات تشخيص سريعة

العرض	الفحص	التصحيح المحتمل
InvalidArgumentError في الجمع	اطبع الشكلين والنوعين	أضف محورًا مقصودًا أو أصلح المصدر
عدد عناصر مختلف في reshape	احسب حاصل ضرب الأبعاد	استعمل `-1` لمحور واحد فقط
تتبع متكرر	ثبات نوع وشكل الإدخال	حدد `input_signature` أو قلل التنوع
ذاكرة GPU ممتلئة	حجم الدفعة والتفعيلات	خفّض الدفعة أو استخدم mixed precision
أداء يتغير مع الحشو	اختبر القناع	مرر mask وتأكد من استهلاكه

3.12 البث بالتفصيل: محور مفقود لا يساوي قيمة مفقودة

في البث، نقارن الأبعاد من اليمين إلى اليسار. `(5,1,3)` مع `(1,4,1)` ينتجان `(5,4,3)`: كل محور إما متساوٍ أو أحد جانبيه واحد. أما `(5,3)` مع `(5,)` فيحاذي 3 مع 5 من جهة اليمين، ويفشل غالبًا رغم أن المستخدم قصد جمع قيمة لكل صف. الحل هو إعادة شكل المتجه إلى `(5,1)` أو تحديد محور الميزات المقصود. لا تضف `expand_dims` قبل أن تشرح معنى المحور.

البث يوفر الذاكرة لأنه لا يكرر القيمة فعليًا في Tensor جديد بالضرورة، لكنه يوسع الحساب على عناصر كثيرة. عملية تطبيع غير مقصودة قد تولد مصفوفة هائلة أو تعطي نتيجة صحيحة الشكل وخاطئة المعنى. اكتب اختبارًا بأعداد صغيرة تستطيع حسابه يدويًا، ثم انتقل لحجم الإنتاج.

3.13 المتغيرات وسجل الحالة

`tf.Variable` يحتفظ بقيمة قابلة للتحديث، وله خصائص مثل `trainable` و`dtype` و`shape`. أوزان Keras تتبعها الطبقة وتحفظها، لكن Tensor عادي محفوظ محليًا لا يصبح وزنًا مدربًا. في حلقة مخصصة، مرر قائمة المتغيرات القابلة للتدريب إلى `tape.gradient`، وتأكد أن التدرج ليس `None`. للمتغيرات غير القابلة للتدريب، أضفها عبر واجهة الطبقة المناسبة لكي تدخل checkpoint.

المورد `RaggedTensor` يخزن صفوفًا متفاوتة الطول بوضوح، بينما `SparseTensor` يمثل الفهارس والقيم والعقد الكلي. استخدم Ragged للنصوص غير المحشوة أو التسلسلات المتغيرة عندما تدعم الطبقات العملية، واستخدم Sparse لمصفوفة تفاعلات قليلة كثافة إن كانت العمليات اللاحقة تدعمه. إنشاء بنية متفرقة لا يضمن أن Dense أو accelerator سينفذها بكفاءة؛ قد يحدث تحويل داخلي إلى dense.

3.14 الجهاز والنقل

يمكن أن يكون Tensor على CPU أو GPU أو TPU. `tf.device` يطلب جهازًا، لكن TensorFlow يختار تلقائيًا عند عدم تحديده. نقل موتر كبير بين CPU وGPU يتطلب وقتًا وذاكرة؛ جمع البيانات الصغيرة متقطعة من CPU داخل كل خطوة قد يهدر فائدة GPU. جهز دفعة متصلة عبر `tf.data` وحافظ على تدفق نقل واضح. لا تثبت مكان كل عملية يدويًا إلا لسبب قياس، فقد تمنع المجدول من اختيار تنفيذ أفضل.

3.15 اختبار خصائص الأشكال

اكتب assertions عند حدود البيانات: عدد الصفوف، ترتيب المحاور، النوع، المجال، وخلو القيم من NaN. اختبار property بسيط يجرب أكثر من حجم batch ويضمن أن النموذج لا يعتمد على حجم ثابت. عند خلط Ragged وسلاسل padded، افحص mask في كل طبقة. هذه الاختبارات أرخص من اكتشاف خطأ shape بعد ساعات تدريب أو داخل API.

3.16 مثال حساب يدوي للبث

لنفترض مصفوفة قياسها `(2,3)` وصف متوسط `(3,)`. يجمع TensorFlow المتوسط مع كل صف لأن آخر محور متوافق؛ كل عمود يمثل ميزة واحدة. إذا كان لكل صف عتبة مختلفة، فلا تستخدم `(2,)` مباشرة، بل شكل `(2,1)` كي يمد العمود على محور الميزات. عينة تحقق صغيرة تكشف المقصود:

```python
import tensorflow as tf

scores = tf.constant([[0.2, 0.8, 0.4], [0.9, 0.1, 0.5]])
row_thresholds = tf.constant([[0.5], [0.3]])
decisions = scores ⩾ row_thresholds
print(decisions.numpy())
```

3.17 أمثلة تنفيذية إضافية

تبديل محاور الصورة

```python
import tensorflow as tf
x = tf.zeros((4, 3, 32, 32))
y = tf.transpose(x, (0, 2, 3, 1))
print(y.shape)  # (4, 32, 32, 3)
```

تسلسلات متباينة الطول

```python
import tensorflow as tf
tokens = tf.ragged.constant([[4, 7, 2], [9, 1]])
print(tokens.row_lengths().numpy())  # [3 2]
```

3.18 توثيق عقد البيانات

احفظ مثال Tensor مرجعيًا وقيمته المتوقعة، واختبره على CPU وGPU. وثق التحويلات والنوع والمجال، وتحقق من أن preprocessing لا تستعمل label أو معلومة مستقبلية. في مشروع فريق، العقد المكتوب يوفر على كل مستخدم استنتاج ما يعنيه كل محور.

3.19 فهرسة آمنة للأمثلة

فرّق بين `x[i]` الذي يزيل محور batch لأول مثال و `x[i:i+1]` الذي يحافظ عليه. عند إرسال دفعة واحدة إلى خدمة، يبقى الشكل `(1,features)` بينما المثال المنفرد `(features,)`. اختبر الدالتين عند حدود التدريب والاستدلال حتى لا ينجح مسار ويفشل آخر.

3.20 سجلات المتغيرات

عند تغيير `Variable.assign`، تأكد من أن الدلالة توافق نوع الحالة. `tf.Variable` قد يكون scalar أو مصفوفة، ولا يحفظ وحده على القرص إلا ضمن checkpoint أو Keras layer. سجل القيم الابتدائية وإصداراتها إذا كانت جزءًا من reproducibility.

3.21 توحيد أنواع البيانات

عند جمع صورة float مع mean من int، حوّل النوع بقرار معلن. استخدم `tf.cast` فقط بعد تحديد معنى التحويل، وافحص القيم بعده. خلط `float16` و `float32` قد يسبب تحويلًا ضمنيًا أو خطأ dtype؛ صرّح بالنوع عند حد كل عملية.

3.22 أخطاء شائعة

> ⚠ **تحذير**
>
> `(32, 784)` دفعة من 32 مثالًا، بينما `(784,)` مثال واحد مسطّح. البعد الأول قد يمثل الدفعة ولا يدخل في [!DANGER] . `Input(shape=...)` تحويل نوع الأعداد بصمت قد يرفع الذاكرة أو يسبب عدم توافق. صرّح بـ `dtype` وافحصه عند حدود الإدخال.

3.23 تمارين

1. احسب رتبة وشكل Tensor يحوي 12 عنصرًا أعيد تشكيله إلى `(3, 2, 2)`. (المتوقع: الرتبة 3 و12 عنصرًا.)
2. اكتب فهرسة تستخرج القناة الثانية من دفعة صور. (المتوقع: شكل `(batch, height, width)`.)
3. اجمع انحيازًا من 4 ميزات على دفعة من 10 أمثلة. (المتوقع: `(10, 4)`.)
4. فسّر الفرق بين `stack` و `concat` على مصفوفتين. (المتوقع: يزداد عدد المحاور في الأولى.)

5. صحّح ترتيب محاور صورة قادمة بقنوات أولًا. (المتوقع: `(N, H, W, C)`.)
6. اختر Ragged أو Sparse لثلاث حالات وعلّل. (المتوقع: قرار مرتبط ببنية البيانات وكثافتها.)

3.24 ملخص

- اقرأ الرتبة والشكل والنوع والجهاز معًا.
- استخدم `transpose` لتبديل المحاور و `reshape` لتغيير التقسيم مع حفظ ترتيب العناصر.
- البث مفيد لكنه قد يخفي خطأً دلاليًا: افحص الأشكال عند حدود المكونات.

الجزء الأول · الأساسات: كيف تفكّر في TensorFlow؟

الفصل الرابع

04

الاشتقاق التلقائي والرسوم البيانية

حكاية الفصل

في شركةٍ للتوصيل، بنى فريقُ التوقّعات نموذجًا يتنبّأ بعدد الطلبات في الساعة. في الأسبوع الأول، كانت الخسارة تنخفض بانتظام، ثم توقّفت عند رقمٍ ثابت لا يتحرّك. أضاف مهندسٌ شابٌّ سطرًا واحدًا يطلب مراقبة المدخل، فعاد النموذج إلى التعلّم. بعد أيام، اشتكى الفريق من أنّ الحقبة الواحدة صارت تستغرق ثماني دقائق بدلًا من دقيقة، مع أنّ النموذج والبيانات لم يتغيّرا. كان الخطأ في مكانٍ آخر تمامًا: دالةٌ تُعرَّف من جديد في كل خطوة. فكيف يحسب TensorFlow المشتقة أصلًا؟ ولماذا يتحوّل الكود نفسه من سريعٍ إلى بطيء؟

4.1 أهداف الفصل

- [ ] فهم المشتقة وقاعدة السلسلة بلغةٍ عملية، لا برموزٍ مجرّدة.
- [ ] استخدام `tf.GradientTape` بثقة: المراقبة، والاشتقاق، والاشتقاق الثاني، والجاكوبي.
- [ ] بناء حلقة تدريب يدوية، ثم اشتقاق دالةٍ مركّبة والتحقق من النتيجة عدديًا.
- [ ] فهم التتبّع (tracing) في `tf.function`، وضبط `input_signature` و `reduce_retracing`.
- [ ] التمييز بين التنفيذ الفوري (eager execution) والرسم البياني (graph)، ومعرفة متى يفيد XLA.

4.2 المشتقة: لغة التغيير

تخيّل أنك تدير أسطولَ درّاجاتٍ للتوصيل، وتسأل: إذا أضفتُ سائقًا واحدًا، كم تزيد تكلفةُ الساعة؟ المعدّلُ الذي يجيب عن هذا السؤال هو **المشتقة (derivative)**: مقدارُ تغيّر المخرَج مقابل تغيّرٍ صغير جدًا في المدخل. لا يهمّنا التغيّرُ الكبير، لأنّ سائقًا واحدًا قد يُغيّر جدولَ العمل كله؛ يهمّنا الميلُ اللحظي عند نقطةٍ محدّدة.

رياضيًا، إذا كانت f قابلةً للاشتقاق عند x_0، فمشتقتها هناك هي نهاية معدّل التغيّر:

$$\frac{df}{dx}(x_0) = \lim_{h \to 0} \frac{f(x_0 + h) - f(x_0)}{h}$$

هنا x_0 النقطةُ التي نقيس عندها، وh إزاحةٌ صغيرة، و$f(x_0 + h) - f(x_0)$ التغيّرُ في المخرَج. القسمةُ على h تحوّل التغيّرَ إلى «تغيّرٍ لكل وحدة»، والنهايةُ تُصغّر h حتى يختفي أثرُ التقريب.

لكن الشبكةَ العصبية ليست دالةً واحدة، بل دالةٌ داخل دالة: طبقةٌ تُغذّي طبقة، وخسارةٌ تجمع مخرجات الطبقة الأخيرة. هنا نحتاج **قاعدة السلسلة (chain rule)**:

$$\frac{d}{dx} f\big(g(x)\big) = f'\big(g(x)\big) \cdot g'(x)$$

$g(x)$ هي القيمةُ الداخلية، وf' مشتقةُ الدالة الخارجية محسوبةً عند تلك القيمة، و$g'(x)$ مشتقةُ الدالة الداخلية. القاعدة تقول باختصار: **مشتقة الخارج × مشتقة الداخل**. وإن كانت $y = f(g(h(x)))$، كرّرنا الضربَ ثلاث مرات. الشبكةُ العميقة ليست إلا هذا التكرار مئات المرات.

لماذا لا نحسب المشتقات يدويًا؟ لأن عدد الوسائط قد يبلغ الملايين، ولكلّ وسيطٍ مشتقةٌ جزئية، ثم نجمعها في متجهٍ واحد هو **التدرّج (gradient)** الذي يشير إلى اتجاه أسرع زيادةٍ في الخسارة. السيرُ عكسه هو جوهر التدريب. الحسابُ اليدوي ممكن لشبكةٍ من طبقتين، ومستحيل لشبكةٍ من مئة طبقة. لذلك يحتاج TensorFlow إلى آلةٍ تحسب المشتقة نيابةً عنك، بدقّة الرموز وسرعة الأرقام.

مثال عددي: لماذا يشير التدرّج إلى تعديل الوزن؟

لنفترض أن النموذج يقدّر أجرة توصيل خطية: `prediction = weight × distance` . لمسافة مقدارها 2، ووزن حالي يساوي 3، وتكلفة صحيحة تساوي 10، يكون التنبؤ 6 والخطأ `6 - 10 = -4` . نستخدم مربع الخطأ، فتكون الخسارة 16. مشتقة الخسارة بالنسبة إلى الوزن هي `2 × (-4) × 2 = -16` ؛ والإشارة السالبة تقول إن زيادة الوزن قليلًا تخفض الخسارة عند هذه النقطة.

```python
import tensorflow as tf

weight = tf.Variable(3.0)
distance = tf.constant(2.0)
target = tf.constant(10.0)

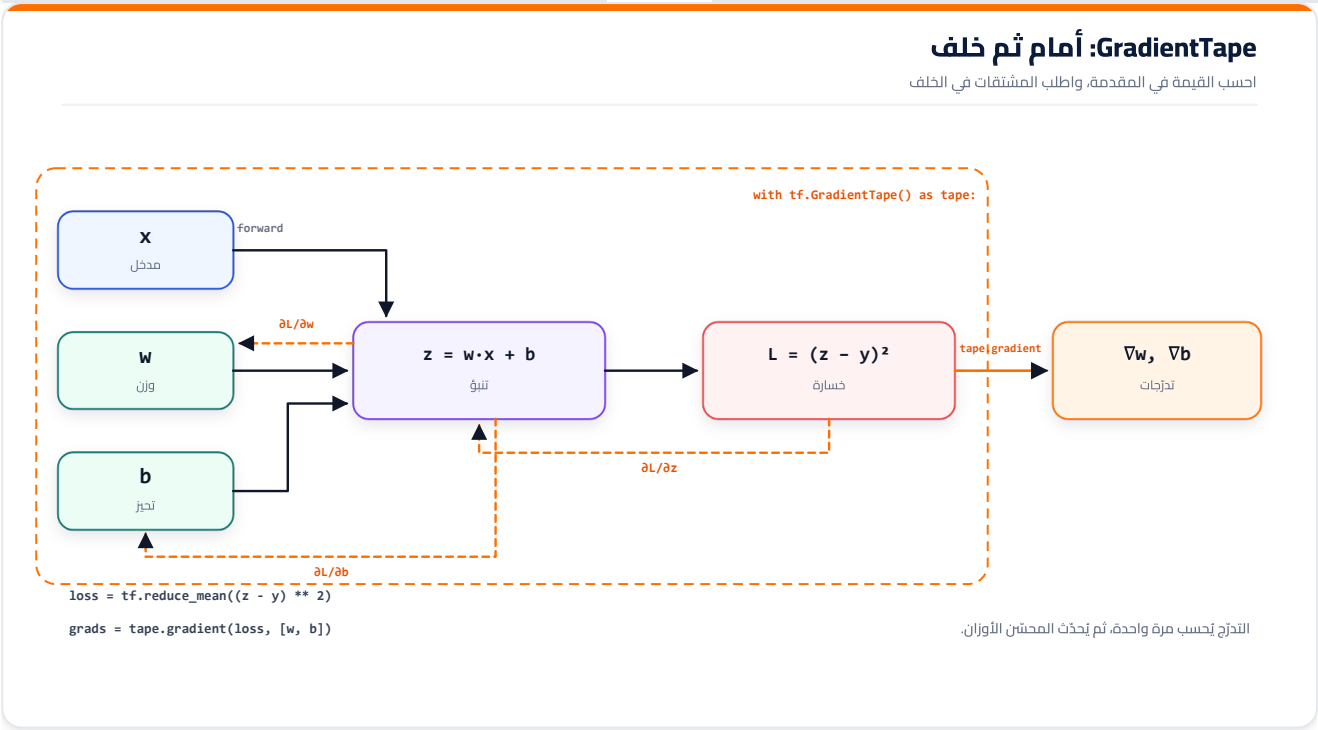
with tf.GradientTape() as tape:
    prediction = weight * distance
    loss = tf.square(prediction - target)

gradient = tape.gradient(loss, weight)
print("prediction:", prediction.numpy())  # 6.0
print("loss:", loss.numpy())              # 16.0
print("d(loss)/d(weight):", gradient.numpy())  # -16.0
```

لو طبقنا خطوة تحديث بمعدل تعلم `0.1`، يصبح الوزن `3 - 0.1 × (-16) = 4.6`، أي يتحرك في اتجاه يقلل الخسارة المحلية. لكن هذا المعدل كبير نسبيًا وقد يتجاوز أفضل قيمة؛ لذلك يضبط المحسّن حجم الخطوة، ويتكرر التحديث على أمثلة كثيرة بدل الاعتماد على عينة واحدة.

4.3 الشريط: كيف يتذكّر TensorFlow؟

في كل مرة تُنفّذ عمليةً على تنسور، لا يحسب TensorFlow المشتقةَ فورًا. بل يسجّل العمليةَ ومدخلاتها على **شريط (tape)**، تمامًا كما يسجّل محاسبٌ كلَّ حركةٍ في دفتره. بعد انتهاء الحساب الأمامي، يمرّ على الدفتر بالعكس: من الخسارة إلى الأوزان، ويطبّق قاعدة السلسلة عند كل خطوة. هذه الطريقة تُسمّى **الاشتقاق العكسي (reverse-mode automatic differentiation)**، وكلفتُها قريبةٌ من كلفة تمريرةٍ أمامية واحدة، مهما كان عدد الوسائط. لهذا تستطيع تدريبَ نموذجٍ بملايين المعاملات بحسابٍ واحدٍ للتدرّج.



شكل 4-1 الاشتقاق التلقائي: تمرير أمامي وخلفي عبر الرسم البياني

للشريط قواعد قليلة، لكنّها مصدرُ معظم الأخطاء:

1. **المتغيّرات تُراقَب تلقائيًا.** أي `tf.Variable` تدخل في الحساب يُسجَّل تدرّجها دون طلب.
2. **الثوابت لا تُراقَب.** إن أردت الاشتقاق بالنسبة إلى تنسورٍ عادي، فعليك `tape.watch(x)` صراحةً قبل الحساب.
3. **الشريط غير الدائم يُستهلك مرة واحدة.** استدعاء `tape.gradient` مرةً ثانية يرفع خطأً، إلا إذا أنشأته بـ `persistent=True` ثم حرّرته بـ `del`.
4. **الاشتقاق الثاني يحتاج شريطين متداخلين.** الخارجي يراقب حسابَ التدرّج نفسه.

والآن لنرَ ذلك في كودٍ واحد:

```python
import tensorflow as tf
import keras

keras.utils.set_random_seed(42)

# (1) متغيّر: الشريط يراقبه تلقائيًا
x = tf.Variable(2.0)
with tf.GradientTape() as tape:
```

```
    y = x ** 3 + 2.0 * x            # y = 12 عند x = 2
print("dy/dx =", tape.gradient(y, x).numpy())   # 14.0  (3x² + 2)

# (2) ثابت: لا يُراقَب إلا بطلبٍ صريح
c = tf.constant(3.0)
with tf.GradientTape() as tape:
    y2 = c ** 2
print("بلا watch:", tape.gradient(y2, c))       # None

with tf.GradientTape() as tape:
    tape.watch(c)
    y2 = c ** 2
print("مع watch:", tape.gradient(y2, c).numpy())  # 6.0

# (3) شريط دائم: يسمح بأكثر من اشتقاق
with tf.GradientTape(persistent=True) as tape:
    a = tf.Variable(1.5)
    f = a ** 2
    g = a ** 3
print("df/da =", tape.gradient(f, a).numpy())   # 3.0
print("dg/da =", tape.gradient(g, a).numpy())   # 6.75
del tape                                        # حرّر الموارد

# (4) المشتقة الثانية: شريطان متداخلان
x2 = tf.Variable(2.0)
with tf.GradientTape() as outer:
    with tf.GradientTape() as inner:
        y3 = x2 ** 3
    first = inner.gradient(y3, x2)              # 12.0
second = outer.gradient(first, x2)
print("d²y/dx² =", second.numpy())              # 12.0

# (5) جاكوبي: دالة من R² إلى R³
v = tf.Variable([2.0, 3.0])
with tf.GradientTape() as tape:
    out = tf.stack([v[0] ** 2, v[0] * v[1], v[1] ** 3])
J = tape.jacobian(out, v)
print("shape =", J.shape)                       # (3, 2)
print(J.numpy())
# [[ 4.  0.]
#  [ 3.  2.]
#  [ 0. 27.]]
```

لاحظ الفرق بين `gradient` و`jacobian`: الأول يعيد متجهًا بحجم المدخل عندما يكون المخرَج رقمًا واحدًا (خسارةً مثلًا)، والثاني يعيد مصفوفةً كاملة عندما يكون المخرَج متجهًا. في التدريب نستخدم الأول غالبًا، لأنّ الخسارة عددٌ واحد؛ أمّا الثاني فيظهر في حساب مصفوفة المعلومات أو في تحليل حساسية مخرجاتٍ متعدّدة.

4.4 من الاشتقاق إلى التدريب

الآن لنُغلق الحلقة. أمامك مهمةٌ بسيطة: شركةُ شحنٍ تريد معادلةً تربط وزنَ الطرد بتكلفة توصيله. العلاقة خطية تقريبًا، لكنّ القياسات تحمل ضجيجًا. نموذجنا:

$$\hat{y} = wx + b$$

حيث $\hat{y}$ التكلفةُ المتوقّعة، وx الوزن، وw سعرُ الكيلوغرام، وb رسمٌ ثابت. نُدرّب النموذج بتقليل متوسط مربع الخطأ:

$$L(w, b) = \frac{1}{N} \sum_{i=1}^{N} \left(w x_i + b - y_i \right)^2$$

N عددُ الطرود، وy_i التكلفةُ الحقيقية للطرد i. الأسُّ التربيعي يعاقب الأخطاء الكبيرة بشدّة، والمتوسط يجعل الخسارةَ مستقلةً عن حجم الدفعة.

في Keras المعتاد، يتولّى `fit` كلَّ شيء. لكن لفهم ما يجري، سنكتب الحلقةَ بأيدينا:

```python
import numpy as np
import tensorflow as tf
import keras

keras.utils.set_random_seed(42)

# بيانات: تكلفة الشحن تزيد خطيًا مع الوزن، مع ضجيج بسيط
rng = np.random.default_rng(42)
x = rng.uniform(0.5, 5.0, size=(200, 1)).astype("float32")
y = 4.0 * x + 2.5 + rng.normal(0.0, 0.8, size=(200, 1)).astype("float32")

# وسطان يُتعلّمان: نبدأ من الصفر
w = tf.Variable(0.0, name="w")
b = tf.Variable(0.0, name="b")
optimizer = keras.optimizers.SGD(learning_rate=0.02)

for epoch in range(200):
    # كل خطوة تحتاج شريطًا جديدًا: الشريط يُستهلك بعد gradient
```

```
    with tf.GradientTape() as tape:
        y_pred = w * x + b
        loss = tf.reduce_mean(tf.square(y_pred - y))

    # التدرّج بالنسبة إلى الوسيطين معًا
    dw, db = tape.gradient(loss, [w, b])

    # المحسّن يطبّق: variable -= learning_rate * gradient
    optimizer.apply_gradients(zip([dw, db], [w, b]))

    if epoch % 50 == 0:
        print(f"epoch {epoch:3d} | loss = {loss.numpy():.3f}")

print("w =", float(w.numpy()), "| b =", float(b.numpy()))
# المتوقع: loss ينخفض من نحو 200 إلى أقل من 1،
# w يقترب من 4.0، و b يقترب من 2.5
```

ثلاثة أسطر تستحقّ التوقّف. `tape.gradient(loss, [w, b])` يعيد قائمةً بنفس ترتيب الطلب. `zip` يزاوج كلَّ تدرّجٍ مع متغيّره. و`apply_gradients` لا يعرف شيئًا عن الشبكات؛ إنّه فقط يقرأ التدرّج ويُحدّث المتغيّرات. هذا هو التدريب في أعمق صوره: لا أكثر.

والآن دالةٌ مركّبة، حيث تظهر قاعدة السلسلة بوضوح. لتكن $h(x) = \sin(x^2)\, e^{x/3}$. لا نعرف مشتقّتها بالحدس، لكن يمكننا التحقق من الإجابة بطريقتين: الشريط، والفرق العددي:

python

```
import tensorflow as tf

def h(x):
    return tf.sin(x ** 2) * tf.exp(x / 3.0)

x0 = tf.Variable(1.2)
with tf.GradientTape() as tape:
    value = h(x0)
dy = tape.gradient(value, x0)

# تحقق عددي: (f(x+ε) - f(x-ε)) / 2ε
eps = 1e-3
numeric = (h(tf.constant(1.2 + eps)) - h(tf.constant(1.2 - eps))) / (2.0 * eps)

print("autodiff =", round(float(dy), 6))      # 0.95999
print("numeric  =", round(float(numeric), 6))  # 0.95999 تقريبًا
# المتوقع: القيمتان متطابقتان إلى خمس منازل
```

التطابق ليس صدفة: الشريط طبّق قاعدة السلسلة على $\sin$ وx^2 و$\exp$ بالترتيب، والفرقُ العددي قرّب النهايةَ نفسها. الفرقُ أنّ الأول دقيقٌ وسريع، والثاني يعاني خطأَ التقريب ويتطلّب استدعاءين لكلّ مشتقة. في الشبكات الكبيرة، الفرقُ بين الطريقتين هو الفرق بين دقائق وأسابيع.

4.5 tf.function: الرسم البياني الذي يُبنى مرة واحدة

حتى الآن، كان TensorFlow ينفّذ كلَّ سطرٍ فور كتابته. هذا مريح للتصحيح، لكنه مكلف: كلُّ عمليةٍ صغيرة تُرسَل إلى المحرّك على حدة، فيضيع وقتٌ كبير في التنسيق بدل الحساب. الحلُّ هو `tf.function`: تُغلّف الدالة، فينفّذها TensorFlow مرةً واحدة لتسجيل عملياتها في **رسم بياني (graph)**، ثم يعيد استخدام الرسم في الاستدعاءات التالية.



شكل 4-2 tf.function: من التنفيذ الفوري إلى الرسم البياني

تلك المرّة الأولى تُسمّى **التتبّع (tracing)**. أثناءها يمرّ بايثون على أسطر الدالة، ويُحوّل كلَّ تنسورٍ إلى موضعٍ في الرسم. وبعدها لا يعود بايثون إلى العمل؛ الرسمُ وحدَه هو الذي يُنفَّذ. من هنا تأتي قاعدةٌ ذهبية: **الأشكال المختلفة تعني تتبّعًا جديدًا**. استدعِ الدالة بدفعةٍ من 32 صفًا، ثم بـ64، فتحصل على رسمين. استدعِها بألف شكلٍ مختلف، فتحصل على ألف رسم، وذاكرةٌ تنتفخ بلا فائدة.

المعيار	التنفيذ الفوري (eager)	الرسم البياني داخل `tf.function`
سهولة التصحيح	عالية: كل سطر يُنفَّذ ويمكن طباعته	أقل: الأخطاء تظهر وقت التتبّع
سرعة الحلقات الكثيفة	أبطأ بسبب كلفة الإرسال لكل عملية	أسرع: العمليات تُدمج وتُنفَّذ كوحدة
المرونة في الأشكال	كل استدعاء حرّ	شكل جديد = تتبّع جديد
آثار بايثون (`print` مثلًا)	تُنفَّذ كل مرة	تُنفَّذ مرة واحدة وقت التتبّع

هذا الجدول يفسّر مفارقةً يقع فيها كثيرون: يضعون `print(loss)` داخل دالةٍ مُغلّفة، فيرون الرقم مرةً واحدة ثم يتوقّفون عن رؤيته. ليست الخسارة ثابتة؛ الطباعةُ هي التي نُفّذت مرةً واحدة. للحصول على طباعةٍ في كل خطوة، استخدم `tf.print` التي تتحوّل إلى عمليةٍ داخل الرسم.

ولضبط التتبّع، هناك أداتان: `input_signature` تُعلن مسبقًا شكلَ المدخل ونوعه، فيُبنى رسمٌ واحد يقبل أي حجمٍ في البُعد المعلن `None` . و `reduce_retracing=True` تسمح لـ TensorFlow بتخفيف الأشكال تلقائيًا بدل بناء رسمٍ لكل حجم. أمّا **AutoGraph** فهو المحوّل الذي يجعل حلقات `for` وشروط `if` في بايثون تعمل داخل الرسم، ويمكنك رؤية الناتج بـ `tf.autograph.to_code(my_function)` .

python

```python
import tensorflow as tf
import keras

keras.utils.set_random_seed(42)

traces = {"count": 0}

@tf.function
def scale(x):
    traces["count"] += 1                # يُنفَّذ وقت التتبّع فقط
    tf.print("trace:", traces["count"])  # tf.print يُنفَّذ وقت التشغيل
    return x * 2.0 + 1.0

print(scale(tf.constant([1.0, 2.0])).numpy())        # [3. 5.]
print(scale(tf.constant([1.0, 2.0, 3.0])).numpy())   # [3. 5. 7.]
print("عدد التتبعات =", traces["count"])             # 2

# بصمة ثابتة: شكل مجهول في البُعد الأول، فتتبع واحد يكفي
traces2 = {"count": 0}

@tf.function(input_signature=[tf.TensorSpec(shape=(None,), dtype=tf.float32)])
def scale_fixed(x):
```

```
    traces2["count"] += 1
    return x * 2.0 + 1.0

scale_fixed(tf.constant([1.0]))
scale_fixed(tf.constant([1.0, 2.0, 3.0]))
print("عدد التتبعات =", traces2["count"])            # 1

# reduce_retracing: سمح بتخفيف الأشكال بدل رسمٍ لكل حجم
@tf.function(reduce_retracing=True)
def normalize(x):
    return (x - tf.reduce_mean(x)) / (tf.math.reduce_std(x) + 1e-7)

print(normalize(tf.constant([1.0, 2.0, 3.0])).numpy())
# المتوقع: [-1.2247  .0  1.2247]
```

إن احترت في سلوكٍ غريب، فعّل التنفيذ الفوري داخل الدوال مؤقتًا بـ `tf.config.run_functions_eagerly(True)`. سيتوقّف التتبّع، وسترى كلَّ سطرٍ يعمل، لكنّ الأداء سينخفض. استخدمها للتشخيص ثم أعدها إلى `False`؛ لا تتركها في كود الإنتاج.

4.6 XLA: حين يُعاد تشكيل الرسم

بعد أن صار لدينا رسمٌ بياني، يمكننا أن نطلب من مُصرّف **XLA** (اختصار Accelerated Linear Algebra) أن يُعيد كتابته. الفكرةُ أنّ العمليات الصغيرة المتتالية، مثل `x * 2 + 1` ثم `reduce_mean` ثم `square`، تُدمج في نواةٍ واحدة تُنفَّذ دفعةً واحدة، فتنخفض زياراتُ الذاكرة. هذا هو **اندماج العمليات (operator fusion)**، وهو أكبر مكسبٍ يأتي من XLA.

تفعيلُه بسيط: `tf.function(jit_compile=True)`، أو `model.compile(..., jit_compile=True)` في Keras. لكنّه ليس مفتاحًا سحريًا:

- **يفيد** عندما تكون العمليات كثيرة وصغيرة، أو عندما تكون الذاكرة هي عنق الزجاجة، أو على مسرّعاتٍ تدعم XLA جيدًا.
- **لا يفيد** عندما يكون الوقت ضائعًا في قراءة البيانات أصلًا، أو عندما تتغيّر الأشكال كلَّ دفعة فتُعاد الترجمة، أو عندما تحتوي الدالة عمليةً لا يدعمها XLA.
- **أول استدعاء يكون أبطأ** لأنّ الترجمة تحدث مرةً واحدة. لذلك لا تحكم على الأداء من قياسٍ واحد.

```python
import time
import tensorflow as tf

# دالة كثيفة: ضرب مصفوفتين كبيرتين
@tf.function
def matmul_graph(a, b):
    return tf.linalg.matmul(a, b)

@tf.function(jit_compile=True)
def matmul_xla(a, b):
    return tf.linalg.matmul(a, b)

a = tf.random.normal((1024, 1024))
b = tf.random.normal((1024, 1024))

# تسخين: أول استدعاء يبني الرسم ويُصرّف النواة
matmul_graph(a, b)
matmul_xla(a, b)

def bench(fn, repeats=20):
    start = time.perf_counter()
    for _ in range(repeats):
        fn(a, b)
    return (time.perf_counter() - start) / repeats

print("graph:", round(bench(matmul_graph) * 1000, 2), "ms")
print("xla  :", round(bench(matmul_xla) * 1000, 2), "ms")
# المتوقع: رقمان متقاربان على CPU؛ الفرق يظهر أوضح
# على GPU أو مع سلاسل طويلة من العمليات الصغيرة
```

لاحظ أنّ القياس يشمل كلفة استدعاء بايثون، وهي جزءٌ من الحقيقة العملية لكنها تحجب الفرق أحيانًا. في القياس الجدّي، استخدم `tf.profiler` أو كرّر العملية داخل الدالة نفسها حتى يصبح الحساب هو الغالب.

4.7 حالة واقعية: ثماني دقائق بدل دقيقة

عاد فريقُ التوصيل إلى الكود. كانوا قد كتبوا حلقةً مخصّصة على نمط ما رأيناه، ثم قرّروا الاستفادة من الرسم البياني، فوضعوا `@tf.function` على دالة الخطوة **داخل** الحلقة:

```python
for batch in dataset:
    @tf.function                # خطأ: زخرفة جديدة في كل تكرار
    def train_step(batch):
        ...
```

كلُّ تكرارٍ يُنشئ كائنًا جديدًا، فيُبنى رسمٌ جديد، وتُستهلك الذاكرة، ويصبح الزمنُ تراكميًا. الحلُّ كان نقلَ الدالة خارج الحلقة، وإضافة `input_signature` لتثبيت شكل الدفعة. عاد الزمنُ إلى دقيقة، ثم انخفض أكثر بعد إزالة `print` من داخل الخطوة واستبداله بـ `tf.print` كلّ مئة خطوة. لم يتغيّر النموذج؛ تغيّر مكانُ التعريف فقط.

هذه هي القاعدة: **الرسم يُبنى مرة، والحلقة تدور كثيرًا**. كلُّ ما يُنشأ داخل الحلقة من دوال أو طبقات أو أوزان يُنشأ آلاف المرات. ضعْ التعريف خارجها، واتركْ داخلها البياناتَ فقط.

4.8 أخطاء شائعة

⚠ تحذير

العرض: `tape.gradient` يعيد `None` بلا رسالة واضحة. **السبب:** التنسور المُشتق بالنسبة إليه ثابتٌ لم يُمرَّر إلى `tape.watch`، أو أنّه أُنشئ بعد انتهاء التسجيل. **الحل:** مرّر المدخلات كـ `tf.Variable`، أو استدعِ `tape.watch(x)` داخل سياق الشريط وقبل الحساب.

⛔ خطر

العرض: `RuntimeError: A non-persistent GradientTape can only be used to compute one set of gradients`. **السبب:** استدعيتَ `tape.gradient` مرتين على الشريط نفسه، أو حسبتَ تدرّجًا داخل حلقةٍ من نفس الشريط. **الحل:** أنشئ شريطًا جديدًا في كل خطوة، أو استخدم `persistent=True` مع `del tape` بعد آخر استخدام.

⚠ تحذير

العرض: الطباعة داخل `tf.function` تظهر مرةً واحدة، والزمن يتضاعف مع كل شكلٍ جديد. **السبب:** الخلط بين زمن التتبّع وزمن التشغيل، وبناء رسمٍ لكل حجم دفعة. **الحل:** استخدم `tf.print` للطباعة وقت التشغيل، و`input_signature` أو `reduce_retracing` لتثبيت عدد الرسوم.

4.9 تمارين

1. **اشتقاق يدوي:** احسب مشتقة $f(x) = 3x^2 + 5x - 2$ عند $x = 4$، ثم تحقّق بالشريط. (المتوقع: 29، وهو ما يعيده `tape.gradient`.)
2. **الثوابت:** اشتقّ $y = x^2$ بالنسبة إلى `tf.constant(3.0)` بلا `watch`. ماذا يعيد؟ أصلح الخطأ. (المتوقع: `None`، ثم 6.0 بعد المراقبة.)
3. **المشتقة الثانية:** احسب المشتقة الثانية لـ $f(x) = x^4$ عند $x = 1.5$ بشريطين متداخلين. (المتوقع: $12x^2$، لأنّ 27.0.)
4. **الجاكوبي:** احسب مصفوفة جاكوبي لـ $f(x, y) = (x + y,\ xy)$ عند $(2, 3)$. (المتوقع: مصفوفة 2×2 قيمتها `[[1, 1], [3, 2]]`.)
5. **التتبّع:** غلّف دالةً تطرح متوسط المدخل، واستدعها بثلاثة أحجام مختلفة، ثم أضف `input_signature`. قارن عدد التتبعات. (المتوقع: ثلاثة رسوم قبل التوقيع، ورسم واحد بعده.)
6. **XLA:** شغّل كود ضرب المصفوفات مع وبدون `jit_compile` خمس مرات، وسجّل أول استدعاء و subsequent. (المتوقع: أول استدعاء لـXLA أبطأ، ثم نتائج متقاربة أو أفضل قليلًا حسب العتاد.)
7. **تشخيص:** دالةٌ داخل `tf.function` تطبع الخسارة مرةً واحدة ثم تصمت. اشرح السبب واقترح حلّين. (المتوقع: الطباعة حدثت وقت التتبّع؛ الحل `tf.print` أو `tf.config.run_functions_eagerly` للتشخيص.)

4.10 ملخص

- المشتقة تقيس الميل اللحظي، وقاعدة السلسلة تضرب مشتقات الطبقات المتتالية.
- `tf.GradientTape` يسجّل العمليات ثم يمرّ عليها عكسيًا، فتصبح كلفة التدرّج قريبة من تمريرةٍ أمامية.
- المتغيّرات تُراقَب تلقائيًا، والثوابت تحتاج `watch`، والشريط غير الدائم يُستهلك مرةً واحدة.
- `tf.function` يبني رسمًا واحدًا ويعيد استخدامه؛ اختلاف الأشكال يعني تتبّعًا جديدًا.
- `input_signature` و`reduce_retracing` يضبطان عدد الرسوم، و`tf.print` يطبع وقت التشغيل.
- XLA يدمج العمليات الصغيرة، ويفيد حين يكون الحساب كثيفًا والذاكرة عنق الزجاجة.
- القاعدة العملية: عرّف الدوال والطبقات خارج الحلقة، وضعْ البيانات داخلها.

الجزء الأول · الأساسات: كيف تفكّر في TensorFlow؟

الفصل الخامس

05

Keras: من الفكرة إلى النموذج

> **حكاية الفصل**
>
> احتاج فريقٌ إلى تقدير زمن توصيل الطلب. بدأت الفكرة بثلاث طبقات، ثم ظهرت حاجة إلى فرع يتعامل مع نوع المركبة وآخر مع المسافة. تحوّل السؤال من «كيف أكتب طبقة؟» إلى «كيف أنظم النموذج وأقيسه وأحفظه؟». هنا تصبح Keras واجهة دورة العمل كاملة.

5.1 أهداف الفصل

- [ ] اختيار Sequential أو Functional أو subclassing وفق شكل الحساب.
- [ ] ربط طبقات النموذج بالأشكال والخسارة والمقياس.
- [ ] تنفيذ compile وfit وevaluate والحفظ.
- [ ] التمييز بين ملف `.keras` وواجهة export.

5.2 أي بنية تناسب الحساب؟

تسلسل خطي من المدخل إلى المخرج يناسبه `Sequential`. إذا كان هناك مدخلان أو تخطٍّ بين الطبقات أو أكثر من مخرج، يصف Functional API الرسم صراحة. أما subclassing فيفيد حين يحتاج النموذج سلوكًا مخصصًا، لكنه يزيد مسؤولية البناء والتسلسل والتصحيح.



شكل 5-1 مقارنة واجهات بناء نموذج Keras



شكل 5-2 دورة النموذج من الإنشاء إلى الحفظ

الأسلوب	نقاط القوة	الكلفة
Sequential	قصير وواضح للتتابع	لا يصف تفرعًا عامًا
Functional	رسم صريح متعدد المدخلات والمخرجات	يتطلب فهم تدفق الأشكال
Subclassing	تحكم كامل في السلوك	يتطلب عناية بالبناء والحفظ

```python
import keras
from keras import layers

inputs = keras.Input(shape=(8,), name="order_features")
x = layers.Dense(32, activation="relu")(inputs)
x = layers.Dropout(0.1)(x)
outputs = layers.Dense(1, name="delivery_minutes")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="mse", metrics=[keras.metrics.MeanAbsoluteError()])
model.summary()
```

5.3 الخسارة والمقياس يجيبان عن سؤالين مختلفين

في الانحدار البسيط، متوسط مربع الخطأ يكتب على الصورة التالية:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

`N` عدد الأمثلة، و `y_i` الهدف الحقيقي، و `ŷ_i` التنبؤ. تضاعف العقوبة عند تضاعف مقدار الخطأ.

الخسارة توجه التحسين بالتدرج، أما المقياس فيلخص جودة نحتاج إلى قراءتها. في الانحدار، متوسط مربع الخطأ يعاقب الخطأ الكبير أكثر من متوسط الخطأ المطلق. في تصنيف متعدد الفئات مع logits، استخدم خسارة تقبل `from_logits=True` بدل تطبيق softmax مرتين. شكل المخرج وعدد الوحدات جزء من العقد بين البيانات والخسارة.

```python
import numpy as np
import keras
from keras import layers

keras.utils.set_random_seed(42)
x = np.random.default_rng(42).normal(size=(256, 8)).astype("float32")
y = (2 * x[:, :1] - x[:, 1:2]).astype("float32")
model = keras.Sequential([layers.Input((8,)), layers.Dense(16, activation="relu"), layers.De
model.compile(optimizer="adam", loss="mse", metrics=["mae"])
history = model.fit(x, y, validation_split=0.2, epochs=5, batch_size=32, verbose=0)
print(model.evaluate(x[:32], y[:32], verbose=0))
# خسارة و MAE المتوقع: عدديان، وتتغير القيم مع التدريب.
```

يفصل `validation_split` جزءًا من البيانات التي مررتها، لكنه لا يصنع تقسيمًا زمنيًا أو طبقيًا مناسبًا تلقائيًا. جهّز `validation_data` صراحة عندما يعتمد تصميم المسألة على ترتيب زمني أو مجموعات متشابهة.

5.4 دورة الحفظ والنشر

ينشئ Keras 3، `model.export(path)` . في `.keras` احفظ النموذج القابل لإعادة التحميل بصيغة SavedModel للاستدلال؛ لا تخلط الملفين. احتفظ أيضًا بإعداد المعالجة المسبقة، وإصدارات الحزم، وعتبة القرار، لأن وزن النموذج وحده لا يحدد سلوك النظام.

```python
model.save("delivery.keras")
loaded = keras.models.load_model("delivery.keras")
loaded.export("delivery_savedmodel")
```

5.5 حالة واقعية: الطلبات بفرعين

قد يستخدم تقدير التوصيل مدخلًا عدديًا للمسافة ومدخلًا فئويًا لنوع المركبة. يبني Functional API فرعين، ثم يدمجهما قبل المخرج. يحفظ الفريق النموذج مع عقد المدخلات، ويرفض في الخدمة قيمًا غير محددة بدل تمريرها بصمت.

5.6 خط أساس كامل من البيانات إلى القياس

النموذج الذي يصنف قيمة لا يصبح تجربة قابلة للتكرار لمجرد أن `fit` انتهى. يجب تعريف تقسيم يطابق شكل الاستخدام، واختيار baseline واضح، وتسجيل معيار قبول قبل البحث في الطبقات. في تصنيف غير متوازن، قد يضلل `accuracy`؛ أضف precision وrecall أو AUC، واحسبها على التحقق لا على بيانات التدريب فقط. عند استخدام `validation_split`، تذكر أن Keras يأخذ جزءًا من المصفوفات المقدمة ولا يضمن تقسيمًا طبقيًا أو زمنيًا.

الـ callbacks تنظم أحداث التدريب. `EarlyStopping` يوقف عندما يتوقف المقياس عن التحسن، و`ModelCheckpoint` يحفظ أفضل حالة، و`TensorBoard` يسجل المنحنيات. راقب المقياس في الاتجاه الصحيح: `val_auc` يحتاج تعظيمًا بينما `val_loss` يحتاج تقليلًا. لا تحفظ «آخر نموذج» فقط إن كنت ستختار على أفضل حقبة، ولا تستخدم الاختبار لاختيار عدد epochs.

5.7 مدخلان ومخرجان في Functional API

عندما يكون تدفق الحساب رسمًا لا سلسلة، سمّ المدخلات والمخرجات. هذا يجعل توقع الخدمة أوضح ويقلل أخطاء تبديل القيم:

```python
import keras
from keras import layers

distance = keras.Input(shape=(1,), name="distance_km")
vehicle = keras.Input(shape=(), dtype="string", name="vehicle_type")
lookup = layers.StringLookup(output_mode="one_hot")
lookup.adapt(["bike", "car", "van"])
category = lookup(vehicle)
x = layers.Concatenate()([distance, category])
x = layers.Dense(32, activation="relu")(x)
minutes = layers.Dense(1, name="minutes")(x)
late_probability = layers.Dense(1, activation="sigmoid", name="late")(x)
model = keras.Model({"distance_km": distance, "vehicle_type": vehicle},
                    {"minutes": minutes, "late": late_probability})
```

في التطبيق الفعلي يجب تكييف `StringLookup` على قيم التدريب أو توفير مفرداته الثابتة، وتحديد معالجة قيم OOV. إذا كان هناك مخرجان، اكتب خسارة ومقياسًا لكل واحد، وافحص مساهمة كل هدف. أوزان الخسائر ليست مجرد معاملات تجميل؛ قد تجعل التحسين يركز على مهمة على حساب الأخرى.

5.8 الحفظ الآمن ودورة النشر

صيغة `.keras` تحفظ إعداد النموذج والأوزان وحالة المحسن عند توفرها. لا تحتوي شيفرة Python لتعريف الكائنات المخصصة؛ سجل الطبقات أو مرر `custom_objects` من شيفرة موثوقة عند التحميل. Keras الحديثة تستخدم وضع تحميل آمن افتراضيًا لرفض deserialization غير الموثوق لبعض الكائنات. لا تعطّل `safe_mode` لمجرد إسكات خطأ ملف مجهول.

افصل ملف إعادة التدريب عن artifact الاستدلال، وسجل بصمة النموذج ونسخة المعالجة. قبل إتاحة الإصدار، أعد تحميله في عملية جديدة وتحقق من أن مدخلات حقيقية تعطي نتائج مناظرة. النموذج المحفوظ دون أداة تحويل السمات أو ترتيب الفئات ليس منتجًا قابلًا للتشغيل.

5.9 عقود التدريب: خسارة، مقياس، ووزن عينة

حدد ما إذا كانت labels فهارس صحيحة أو متجهات one-hot، ثم اجعل شكل المخرج وخسارته مطابقين. Sparse categorical loss تقبل رقم فئة واحدًا لكل مثال، أما categorical loss فتتوقع توزيعًا على الفئات. عند تمرير logits أوقف softmax في المخرج واستعمل `from_logits=True`؛ وعند إخراج احتمالات استخدم خسارة متوافقة معها. هذا الاختيار يؤثر في الاستقرار العددي ويمنع تكرار softmax.

`sample_weight` يغير مساهمة الأمثلة في loss وmetrics إذا كانت المقاييس تدعمه. استخدمه لأهمية موثقة أو سياسة وزن، لا كحل آلي لكل عدم توازن. راقب مؤشرات الفئة النادرة منفصلة؛ متوسط الدقة قد يخفي collapse إلى الأغلبية. لو عندك عدة مخرجات، حدد `loss_weights` علنًا وسجل خسارة كل مخرج كي تعرف أي هدف لم يعد يتعلم.

5.10 بناء input pipeline مع Keras

`fit` يقبل NumPy arrays و`tf.data.Dataset` وPyDataset وغيرها. عند ملف بيانات كبير أو معالجة متوازية، Dataset يعطيك تحكمًا في القراءة والخلط والدفعة. حدد `steps_per_epoch` فقط عند حاجة معروفة؛ الخطأ فيه قد يوقف التدريب قبل مرور البيانات أو يعيد أمثلة غير محدودة. callback لا يحل محل تقسيم validation؛ جهز تحققًا مستقلاً يتوافق مع توزيع الاستخدام.

5.11 اكشف انهيار التعلم بالتشخيص

ارسم loss وmetric للتدريب والتحقق. إذا كان كلاهما سيئًا، افحص labels، شكل المخرج، learning rate، وfeature scaleg. إذا كان التدريب ممتازًا والتحقق ضعيفًا، افحص overfitting والتسرب واختلاف المجموعتين. إذا ظلت loss ثابتة، جرّب overfit على 16 مثالًا؛ يفترض أن يستطيع نموذج مرن تقليل خسارتها. فشل هذا الاختبار غالبًا خلل في تدفق gradients أو target mapping.

5.12 قابلية إعادة الإنتاج

ثبت البذور عبر `keras.utils.set_random_seed`، لكن وثق أن التكرارية قد تظل متأثرة بنوع الجهاز والعمليات غير الحتمية. احفظ config لكل تجربة: بنية النموذج، optimizer وbatch sizeg، learning rate، تقسيم البيانات، preprocessing، seed، ومقياس الاختيار. لا تخزن history بلا ربطه بإصدار الكود. الباحث يحتاج مقارنة عادلة؛ المطور يحتاج rollback وإعادة إنشاء.

5.13 Model subclassing ومتى تحتاجه

ابدأ بـ Sequential أو Functional لأنهما يقدمان config ورسمًا يسهل فحصه. انتقل إلى subclassing عندما تحتاج تدفقًا يعتمد حالة أو سلوكًا متكررًا لا يعبر عنه الرسم المعتاد. نفذ `get_config` إذا أمكن، وابنِ جميع المتغيرات مرة واحدة، واختبر serialization. المرونة الزائدة تجعل الرسم والتحويل إلى LiteRT أو Serving أصعب؛ لا تستخدم subclassing لمجرد أنه يبدو أكثر احترافًا.

5.14 فصل البحث عن التقييم النهائي

استخدم train لتعلم الأوزان، validationg لاختيار حجم الشبكة والعتبة والحقبة، وtestg للتقدير النهائي مرة بعد تثبيت الخطة. إذا جربت عشرات إعدادات النموذج على test، فهو لم يعد test مستقلًا. في بيانات زمنية أو طبية، قد يكون تقسيم الأمثلة عشوائيًا غير صالح؛ استخدم تاريخًا مستقبليًا أو GroupKFold بحسب وحدة الاستقلال. اذكر عدد الأمثلة والأشخاص والمصادر، لا نسبة التقسيم فقط.

5.15 Keras callbacks تعاقد وليست سحرًا

يحتاج EarlyStopping إلى اسم مقياس موجود في logs؛ خطأ `monitor="val_accuracy"` مع عدم تعريف metric قد يمنع الإيقاف المقصود. استخدم `mode="max"` لمقياس أعلى أفضل، و`restore_best_weights=True` كي لا تستعيد آخر أوزان بعد تدهور. ModelCheckpoint يحفظ الملف وفق

save format المطلوب. TensorBoard callback يكتب event logs؛ منذ TensorFlow 2.21 لم يعد TensorBoard اعتمادًا ضمنيًا في الحزمة، فثبته مباشرة في بيئة المشروع عند استعماله.

5.16 فحص قبل الإنتاج

اختبر inference على حالات حدية: دفعة من مثال واحد، دفعة كبيرة، قيمة OOV، عنصر مفقود، وطول تسلسل عند الحد. تحقق أن ترتيب أسماء المخرجات ثابت. قارن الأصل والمحمّل من `.keras`، ثم صدّر inference وقارن النتائج بعد التحويل ضمن tolerance مسجل. احفظ أيضًا tokenizer أو lookup vocabulary، feature schema، threshold، ونسخة preprocessing. الأثر الصحيح هو «نموذج + عقد + معالجة + إعداد قرار».

5.17 دورة تعلم بحسب المستوى

المبتدئ يطبق `fit` ويتعلم شكل البيانات والخسارة. المتوسط يقارن callbacksg Functional/Sequential وmetricsg على مشكلة معلومة. المحترف يخصص data pipeline وserializationg وCIg. الخبير يفحص contract عبر backend أو runtime ويحدد دلالات القياس وتكلفته. لا تنتقل إلى custom loop قبل أن تفسر سلوك `fit` على baseline.

5.18 أمثلة تنفيذية إضافية

إدارة التدريب عبر callbacks

```python
callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=2, restore_best_weights=True),
    keras.callbacks.ModelCheckpoint("best.keras", save_best_only=True),
]
# مرّر callbacks=callbacks إلى model.fit(...)
```

توقع دفعة

```python
predictions = model.predict(x[:4], verbose=0)
print(predictions.shape)
```

5.19 مواصفات التشغيل والتسليم

أرفق model summary واسم المدخل وشكل الدفعة والتحويلات thresholdg mappingg الفئات ونسخ الحزم ومقاييس validation. أنشئ smoke inference بمدخل اصطناعي fixtureg منزوع الحساسية. عند تحديث النموذج، قارن الإصدارات على مجموعة ثبات. المستهلك يحتاج عقدًا صغيرًا واضحًا لا دفتر التدريب كاملًا.

5.20 تحقق من التسمية والتوزيع

اطبع label وتوقعًا واحدًا بجانب المدخل قبل تشغيل fit. عند sparse labels، تأكد أن المعرفات تبدأ من المجال المتوقع ولا تحتوي قيمة خارج عدد الوحدات. اختبر mapping الفئات بعد reload، لأن ترتيب القائمة قد يتغير بين train وAPI.

5.21 استدلال بدفعات صغيرة

خدمة التنبؤ قد ترسل مثالًا أو مجموعة؛ اختبر الحالتين، وراقب إن كانت المخرجات تحتفظ بمحور batch. لا تستخدم `squeeze` بلا محور محدد، فقد يزيل بعدًا صحيحًا عند batch size=1.

5.22 اختيار metric مرتبط بالتكلفة

قد يكون false negative أغلى من false positive. عرّف مصفوفة تكلفة القرار، ثم اختر threshold على validation مع إبقاء test لتقدير نهائي. إذا تغيرت prevalence في الإنتاج، أعد تقييم calibration والعتبة عبر إجراء مراقب، لا تعديلها آليًا على مجموعة test.

5.23 احتفظ بعقد الخدمة مع النموذج

عند إصدار endpoint جديد، وثق ما إذا كانت الدرجات logits أو احتمالات، وترتيب الفئات ووحداتها. عميل يفسر logit على أنه probability قد يتخذ قرارًا خاطئًا دون استثناء برمجي. أضف versioned schema ومثال request/response إلى بطاقة التسليم.

5.24 أخطاء شائعة

⚠ تحذير

`model.compile()` لا يدرّب النموذج؛ هو يحدد طريقة التحديث والمقاييس، ويبدأ التدريب مع `fit()`. [!DANGER] `save()` و`export()` يخدمان غرضين مختلفين في Keras 3. اختبر التحميل والتوقيع الفعلي قبل تسليم النموذج.

5.25 تمارين

1. مثّل مصنفًا خطيًا بـ Sequential. (المتوقع: طبقات مرتبة ومخرج بعدد الفئات.)
2. صمّم Functional API بمدخلين. (المتوقع: فرعان ودمج موثق.)
3. اختر خسارة لمخرج logits ثنائي. (المتوقع: BinaryCrossentropy مع `from_logits=True`.)
4. أضف EarlyStopping لمراقبة `val_loss`. (المتوقع: إيقاف واستعادة أفضل أوزان.)
5. احفظ `.keras` ثم أعد التحميل. (المتوقع: تنبؤ متقارب على المدخل نفسه.)
6. فسّر متى تصدّر SavedModel. (المتوقع: عند تجهيز مسار الاستدلال والتكامل.)

5.26 ملخص

- اختر بنية النموذج وفق تدفق البيانات.
- اربط شكل المخرج بنوع الخسارة، وافصل مقياس التحسين عن مقياس القراءة.
- وثّق المعالجة والعتبة، واحفظ بصيغة توافق غرض الاستخدام.

الجزء الثاني

بناء الشبكات وتدريبها

الهندسة، التحسين، التقييم، والبيانات

06 الشبكات العصبية من الصفر

07 المحسّنات والتنظيم: فنّ التدريب

08 التقييم والضبط: كيف تعرف أن نموذجك جيد؟

09 tf.data: هندسة خطوط البيانات

10 الرؤية الحاسوبية: CNN من الالتفاف إلى الإنتاج

الفصول 6-10 · 5 فصول

الجزء الثاني · بناء الشبكات وتدريبها

الفصل السادس

06

الشبكات العصبية من الصفر

> **حكاية الفصل**
>
> يريد مصنع فرز قطعًا معدنية من صورة منخفضة الدقة. يبدأ المهندس بقياس سطوع البكسلات، ثم يبني طبقات تتعلم أنماطًا أكثر تعقيدًا. ما الذي تتعلمه الأوزان فعلًا، وكيف ينتقل أثر قرار التصنيف إلى ملايين المعاملات؟

6.1 أهداف الفصل

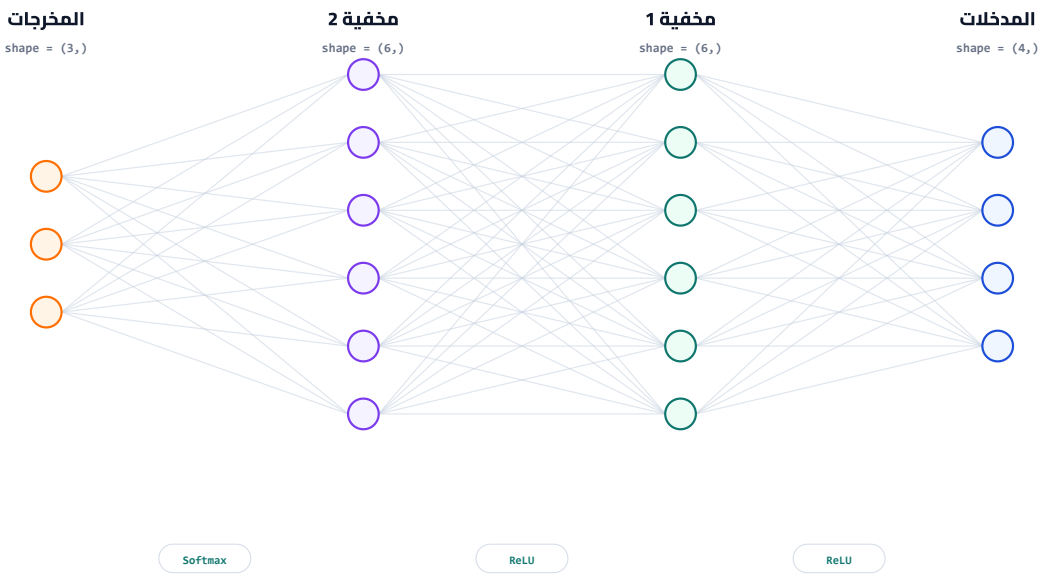
- [ ] تفسير الإدراك والطبقات والانتشار الأمامي.
- [ ] مقارنة التنشيطات والتهيئات الشائعة.
- [ ] اشتقاق تحديث وزن عبر قاعدة السلسلة.
- [ ] كتابة حلقة تدريب بسيطة وفهم حدودها.

6.2 من جمع موزون إلى قرار

تحسب الوحدة العصبية $z = \sum_i w_i x_i + b$، ثم تحول القيمة عبر تنشيط $a = \phi(z)$. المدخل `x_i` سمة، والوزن `w_i` مقدار تأثيرها، والانحياز `b` إزاحة قابلة للتعلم. شبكة متعددة الطبقات تكرر هذه العملية، فتكوّن تمثيلات وسيطة بدل كتابة قواعد السمات يدويًا.

الشبكة العصبية متعددة الطبقات

كل خطّ هو وزن، وكل دائرة عصبون

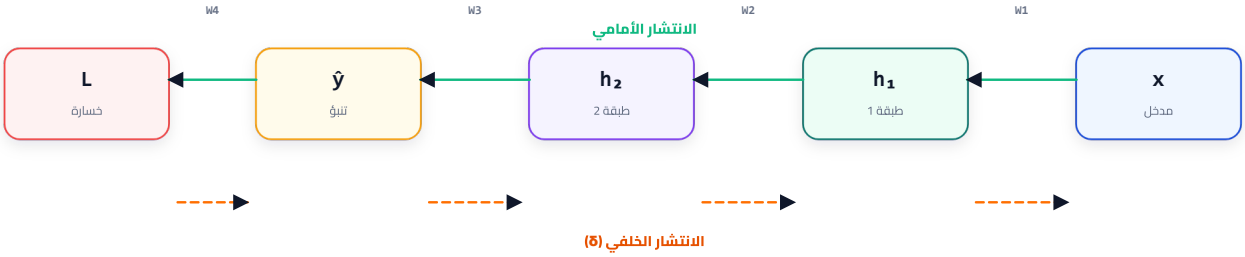


الاتجاه من اليمين (المدخلات) إلى اليسار (المخرجات).

شكل 6-1 الانتشار الأمامي عبر شبكة متعددة الطبقات

الانتشار الخلفي: من الخسارة إلى الأوزان

الأمام يحسب القيمة، والخلف يحسب نصيب كل وزن منها



ماذا يعني؟

الخطأ ينتقل للخلف، وكل وزن يأخذ جزءًا يتناسب مع مساهمته في الخطأ.

قاعدة السلسلة

$$\delta_2 = (W_3^T \, \delta_3) \odot \sigma'(z_2)$$

$$\partial L/\partial W_2 = \delta_2 \, h_1^T$$

شكل 6-2 عودة التدرج لتحديث الأوزان

التنشيط	السلوك	استعمال شائع
Sigmoid	يضغط إلى (0,1)	احتمال ثنائي في المخرج
tanh	متمركز حول الصفر	حالات متكررة تقليدية
ReLU	يصفّر السالب	طبقات مخفية بسيطة
GELU	انتقال ناعم	محولات حديثة
Softmax	يوزع الاحتمال عبر الفئات	مخرج متعدد الفئات

6.3 الانتشار الخلفي قاعدة سلسلة منظمة

إذا كانت الخسارة تعتمد على المخرج، والمخرج يعتمد على وزن، فإن قاعدة السلسلة تفك هذا الاعتماد إلى مشتقات محلية. في طبقة واحدة تكون المشتقة بالنسبة للوزن حاصل ضرب خطأ المخرج والمدخل، وتُحدّث الأوزان بخطوة معاكسة للتدرج. لا تحتاج إلى كتابة المشتقة يدويًا في كل نموذج، لكن فهمها يفسر لماذا يجب أن تبقى العملية داخل الرسم الذي يراقبه GradientTape.

$$w_{t+1} = w_t - \eta \frac{\partial L}{\partial w_t}$$

حيث `w_t` الوزن الحالي، و `L` الخسارة، و `η` معدل التعلّم. الخطوة الكبيرة قد تتجاوز الحد الأدنى، والصغيرة جدًا تبطئ التعلم.

```python
import tensorflow as tf

x = tf.constant([[0.0], [1.0], [2.0], [3.0]])
y = tf.constant([[1.0], [3.0], [5.0], [7.0]])
w = tf.Variable([[0.0]])
b = tf.Variable([0.0])
optimizer = tf.keras.optimizers.SGD(learning_rate=0.05)

for _ in range(100):
    with tf.GradientTape() as tape:
        loss = tf.reduce_mean(tf.square(x @ w + b - y))
    dw, db = tape.gradient(loss, [w, b])
    optimizer.apply_gradients([(dw, w), (db, b)])
print(w.numpy(), b.numpy())
# المتوقع: w تقترب من 2 و b تقترب من 1.
```

6.4 التهيئة والاستقرار

إذا بدأ كل وزن بالقيمة نفسها، تتلقى الوحدات المتناظرة تدرجات متطابقة وتبقى متشابهة. تهيئة Glorot توازن التباين تقريبًا بين الطبقات، وHe تراعي ReLU. أما التدرج المتلاشي فيضعف إشارة التعلم في طبقات سابقة، والانفجار يجعل التحديثات ضخمة؛ التطبيع والقص واختيار المعمارية تساعد، لكنها لا تصلح بيانات أو هدفًا سيئًا.

نظرية التقريب الشامل تقول إن شبكة واسعة بما يكفي يمكنها تقريب دوال مستمرة ضمن شروطها، لكنها لا تقول إن التدريب سيجد الحل أو أن البيانات كافية أو أن الحل يعمم. القدرة التعبيرية ليست ضمانًا لجودة النموذج.

6.5 حالة واقعية: مصنف القطع

يُسطّح الإدخال الصغير إلى متجه، وتُطبّع قيم البكسلات، ثم يخرج النموذج logits لفئتي «سليم» و«معيب». يُقاس الاستدعاء للفئة المعيبة لأن تكلفة تمرير قطعة تالفة أعلى من فحص قطعة سليمة. تبدأ التجربة بخط أساس لوجستي؛ لا تضف طبقات إلا إذا أظهر تحليل الأخطاء نمطًا لا يلتقطه الخط الأساس.

6.6 اشتقاق رجوع الخطأ من طبقة واحدة

خذ طبقة كثيفة لمثال واحد: `z=wx+b` و `a=φ(z)`، وخسارة مربع `L=(a-y)^2/2`. باستخدام قاعدة السلسلة:

$$\frac{\partial L}{\partial w} = (a - y)\phi'(z)x$$

الخطأ `a-y` يقيس اتجاه الفرق، و `φ'(z)` يحول أثره عبر التنشيط، و `x` يحدد كيف يؤثر الوزن في المجموع. في شبكة ذات طبقتين يتكرر هذا التركيب: الخطأ عند المخرج يعود إلى الطبقة السابقة عبر أوزانها ومشتقة تنشيطها. هذا الاشتقاق يشرح لماذا توقف `GradientTape` عن إعطاء تدرج إذا قطعت الحساب بتحويل NumPy أو خرجت من نطاق عملياته.

6.7 التنشيط مشتقةً وسلوكًا

Sigmoid وtanh تتشبعان عند قيم كبيرة موجبة أو سالبة، فتصغر مشتقتهما ويضعف التدرج في الطبقات العميقة. ReLU مشتقتها ثابتة في النصف الموجب وصفر في السالب، ما يسرع كثيرًا من الشبكات لكنه قد يجعل وحدة لا تستجيب. LeakyReLU تحتفظ بميل صغير للجهة السالبة، وGELU انتقال سلس يستخدم كثيرًا في المحولات. Softmax ليست عادة تنشيطًا للطبقات المخفية؛ هي تطبيع متنافس لمجموعة logits في مخرج متعدد الفئات.

التنشيط	مجال المخرج	أثر جانبي	موضع مألوف
Sigmoid	(0,1)	تشبع، وغير متمركز حول الصفر	احتمال ثنائي أو بوابة
tanh	(-1,1)	تشبع عند الأطراف	حالة متكررة
ReLU	[0,∞)	وحدات خاملة في جهة السالب	طبقات كثيفة/تلافيفية
GELU	غير محدود قليلًا	كلفة/سلوك أنعم	Transformer blocks
Softmax	توزيع على الفئات	يعتمد على جميع logits	تصنيف متعدد الفئات

6.8 التهيئة وحجم الطبقة

تحدد التهيئة نطاق قيم الأوزان قبل رؤية البيانات. تهيئة Glorot (Xavier) توازن التباين وفق fan-in وfan-out، وHe تراعي وحدات ReLU. بدء كل الأوزان بصفر يمنع كسر التناظر؛ وحدات الطبقة الواحدة ستتلقى التدرج نفسه وتتعلم النسخة نفسها. أما قيمة عشوائية ضخمة، فقد تمرر تفعيلات وتدرجات ضخمة. اترك اختيار التهيئة الافتراضي الملائم للطبقة ما لم يكن لديك سبب تجريبي لتغييره.

عدد المعاملات في Dense يساوي تقريبًا `input_dim × units + units`. لذلك طبقة بـ 10,000 سمة و1,000 وحدة تخفيّة تحمل أكثر من عشرة ملايين وزن وانحياز، قبل حساب التفعيلات. راقب الحجم والذاكرة وحجم البيانات؛ زيادة عدد المعاملات دون بيانات كافية تسهّل حفظ الأمثلة.

6.9 نموذج رقمي كامل للمبتدئ

لبيانات MNIST، المدخل صورة 28×28 رمادية. الخطوات هي تثبيت البذرة، تحميل البيانات، تطبيع البكسل، تعريف المخرج، اختيار خسارة تتوافق مع logits، ثم تقييم الاختبار مرة بعد انتهاء القرارات:

```python
import keras
from keras import layers

keras.utils.set_random_seed(42)
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
model = keras.Sequential([
    layers.Input((28, 28)), layers.Flatten(),
    layers.Dense(128, activation="relu"), layers.Dense(10),
])
model.compile(optimizer=keras.optimizers.Adam(1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"])
model.fit(x_train, y_train, validation_split=0.1, epochs=5, batch_size=128)
print(model.evaluate(x_test, y_test, verbose=0))
```

هذا نموذج تعليمي كامل، لكنه ليس نتيجة بحث منشورة: الاختيار العشوائي لتقسيم validation قد يخفي اختلاف المصدر، و `accuracy` لا يصف كل الأخطاء، ولا توجد معالجة لتغير المجال. وسّعه فقط بعد تسجيل baseline وتحليل صور الخطأ.

6.10 المقارنة بين التنشيطات على رسم واحد

عند مقارنة ReLU وGELU وtanh، ثبت باقي المتغيرات. قارن منحنى المشتقة، ونسبة الوحدات التي خرجت بصفر، وgradient norm في الطبقات الأولى. قد تتفوق دالة على مجموعة صغيرة بسبب معدل تعلم مناسب بالصدفة؛ كرر التجربة على عدة بذور ثم اعرض mean وstandard deviation. التحسين ليس جدولًا عالميًا منفصلًا عن العمق والتهيئة والتطبيع.

Softmax تحسب احتمالًا لكل فئة وتربط المخرجات بعضها ببعض: زيادة logit واحدة تقلل الحصص النسبية للآخرين. لا تضفها إلى كل مخرج تلقائيًا إذا كانت المهمة multilabel حيث يمكن أن تكون عدة فئات صحيحة معًا؛ استعمل sigmoid مستقلًا لكل label وخسارة ثنائية. هذا الفرق بين multiclass وmultilabelg مصدر خطأ شائع في التصنيف الطبي متعدد المؤشرات.

6.11 من المشتقة إلى تحديث الدفعة

في batch، تحسب الخسارة غالبًا متوسطًا على الأمثلة، فيكون gradient متوسط مساهماتهم. زيادة حجم الدفعة تقلل ضوضاء تقدير التدرج لكنها تستهلك ذاكرة وقد تتطلب تعديل learning rate. SGD العشوائي يعيد تقدير الاتجاه من أمثلة مختلفة كل مرة؛ لذلك يتعرج على سطح الخسارة، بينما momentum ينعّم الحركة. لا تقارن optimizer مع batch size مختلف ثم تنسب كل فرق الخسارة إلى optimizer.

التدرج العددي finite difference يصلح للتحقق من مشتقة بسيطة: غيّر الوزن بمقدار صغير واحسب فرق الخسارة مقسومًا على مقدار التغيير. التحقق العددي مكلف وغير دقيق جدًا مع epsilon صغير أو كبير، لكنه مفيد في طبقة مخصصة ذات معادلة يدوية. TensorFlow يوفر GradientTape للمشتقات العملية؛ يجب أن يكون المدخل أو متغير النموذج قابلًا للتتبع.

6.12 كيف تقرأ model.summary؟

العمود Output Shape يبين حجم التفعيل بعد الطبقة، Paramg # عدد المعاملات. في Dense يرتبط العدد بحجم الإدخال والوحدات. في CNN يتأثر بعدد القنوات لا مساحة الصورة بذاتها، لكن الذاكرة الوسيطة تتأثر بالارتفاع والعرض والدفعة. المعاملات القابلة للتدريب يجب أن تقارب مجموع أوزان الطبقات، وأي layer بها صفر معاملات قد تكون مقصودة مثل pooling أو دليلًا أن فرعًا لم يُبنَ.

6.13 حدود نظرية التقريب الشامل

النتيجة الرياضية تقرر وجود شبكة بقدرة كافية تقرب دالة مستمرة ضمن شروط ونطاق محدود. لا تحدد عدد أمثلة التدريب المطلوب، ولا تضمن أن gradient descent يجد الأوزان، ولا تمنع النموذج من حفظ الضوضاء. الانتظام والتقسيم والاختبار الخارجي تظل واجبة. استخدم النظرية لتفهم القدرة التمثيلية، لا لتسويق تعميم مؤكد.

6.14 تشخيص كل طبقة على حدة

في تجربة مصغرة، اطبع mean/std للتفعيلات وتدرجات كل طبقة، وعدد الأوزان التي تساوي صفرًا بعد ReLU. تفعيل متضخم عبر طبقات متعاقبة قد يدل على تهيئة أو معدل تعلم؛ تدرجات قريبة من صفر في البداية قد تشير إلى التشبع أو مسار قطع. لا تفسر histogram وحده؛ قارنه مع loss وحجم الخطوة وتوزيع السمات.

Gradient clipping يحد مقدار التحديث عندما تتجاوز norm قيمة، لكنه لا يعالج عدم توافق الأهداف أو إدخالًا متضخمًا. إذا احتجته، سجل نسبة الخطوات التي قُصت؛ القص الدائم إشارة أن معدل التعلم أو التصميم يحتاج مراجعة. في الشبكات ذات skip connections، المسار الأقصر يساعد مرور التدرج، لكنه لا يغني عن مراقبة أعداد دقيقة.

6.15 منطق اختيار عدد الطبقات

أضف طبقة أو وحدات فقط عندما يكشف الخطأ الحالي نقصًا قابلًا للتعلم. لو كان النموذج لا يفصل فئتين بسبب تمثيل خطي، أضف nonlinearity. لو تطابق train وانخفض validation، خفّض السعة أو حسن البيانات. لو الاثنين ضعيفان، زد السعة بعد التحقق من الملصقات والخط الأساس. هذا التشخيص يمنع البحث العشوائي في مئات architectures.

6.16 مثال انحدار متعدد الميزات

في مثال الطلب، ميزات مثل المسافة والطقس والساعة تدخل Dense. لكل ميزة مقياس مختلف؛ طبقة Normalization تحفظ إحصاءات train وتطبقها عند inference. لا تستخدم طبقة تتعلم الإحصاءات من كامل البيانات قبل split. قارن مع linear baseline لتعرف هل طبقات غير خطية تضيف فائدة فعلية. للانحدار بمتغير موجب مثل زمن الشحن، افحص هل مخرج خطي يعطي قيمًا سالبة؛ تصميم output مناسب أو تحويل الهدف قد يحل المشكلة.

6.17 ماذا تعرض في التقرير؟

اعرض شكل النموذج، عدد المعاملات، train/validation curves، تعريف loss، عتبة القرار، وconfusion matrix. في بحث عن تعميم، أضف عدة runs أو cross-validation. اذكر صراحة ما لم تختبره: مجال صور مختلف، أو أجهزة منخفضة الدقة، أو فئات نادرة. تفاصيل هذه الحدود تجعل النتائج قابلة للثقة أكثر من رقم دقة بلا سياق.

6.18 أمثلة تنفيذية إضافية

مصنف يخرج logits

```python
import keras
from keras import layers
classifier = keras.Sequential([layers.Input((28, 28, 1)), layers.Flatten(),
    layers.Dense(64, activation="relu"), layers.Dense(10)])
classifier.compile(optimizer="adam",
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True))
```

تثبيت البذرة

```python
import keras
keras.utils.set_random_seed(42)
```

6.19 أخطاء شائعة

⚠ تحذير

لا تستخدم تنشيطًا خطيًا في كل الطبقات ثم تتوقع تمثيلًا غير خطي؛ تركيب التحويلات الخطية يبقى خطيًا.

[DANGER!] لا تخلط مخرجات softmax مع خسارة `from_logits=True` ؛ حدد عقدًا واحدًا بين المخرج والخسارة.

6.20 تمارين

1. احسب ناتج وحدة بمدخلين وأوزان معلومة. (المتوقع: `z` ثم تنشيط محدد.)
2. اشتق خسارة تربيعية بالنسبة لوزن واحد. (المتوقع: تطبيق مباشر لقاعدة السلسلة.)
3. قارن ReLU وSigmoid على قيم سالبة وموجبة. (المتوقع: ملاحظة التشبع والتصفير.)
4. بدّل SGD بـ Adam في المثال وقارن التقارب. (المتوقع: مقارنة لا حكم مطلق.)

5. اشرح لماذا لا تضمن نظرية التقريب تعميمًا جيدًا. (المتوقع: فصل التمثيل عن التعلم والبيانات.)
6. صمّم مصنفًا وحدد مقياس الفئة الحرجة. (المتوقع: قرار مرتبط بكلفة المصنع.)

6.21 ملخص

- الشبكة تركيب وحدات خطية ولاخطية تتعلم تمثيلات.
- الانتشار الخلفي يطبق قاعدة السلسلة، والمحسّن يحدّث الأوزان باتجاه يخفض الخسارة.
- التهيئة والتنشيط يؤثران في استقرار التدريب، والقدرة على التقريب لا تضمن التعميم.

الجزء الثاني · بناء الشبكات وتدريبها

الفصل السابع

07

المحسّنات والتنظيم: فنّ التدريب

حكاية الفصل

حفظ نموذجٌ بيانات مرضى التدريب بدقة ممتازة، لكنه أخفق في مستشفى آخر. لم تكن إضافة طبقات الحل؛ كان النموذج قد حفظ ضوضاء المستشفى الأول. كيف نفرّق بين تعلّم نمط مفيد وحفظ الأمثلة؟

7.1 أهداف الفصل

- [ ] فهم أثر معدل التعلم والزخم وخوارزميات التحسين.
- [ ] التمييز بين التطبيع والتنظيم.
- [ ] تشخيص فرط التعلّم وعدم استقرار الخسارة.
- [ ] بناء تدريب يستخدم الإيقاف المبكر والقص عند الحاجة.

7.2 المحسّن يحدد طريقة الحركة

SGD يخطو وفق التدرج الحالي. الزخم يجمع اتجاهات سابقة لتخفيف التعرج، RMSProp يطبع الخطوة وفق متوسط مربعات التدرج، Adam يجمع تقديري المتوسط والتباين. AdamW يفصل اضمحلال الوزن عن تحديث التدرج. لا توجد خوارزمية تفوز دائمًا؛ ثبّت تقسيم البيانات وقارن على المقياس الذي يخدم الاستخدام.

ثلاثة مسارات إلى القاع

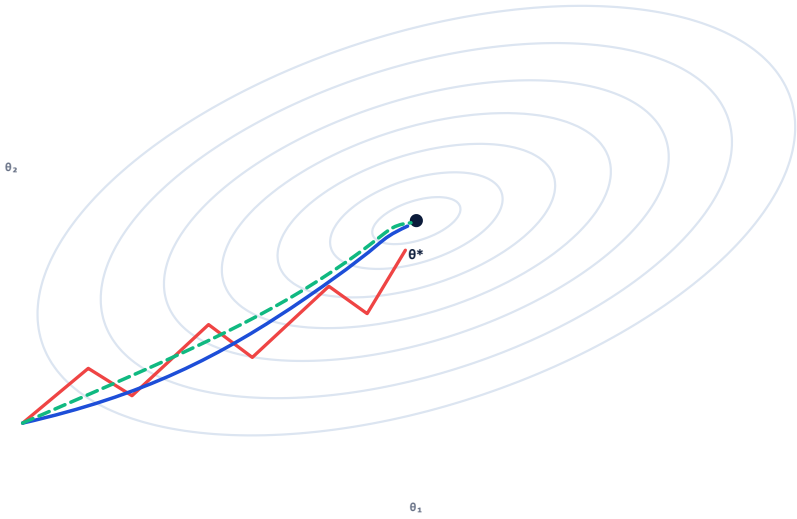
نفس سطح الخسارة، وسلوك مختلف في الاقتراب من الحل

SGD
خطوات بسيطة، تذبذب عالٍ

Momentum
ذاكرة للاتجاه، سلاسة أكبر

Adam
معدّل تعلّم تكيّفي لكل وزن

$\theta \leftarrow \theta - \eta \nabla L(\theta)$
القاعدة المشتركة



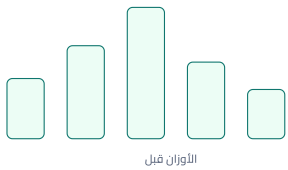
شكل 7-1 اتجاه المحسنات على سطح الخسارة

أربع أدوات لترويض الإفراط في التعلّم

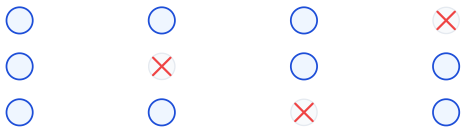
كل أداة تعالج عرضًا مختلفًا؛ لا تستخدمها كلها بلا سبب

L2 / Weight Decay
عقوبة على الأوزان الكبيرة

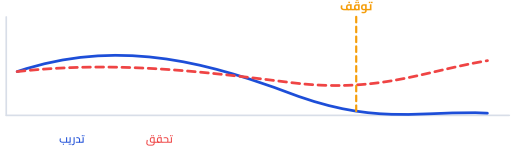
$L = L_0 + \lambda \|w\|^2$
λ كبيرة ← تقليص أقوى



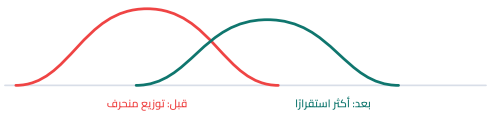
Dropout
تعطيل عصبونات عشوائيًا أثناء التدريب



EarlyStopping
أوقف التدريب قبل أن يبدأ الحفظ



BatchNorm
تطبيع التفعيلات داخل كل دفعة



شكل 7-2 أدوات التنظيم للحد من فرط التعلم

$$v_t = \beta v_{t-1} + (1-\beta) g_t$$

يمثل `g_t` التدرج، و`v_t` متوسطه المتحرك، و`β` معامل الزخم. يخفف المتوسط اهتزاز الإشارة في اتجاهات متضاربة، لكنه قد يحمل اتجاهًا لم يعد مناسبًا إن تغيرت طبيعة البيانات.

العرض	تفسير محتمل	تجربة أولى
خسارة تتذبذب بعنف	معدل تعلم مرتفع أو دفعات صغيرة	خفّض المعدل وراقب التدرج
تدريب جيد وتحقق ضعيف	فرط تعلّم أو انقسام غير ممثل	تنظيم وتحليل الانقسام
كلاهما ضعيف	نموذج محدود أو إعداد تدريب غير كاف	افحص خط الأساس والميزات
تحقق يتدهور بعد تحسن	بدء حفظ بيانات التدريب	إيقاف مبكر واستعادة الأفضل

7.3 التنظيم يحدّ من اعتماد النموذج على تفاصيل العينة

L1 يدفع بعض الأوزان نحو الصفر، وL2 يعاقب الأوزان الكبيرة. Dropout يعطّل وحدات عشوائيًا أثناء التدريب، ولا يعمل بالطريقة نفسها في التنبؤ. BatchNorm يطبّع وفق إحصاءات الدفعة ويملك سلوك تدريب واستدلال مختلفًا؛ LayerNorm يطبّع داخل المثال، ولهذا يناسب تسلسلات كثيرة. التنظيم أداة، وليس بديلًا عن تقسيم صحيح أو بيانات ممثلة.

```python
import keras
from keras import layers

model = keras.Sequential([
    layers.Input((64,)),
    layers.Dense(128, activation="relu", kernel_regularizer=keras.regularizers.l2(1e-4)),
    layers.Dropout(0.25),
    layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer=keras.optimizers.AdamW(learning_rate=3e-4, weight_decay=1e-4),
              loss="binary_crossentropy", metrics=[keras.metrics.AUC(name="auc")])
callbacks = [keras.callbacks.EarlyStopping(monitor="val_auc", mode="max",
                                           patience=3, restore_best_weights=True)]
```

7.4 معدل التعلم والقص والدقة المختلطة

ابدأ بمعدل تعلم معقول وسجل المنحنيات. جدولة Cosine تخفض المعدل تدريجيًا؛ `ReduceLROnPlateau` يتفاعل مع ثبات مقياس التحقق. قص معيار التدرج يحد من انفجار التحديث لكنه لا يعالج السبب الجذري. الدقة المختلطة تقلل كلفة الذاكرة والحساب على مسرّعات مناسبة، لكنها تتطلب الانتباه إلى دعم العتاد ودقة العمليات الحساسة.

7.5 حالة واقعية: تعميم نموذج المستشفى

قسّم البيانات بحسب المستشفى أو المريض قبل التدريب حتى لا تظهر صور الشخص نفسه في مجموعتين. راقب خسارة التحقق والاستدعاء لكل موقع. إذا تحسنت مجموعة التدريب وحدها، لا تعالج الأمر بمجرد رفع epochs؛ افحص التسرب والتحول بين المواقع، ثم جرّب augmentation أو تقليل سعة النموذج أو regularization.

7.6 مسارات ضبط منهجية

ابدأ بمعدل تعلم يحقق نزولًا واضحًا ومستقرًا، ثم غيّر عاملًا واحدًا في كل تجربة. مخطط الخسارة مقابل الزمن يكشف المعدل غير المناسب: انفجار متكرر يوحي بخطوة كبيرة، ونزول بطيء جدًا قد يشير إلى خطوة صغيرة أو تدرج ضعيف. لا تقارن runs على تقسيمات مختلفة. سجل البذرة، ونسخة البيانات، وحجم الدفعة، والتطبيع، والمعدل، والمقياس، ووقت التدريب.

SGD مع momentum قد يعمم جيدًا ويحتاج ضبط معدل وجدول بعناية. Adam يضبط الخطوات لكل معامل ويساعد على بدء سريع، لكنه قد يحتاج tuning للتعميم. AdamW يفصل اضمحلال الأوزان عن التحديث التكيفي. L2 regularization يضيف حدًا إلى الخسارة، في حين weight decay يضمحل المعامل ضمن قاعدة المحسن؛ تنفيذ Adam القديم قد يربطهما بخلاف AdamW. لا تفسر `weight_decay=...` على أنه مكافئ تلقائيًا للـ kernel L2 في كل محسن.

7.7 تطبيع داخل الشبكة لا يطبع البيانات الأصلية

BatchNormalization يستخدم إحصاءات الدفعة أثناء التدريب ويخزن متوسطًا وتباينًا متحركين للاستدلال. لذلك يجب اختبار المسارين `training=True` و `training=False`، مع الدفعات الصغيرة جدًا قد تكون الإحصاءات مشوشة. LayerNormalization يطبّع محاور المثال دون الاعتماد على بقية أمثلة الدفعة، ولهذا

يناسب كثيرًا من طبقات التسلسل. كلاهما يغيران optimization، لكنهما لا يغنيان عن تطبيع المدخلات عند اختلاف وحدات الميزات.

7.8 وصفة تشخيص عملية لفرط التعلّم

إذا انخفضت خسارة التدريب واستقرت خسارة التحقق ثم ارتفعت، فهذه إشارة إلى حفظ تفاصيل التدريب. افحص أولًا التسرب وتقسيم البيانات؛ التنظيم لا يصلح اختبارًا ملوثًا. بعد ذلك جرّب بالترتيب: تقليل عدد epochs مع استعادة الأفضل، augmentation لا يغير حقيقة الملصق، dropout أو weight decay معتدل، ثم نموذج أصغر. لكل خطوة سجل هل تحسنت فجوة التعميم والمقياس الذي يهم المجال.

7.9 الدقة المختلطة والاستقرار العددي

تضبط سياسة الدقة قبل بناء الطبقات. `mixed_float16` قد يسرع وحدات Tensor Core على GPU حديثة، و`mixed_bfloat16` مناسب لعتاد يدعم نطاق bfloat16. تحتفظ Keras بمتغيرات كثيرة بدقة float32 وتطبق loss scaling تلقائيًا في إعدادات `fit` المعتادة. افحص الخسارة وعدم وجود NaN، وقارن مقياس التحقق وذاكرة الذروة. إذا كان الجهاز CPU لا يملك عتادًا ملائمًا، قد يكون المسار أبطأ.

```python
import keras

# اضبط السياسة قبل بناء النموذج، ثم تحقق من dtype للطبقات والمخرجات.
keras.mixed_precision.set_global_policy("mixed_float16")
model = make_model()
print(model.layers[0].compute_dtype, model.layers[0].variable_dtype)
```

اجعل آخر طبقة إخراج حساسة بدقة مناسبة عند الحاجة، خصوصًا logits وخسائر ذات مجال عددي ضيق. لا تقارن نسخة مختلطة بأخرى مختلفة في batch أو عدد خطوات، وإلا لن تعرف مصدر فرق السرعة.

7.10 جداول معدل التعلم

جدول ثابت الخطوات يخفض المعدل بعد عدد محدد من الخطوات، Cosineg يخفّضه بسلاسة نحو حد أدنى. `ReduceLROnPlateau` ينتظر ثبات metric لكنه يعتمد على ضجيج validation. patienceg حدد وحدة الزمن: هل `decay_steps` حقبة أو دفعة؟ في التدريب الموزع قد يتغير عدد الخطوات لكل حقبة. سجل المعدل الفعلي مع كل epoch، ولا تختَر جدولًا طويلًا يجعل التدريب لا يصل أبدًا إلى مرحلة التناقص.

اختبار range صغير يرفع معدل التعلم تدريجيًا ويراقب loss؛ قد يساعد على اقتراح نطاق آمن، لكنه heuristic لا يغني عن tuning. تجارب البحث العشوائي أو Hyperband يجب أن تستخدم تقسيمًا ثابتًا budgetg واضحًا، وتستبعد تجربة فشلت لأسباب بيئية بدل عدّها قيمة سيئة للنموذج.

7.11 Dropoutg BatchNorm في inference

BatchNorm يتبدل بين inferenceg training: أثناء التدريب يستخدم إحصاءات batch ويحدث moving averages، وأثناء الاستدلال يستخدم القيم المتراكمة. إذا استدعيت النموذج يدويًا بـ `training=True` عند التنبؤ، تتغير النتائج مع تركيب الدفعة. Dropout يعطل وحدات عشوائيًا أثناء التدريب ويصبح identity تقريبًا في inference. اختبر batch منفردًا ومجموعة مجمعة للتأكد من ثبات السلوك المنشور.

7.12 تنظيم يعكس فرضية

Dropout يحد من الاعتماد على وحدة مفردة، لكنه قد يبطئ تقارب نموذج صغير أكثر مما يحسن التعميم. L2 يشجع معاملات أصغر، وL1 قد ينتج أوزانًا صفرية لكنه لا يضمن تفسير السمات. Augmentation فرضية أن التحويل يحافظ على label؛ إذا كانت العبارة «لا» وانقلبت بالقص أو augmentation، فتعلم النموذج label غير صحيح. راجع عينات التحويل المرئي والملصقات مع خبير المجال.

7.13 خطة اختبار لعدم الاستقرار

إذا ظهرت loss=NaN، افحص أول دفعة تخرج فيها المشكلة، وقيم المدخلات والهدف، gradient normg، والدقة العددية. جرب خفض learning rate، إزالة عملية exponential غير مستقرة، أو تفعيل clipping إذا انفجر gradient. لا تعالج NaN بإخفائها عبر `nan_to_num` دون معرفة السبب. اكتب assertion للقيم finite عند مدخل pipeline ومخرج النموذج خلال التشخيص.

7.14 سجل المقارنة

التجربة	التغيير الوحيد	نتيجة التدريب	نتيجة التحقق	تكلفة
baseline	Adam، لا تنظيم إضافي	يسجل بعد التدريب	مرجع المقارنة	وقت/ذاكرة
A	خفض learning rate	يوضح استقرار الحركة	هل تحسن المقياس؟	زمن أطول محتمل
B	weight decay أو dropout	فجوة تعميم أصغر؟	تحسن على شرائح؟	قد يبطئ التقارب

تجنب اختبار كل التركيبات يدويًا بلا تسجيل. اكتب فرضية مسبقة لكل تغيير، واحتفظ بنتيجة فاشلة حين تفسر حدود المجال.

7.15 فرط التعلّم تحت المجهر

راقب فجوة التعميم `val_loss - train_loss` على امتداد epochs، لكن تذكر أن loss تعتمد مقياس الخسارة والتوزيع. افحص منحنى تعلم بزيادة كمية بيانات التدريب؛ إذا تحسن التحقق مع المزيد، فالعينة قد تكون غير كافية. إذا لم يتحسن، راجع جودة السمات أو label noise أو انحياز النموذج. سجل أخطاء يدوية بحسب الفئة والمصدر بدل الاعتماد على الخط البياني وحده.

7.16 وزن الفئات والـ focal loss

class weights تعظم أثر الأخطاء في الفئة الأقل تمثيلًا، لكنها تغير معايرة الاحتمال وقد ترفع false positives. Focal loss تقلل وزن الأمثلة السهلة وتركز على الصعبة، وملائمة لبعض كشف الأجسام، لكنها ليست الخيار الافتراضي لكل عدم توازن. قارنها مع baseline وقيّم precision-recall calibrationg على توزيع الإنتاج. إذا كان هناك resampling، نفذه في train فقط ولا تكرر أمثلة validation.

7.17 Early stopping ومشكلة epoch المختارة

patience يجب أن تسمح بتذبذب metric لا أن تطيل التدريب بلا داعٍ. اكتب `restore_best_weights=True`، وحدد min_delta وmonitorg حسب الهدف. إذا كانت validation صغيرة أو noisy، قد تكون نقطة الأفضلية عرضية؛ استخدم smoothing للتشخيص لا لتحويل score الرسمي، وراجع folds أو فترة تحقق أكبر. لا توقف على `loss` إذا كان قرار النجاح يستند إلى recall عند threshold.

7.18 تنظيم البيانات أهم من تنظيم الوزن

قبل L1/L2، تأكد من أن split يمنع تكرار أصل الصورة أو العميل. تسرب المعالجة المسبقة يجعل التحقق متفائلًا حتى مع نموذج بسيط. ثبت augmentations الصحيحة دلاليًا، واستخدم data collection لسد فجوات المجال. النموذج ينتظم وفق المعلومات المتاحة له؛ لا يستطيع dropout إزالة تحيز label موروث.

7.19 جدول قرار للتقنيات

العلامة المرصودة	أول تشخيص	تجربة محددة
train valg مرتفعان	underfit أو pipeline خطأ	overfit لدفعة صغيرة، ثم سعة أعلى
train منخفض valg مرتفع	overfit أو split مختلف	تحقق من التسرب ثم تنظيم/بيانات
loss انفجرت	LR أو أعداد غير مستقرة	خفض LR وفحص المدخل والgradient
recall منخفض لفئة نادرة	imbalance أو threshold	PR curve وأوزان على train
الجودة جيدة والزمن سيئ	input أو kernel بطيء	profiler ثم اختبار mixed precision

7.20 أمثلة تنفيذية إضافية

قص التدرج

```python
optimizer = keras.optimizers.Adam(learning_rate=3e-4, clipnorm=1.0)
model.compile(optimizer=optimizer, loss="binary_crossentropy")
```

جدول Cosine

```python
schedule = keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=1e-3, decay_steps=1000, alpha=0.1)
optimizer = keras.optimizers.Adam(learning_rate=schedule)
```

7.21 أخطاء شائعة

⚠ تحذير

BatchNorm مع دفعات صغيرة جدًا قد يقدّر إحصاءات غير مستقرة؛ قارنه بـ LayerNorm أو إعدادات ملائمة.

[DANGER!] لا تضف Dropout وL1 وL2 وكل منظمات التدريب دفعة واحدة؛ عندها يصعب معرفة سبب التحسن أو التدهور.

7.22 تمارين

1. ارسم منحنيات تدريب وتحقق وحدد علامة فرط التعلّم. (المتوقع: فجوة متزايدة بعد تحسن التدريب.)
2. قارن SGD وAdam على إعداد ثابت. (المتوقع: تسجيل المنحنى والزمن لا الدقة النهائية وحدها.)
3. أضف EarlyStopping مع استعادة أفضل أوزان. (المتوقع: نموذج يطابق أفضل حقبة تحقق.)
4. اختر BatchNorm أو LayerNorm لتسلسل قصير وعلّل. (المتوقع: ربط القرار ببعد التطبيع والدفعة.)
5. اقترح سببًا لتذبذب خسارة التدريب. (المتوقع: اختبار معدل التعلم وحجم الدفعة.)
6. فسّر متى يفيد قص التدرج. (المتوقع: الحد من تحديثات منفجرة لا تصحيح بيانات خاطئة.)

7.23 ملخص

- المحسّن ومعدل التعلم يحددان مسار خفض الخسارة.
- التنظيم يقلل فرط التعلّم، لكن يجب تشخيص السبب باستخدام مجموعات مستقلة.
- سجّل المنحنيات، واستعد أفضل أوزان، ولا تضف تقنيات متعددة بلا تجربة واضحة.

الجزء الثاني · بناء الشبكات وتدريبها

الفصل الثامن

08

التقييم والضبط: كيف تعرف أن نموذجك جيد؟

> **حكاية الفصل**
>
> في شركة اتصالات، بنى فريقٌ نموذجًا للتنبؤ بفقدان العملاء (churn). أظهر التقرير دقة 92%، فاحتفل الجميع. بعد شهرٍ من التشغيل، لاحظ مدير التسويق أن النموذج لم يُنبّه إلى أي عميلٍ راحلٍ تقريبًا: كان يقول «سيبقى» للجميع، لأن 92% من العملاء يبقون فعلًا. لم يكن الحساب خاطئًا؛ كانت **الدقة** وحدها هي الكذبة. فكيف نقيس نموذجًا لا يكتفي بأن يبدو صحيحًا؟

8.1 أهداف الفصل

- [ ] تقسيم البيانات إلى تدريب وتحقق واختبار، وكشف التسريب قبل أن يفسد النتائج.
- [ ] استخدام التحقق المتقاطع الطبقي (stratified cross-validation) بدل الاعتماد على تقسيم واحد.
- [ ] قراءة مصفوفة الالتباس (confusion matrix) واشتقاق الإحكام والاستدعاء وF1 منها.
- [ ] تفسير منحنيات ROC وPR والمعايرة (calibration)، وربطها بقرارٍ عملي لا برقمٍ مجرّد.
- [ ] ضبط فرط الوسائط (hyperparameters) وتتبّع التجارب وتحليل الأخطاء بمنهجٍ قابل للتكرار.

8.2 ثلاثة صناديق: لماذا لا تكفي مجموعة واحدة؟

تخيّل طالبًا يدرس من أسئلة الامتحان نفسها، ثم يتفاجأ بعلامة متواضعة. النموذج يفعل الشيء ذاته حين نُقيّمه على البيانات التي تدرّب عليها. لذلك نقسم البيانات إلى ثلاثة صناديق: **التدريب (train)** الذي تُتعلَّم منه الأوزان، **التحقق (validation)** الذي نختار به فرط الوسائط ونوقف التدريب مبكرًا، و**الاختبار (test)** الذي لا نلمسه إلا مرة واحدة في النهاية. نسبة شائعة هي 70/15/15 أو 80/10/10، لكن النسبة تتبع حجم البيانات: كلما قلّت، زادت أهمية التحقق المتقاطع.

الاختبار أشبه بورقة مغلقة. إن نظرت إليها في كل تجربة، ثم اخترت النموذج الذي يناسبها، فقد حوّلتها إلى مجموعة تحقق ثانية، وصار تقديرك متفائلًا بلا أن تشعر. القاعدة: **الاختبار يُستخدم للحكم النهائي، لا للبحث.**

الخطر الأكبر هنا هو **التسريب (leakage)**: أن تصل معلومة من خارج مجموعة التدريب إليها. صورتُه الأشهر حساب متوسط وتطبيع البيانات على المجموعة كاملة قبل التقسيم، فيحمل كل مثال تدريبي أثرًا من عينات الاختبار. ومن صوره أيضًا:

- **تسريب الهدف (target leakage):** ميزة تُسجَّل بعد الحدث الذي تتنبأ به. نموذجٌ يتوقع إعادة دخول المريض للمستشفى يستخدم «عدد الزيارات اللاحقة»، وهي معلومة غير متاحة لحظة القرار.
- **تسريب المجموعات:** صور المريض نفسه أو معاملات العميل نفسه تتوزع بين التدريب والاختبار، فيبدو النموذج حافظًا وهو يتعرّف على الهوية.
- **تسريب زمني:** خلط صفوف سلسلة زمنية عشوائيًا، فيتدرّب النموذج على المستقبل ويتقيّم على الماضي.
- **تكرار العينات:** نسخةٌ من كل مثال في مجموعتين، أو إعادة عينات الأقلية قبل التقسيم.

الحل بنيوي لا لفظي: اقسم أولًا، ثم لائم كل معالجة داخل طية التدريب فقط. اجعل التطبيع والتشفير واختيار الميزات جزءًا من خط أنابيب يُعاد تركيبه في كل تقسيم. وإذا كان لديك معرفة بالمجموعات أو الزمن، فاستخدم تقسيمًا يحترمهما بدل التقسيم العشوائي.

8.3 التحقق المتقاطع: أن تختبر أكثر من مرة

تقسيم واحد قد يعطيك حظًا جيدًا أو سيئًا. **التحقق المتقاطع (cross-validation)** يعالج ذلك بتقسيم البيانات إلى K طيات (folds)، يتدرّب النموذج على $K-1$ ويتحقق على الطية المتبقية، ثم يدور الدور. النتيجة توزيعٌ من الدرجات، لا رقم واحد.

في التصنيف غير المتوازن نستخدم **التحقق المتقاطع الطبقي (StratifiedKFold)**، الذي يحافظ على نسبة الفئات في كل طية. وإذا كانت العينات مرتبطة بمجموعات، فـ `GroupKFold` يمنع تسرّب المجموعة. وللسلاسل الزمنية، `TimeSeriesSplit` يجعل كل تدريب يسبق تحققه زمنيًا.

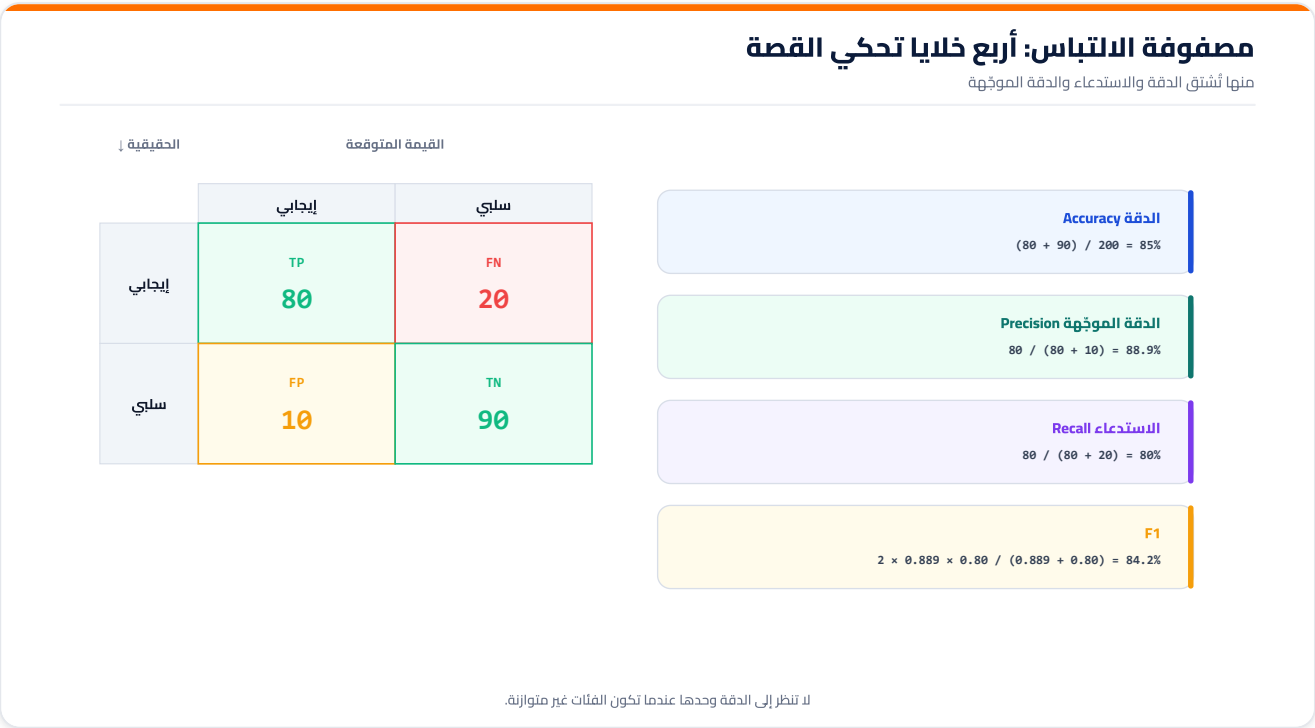
لماذا يهمّ التشتت؟ لأن متوسط الدرجات وحده لا يكفي. الخطأ المعياري للمتوسط يقارب:

$$\mathrm{SE} = \frac{s}{\sqrt{K}}$$

حيث s الانحراف المعياري للدرجات بين الطيات، وK عدد الطيات. كلما زاد K ضاق الخطأ، لكن زاد زمن التدريب. خمس طيات توازنٌ عملي شائع، وعشرٌ مفيدة عندما تكون البيانات صغيرة والزمن متاحًا. ولا تختر الطية التي أعطت أفضل رقم؛ فذلك تسريبٌ صغير لكنه حقيقي.

8.4 مصفوفة الالتباس: تشريح الخطأ

الدقة (accuracy) تجيب عن سؤال واحد: كم نسبة ما أصبت؟ لكنها لا تقول **أي** الأخطاء ارتكبت. مصفوفة الالتباس تفصّل ذلك في أربع خلايا: الموجب الصحيح (TP)، والموجب الخاطئ (FP)، والسالب الصحيح (TN)، والسالب الخاطئ (FN).



شكل 8-1 مصفوفة الالتباس: من أين تأتي الأخطاء؟

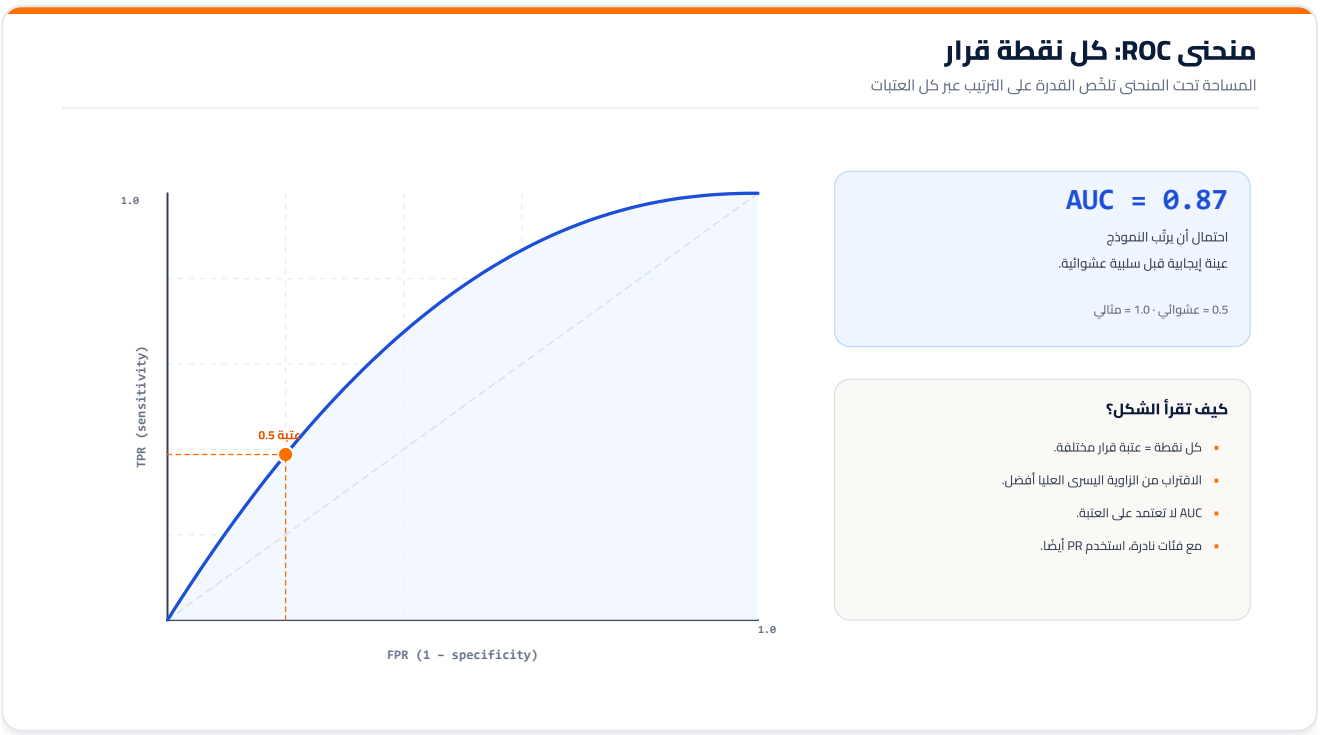
من هذه الخلايا نستخرج مقياسين متضادّين في الغالب. **الإحكام (precision)** هو نسبة ما تنبّأت به موجبًا وكان موجبًا فعلًا: $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$. و**الاستدعاء (recall)** هو نسبة الحالات الموجبة الحقيقية التي أمسكت بها: $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$. الأول يجيب: «حين أطلق الإنذار، كم مرة أكون محقًا؟». الثاني يجيب: «من كل الحالات التي كان يجب أن أمسكها، كم أمسكت؟».

يقيس **F1** التوازن بينهما:

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R}$$

حيث P الإحكام، وR الاستدعاء. إن كان أحدهما صفرًا، صار F_1 صفرًا؛ فهو لا يكافئ نموذجًا يضحّي بأحدهما كليًا. وهناك متوسطات أخرى مثل المتوسط الحسابي المرجّح، لكن F1 يبقى الأكثر شيوعًا حين تريد رقمًا واحدًا يلخّص التوازن.

منحنى ROC يرسم معدل الموجب الصحيح مقابل معدل الموجب الخاطئ عند كل عتبة ممكنة، و**AUC** هي المساحة تحته. قيمتها 0.5 تعني ترتيبًا عشوائيًا، و1 تعني ترتيبًا مثاليًا. هي مقياس لا يعتمد على العتبة، لذا تفيد في مقارنة قدرة النموذج على الترتيب، لكنها لا تخبرك بالعتبة التي تناسب عملك.



شكل 8-2 منحنى ROC ومساحة AUC

في البيانات النادرة، قد يبدو ROC متفائلًا لأن العدد الهائل من السلبيات يجعل معدل الموجب الخاطئ صغيرًا حتى مع أخطاء كثيرة. هنا يكون **منحنى الإحكام-الاستدعاء (PR)** أصدق: يركز على الفئة الموجبة وحدها. القاعدة العملية: استخدم PR عندما تكون الأقلية هي ما يهمّك، وROC عندما تريد صورة عن الترتيب عبر العتبات.

8.5 المعايرة: هل الثقة تعني الاحتمال؟

نموذجٌ يخرج 0.9 لا يعني بالضرورة أن تسعة من كل عشرة أمثلة كهذه موجبة. **المعايرة (calibration)** تسأل: هل تطابق الثقة المعلنة مع التكرار الفعلي؟ النموذج المُفرِط في الثقة (overconfident) يقول 0.95 ويصيب في 70% فقط، وهو خطير حين تُبنى القرارات على الاحتمال نفسه: حدّ الرفض، تسعير التأمين، ترتيب حالات المراجعة.

أبسط مقياس هو **درجة برير (Brier score)**: متوسط مربع الفرق بين الاحتمال المتوقع والنتيجة الحقيقية. ومقياسٌ آخر أسهل في التفسير هو **خطأ المعايرة المتوقع (ECE)**:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left| \mathrm{acc}(B_m) - \mathrm{conf}(B_m) \right|$$

حيث M عدد حزم الثقة (مثل عشر حزم بين 0 و1)، وB_m مجموعة العينات التي وقعت ثقتها في الحزمة m، و$|B_m|$ عددها، وN العدد الكلي، و$\mathrm{acc}(B_m)$ نسبة الإجابات الصحيحة داخل الحزمة، و$\mathrm{conf}(B_m)$ متوسط الثقة فيها. الرمز $|\cdot|$ يعني القيمة المطلقة.

ارسم **مخطط الموثوقية (reliability diagram)**: الثقة على المحور الأفقي، والدقة الفعلية على الرأسي. النموذج المعاير يقترب من الخط القطري. ومن أبسط العلاجات **تدريج الحرارة (temperature scaling)**: اقسم لوغاريتمات المخرجات على ثابت $T > 1$ قبل softmax، فيلين التوزيع دون تغيير الترتيب. لاحظ أن أوزان الفئات وخسارة التركيز تُغيّر الاحتمالات عمدًا، لذا عاير النموذج بعد ضبطها إن كنت ستستخدم الاحتمال قرارًا.

8.6 عندما تكون الأقلية هي المهمة

في كشف الاحتيال، قد تكون نسبة الحالات الموجبة 0.5%. نموذجٌ يتنبأ «سليم» دائمًا يحقق دقة 99.5%، ويفشل في المهمة كلها. الحل ليس رقمًا سحريًا واحدًا، بل مجموعة أدوات.

أوزان الفئات (class weights) تكلّف النموذج أكثر عند الخطأ في الأقلية. وزن الفئة c يُحسب عادة:

$$w_c = \frac{N}{K \cdot N_c}$$

حيث N العدد الكلي للعينات، وK عدد الفئات، وN_c عدد عينات الفئة c. فإذا كان لديك 950 سالبًا و50 موجبًا، يصبح وزن الموجب 10 ووزن السالب نحو 0.53. مرّرها إلى `fit` عبر `class_weight`.

إعادة العينات (resampling) نوعان: تكرار الأقلية (oversampling) يحافظ على البيانات لكنه يضاعف خطر الحفظ، وحذف من الأكثرية (undersampling) يسرّع التدريب لكنه يرمي معلومة. والتوليد الاصطناعي مثل SMOTE يخلق عينات بين الجيران، وهو مفيد أحيانًا ومضرّ أحيانًا في البيانات عالية الأبعاد. الأهم: أعد العينات **بعد** التقسيم، وإلا تسرّبت نسخٌ من التدريب إلى الاختبار.

خسارة التركيز (focal loss) فكرة مختلفة: بدل وزن الفئة فقط، خفّض وزن الأمثلة السهلة التي صار النموذج يتقنها، وركّز على الصعبة.

$$\mathrm{FL}(p_t) = -\alpha_t \, (1 - p_t)^{\gamma} \log(p_t)$$

حيث p_t احتمال الفئة الصحيحة، وα_t وزنًا للفئة، وγ معامل التركيز: كلما كبر، قلّت مساهمة الأمثلة السهلة. عند $\gamma = 0$ تعود الخسارة إلى الانتروبيا المتقاطعة المعتادة.

الأسلوب	ما يغيّره	الخطر	متى يناسبك؟
أوزان الفئات	يكلف الخطأ في الأقلية أكثر	احتمالات غير معايرة	بداية سريعة وبلا تغيير في البيانات
تكرار الأقلية	يوازن الدفعات	حفظٌ وتكرار حرفي	بيانات قليلة ومعالجة بسيطة
حذف الأكثرية	يقلّل حجم البيانات	فقدان معلومات	بيانات ضخمة جدًا وزمن تدريب حرج
خسارة التركيز	يركّز على الأمثلة الصعبة	حساسية لـγ وα	تصنيف نادر وكثيف الأخطاء
تحريك العتبة	يوازن الإحكام والاستدعاء	لا يغيّر النموذج نفسه	عندما تعرف تكلفة كل خطأ

8.7 الضبط والتتبع وتحليل الأخطاء

بعد أن يصبح النموذج سليمًا، يبدأ السؤال: أي فرط وسائط نختار؟ **KerasTuner** يجرّب فضاءً محددًا: عدد الطبقات، الوحدات، معدل التعلّم، الإسقاط. **البحث العشوائي (RandomSearch)** أبسط من البحث الشبكي وأفضل منه غالبًا في الأبعاد الكثيرة. أما **Hyperband** فيبدأ تجارب كثيرة بدورات قصيرة، ويوقف الضعيف منها مبكرًا، ويُمنح الجيد مزيدًا من الدورات. هذا «التقليم المتتالي» يوفّر زمنًا كبيرًا حين يكون التدريب مكلفًا.

اختر هدفًا واحدًا يعبّر عن قرارك، مثل `val_auc` أو `val_precision` عند استدعاء ثابت. لا تُحسّن دقة خامّة في بيانات غير متوازنة، ولا تبحث على مجموعة الاختبار. سجّل مع كل تجربة: البذرة، وإصدار البيانات، والمعالجة، والمقياس، والزمن؛ فالتجربة التي لا يمكن إعادتها ليست نتيجة، بل ذكرى.

TensorBoard يجعل هذا مرئيًا: منحنيات الخسارة والمقاييس، توزيع الأوزان، الصور، ومنحنيات PR. اكتب سجلًّا لكل تشغيل تحت مجلد مختلف، ثم قارن. وإلى جانب الأدوات، يبقى **تحليل الأخطاء (error analysis)** أقوى خطوة: خذ مئة مثال خاطئ واقرأها بعينك. صنّفها إلى ضوضاء في الوسم، وحالات غامضة، وميزة ناقصة،

وانحراف في شريحة معينة. ثم اختبر الفرضية على شريحة أوسع. كثيرًا ما يكون التحسين الأكبر في البيانات أو المعالجة، لا في المعمارية.

تجربة صغيرة: من الاحتمال إلى مصفوفة الالتباس

يعيد النموذج احتمال الانتماء إلى الفئة الإيجابية، لكن مدير التشغيل يحتاج إلى قرارٍ واضح. نختار العتبة على مجموعة التحقق وفق كلفة الخطأ، ثم نثبتها قبل لمس الاختبار. المثال الآتي يحسب المقاييس مباشرة كي نرى أثر العتبة بدل الاكتفاء برقم الدقة:

```python
import numpy as np
from sklearn.metrics import confusion_matrix, precision_recall_fscore_support

# 1 تعني حالة تستلزم مراجعة بشرية.
y_true = np.array([1, 1, 1, 0, 0, 0, 0, 0])
prob = np.array([0.91, 0.62, 0.38, 0.81, 0.44, 0.31, 0.20, 0.05])
threshold = 0.60  # يختارها الفريق على التحقق وفق سعة المراجعة والكلفة
y_pred = (prob >= threshold).astype(int)

print(confusion_matrix(y_true, y_pred, labels=[0, 1]))
precision, recall, f1, _ = precision_recall_fscore_support(
    y_true, y_pred, average="binary", zero_division=0
)
print(f"precision={precision:.2f}, recall={recall:.2f}, F1={f1:.2f}")
# المتوقع: precision=0.67 وrecall=0.67 وF1=0.67 ثم [[2 ,1] ,[1 ,4]]
```

تُظهر المصفوفة أن صفوفها تمثّل الحقيقة وأعمدتها التنبؤ، بالترتيب الذي طلبناه `[0, 1]`. إذا كانت مراجعة الحالة السلبية مكلفة، فقد نرفع العتبة؛ وإذا كان تفويت حالة خطرة غير مقبول، نخفضها ونقبل بمراجعات أكثر. لا تُشتق العتبة من الاختبار، وإلا أصبح الاختبار جزءًا من التدريب.

8.8 حالة واقعية: حين صار قرار المراجعة أهم من AUC

شركة مدفوعات لديها 0.6% عمليات احتيال، وفريق مراجعة يستطيع فحص 300 تنبيه يوميًا. النموذج الأول حقق AUC مرتفعًا، لكن عند العتبة التي تعطي إحكامًا 0.95 لم يكتشف سوى 18% من الاحتيال. النموذج الثاني كان أقل في AUC، لكن منحنى PR لديه أفضل في المنطقة التي تهمّ: استدعاء 0.64 عند إحكام 0.28. بعد معايرة الاحتمالات على مجموعة تحقق، اختار الفريق عتبةً تُنتج نحو 280 تنبيهًا يوميًا، وربط تكلفة الاحتيال المفقود بتكلفة المراجعة. النتيجة لم تكن «نموذجًا أفضل»، بل **قرارًا أوضح**: كم حالة نريد أن نمسك، وبأي ثمن؟

8.9 أمثلة تنفيذية إضافية

اختيار العتبة

```python
import numpy as np
threshold = 0.60
y_pred = (validation_prob ⩾ threshold).astype("int32")
# Do not tune a threshold on the test set.
```

تقرير التصنيف

```python
from sklearn.metrics import classification_report
print(classification_report(y_true, y_pred, zero_division=0))
```

8.10 أخطاء شائعة

⛔ خطر

العرض: دقة ممتازة في التدريب والتحقق، ثم انهيار في الإنتاج. **السبب:** تطبيع أو اختيار ميزات حُسب على كل البيانات قبل التقسيم، أو ميزة تُعرف بعد الحدث. **الحل:** اقسم أولًا، ولائم المعالجة داخل طية التدريب، وراجع كل ميزة بسؤال: هل تتوفر فعلًا لحظة التنبؤ؟

⚠ تحذير

العرض: تتحسّن النتيجة في كل مرة تعيد فيها ضبط العتبة على الاختبار. **السبب:** الاختبار تحوّل عمليًا إلى مجموعة تحقق. **الحل:** اضبط على التحقق، وافتح الاختبار مرة واحدة، ثم سجّل النتيجة كما هي.

⚠ تحذير

العرض: نموذجان لهما أرقام مختلفة، وأحدهما «أفضل» بفارق 0.3%. **السبب:** تقسيمات مختلفة، أو معالجة مختلفة، أو بذرة واحدة بلا تكرار. **الحل:** وحّد التقسيم والبذور، وقارن المتوسط والتشتت عبر طيات متعددة، لا رقمًا واحدًا.

8.11 تمارين

1. **حساب يدوي:** مصفوفة تعطي TP=40، FP=10، FN=60، TN=890. احسب الإحكام والاستدعاء وF1. (المتوقع: الإحكام 0.8، الاستدعاء 0.4، F1 نحو 0.533.)
2. **تقسيم طبقي:** أنشئ بيانات بنسبة 8% موجبة، واقسمها بـ `train_test_split(stratify=y)`. اطبع نسبة الموجب في كل جزء. (المتوقع: نسبة قريبة من 8% في الثلاثة.)
3. **كشف تسريب:** مشروعٌ يطبّع كل الأعمدة قبل التقسيم، وفيه مرضى مكررون بين المجموعتين. اذكر التسريبين وخطة الإصلاح. (المتوقع: تسريب معالجة وتسريب مجموعات؛ الحل خط أنابيب داخل الطية وتقسيم بالمجموعة.)
4. **معايرة:** نموذج أعطى ثقة 0.9 لمئة عينة، وكان منها 70 موجبة. ما مساهمة هذه الحزمة في ECE؟ (المتوقع: 0.2 مضروبة في وزن الحزمة 0.1 إن كانت تمثل عُشر البيانات، أي 0.02.)
5. **أوزان:** بيانات فيها 950 سالبًا و50 موجبًا. احسب وزن كل فئة بالمعادلة. (المتوقع: نحو 0.526 للسالب و10 للموجب.)
6. **Hyperband:** لماذا قد يكون أسرع من البحث العشوائي بثلاثين تجربة كاملة؟ (المتوقع: لأنه يوقف التجارب الضعيفة بعد دورات قليلة ويخصص الزمن للواعد منها.)
7. **تحليل أخطاء:** لديك 25 حالة موجبة فاتت النموذج. اقترح ثلاث فرضيات وطريقة اختبار كل واحدة. (المتوقع: وسم خاطئ، ميزة ناقصة، انحراف شريحة؛ تُختبر بمراجعة العينات ومقارنة الشرائح.)
8. **اختيار مقياس:** بيانات احتيال بنسبة 1% وفريق مراجعة محدود. أتختار ROC أم PR؟ (المتوقع: PR، لأنه يوضّح الأداء في الفئة النادرة وقرار العتبة.)

8.12 ملخص

- مجموعة الاختبار تُفتح مرة واحدة؛ كل نظرة إضافية إليها تسرّب معلومة إلى قرارك.
- التسريب يبدأ من المعالجة والتكرار والزمن والمجموعات، لا من النموذج وحده.
- التحقق المتقاطع الطبقي يعطيك توزيعًا لا رقمًا، ويكشف هشاشة النموذج.
- الإحكام والاستدعاء يقيسان وجهين مختلفين للخطأ، وF1 يوازن بينهما بلا سياق التكلفة.
- ROC يقيس الترتيب، وPR أنسب للفئات النادرة، والمعايرة تقيس صدق الاحتمال.
- أوزان الفئات وإعادة العينات وخسارة التركيز تعالج عدم التوازن، لكنها تُغيّر الاحتمالات.
- الضبط بلا تتبّع تجارب وبلا تحليل أخطاء يعيدك إلى نفس النقطة بثقة أكبر.

الجزء الثاني · بناء الشبكات وتدريبها

الفصل التاسع

09

tf.data: هندسة خطوط البيانات

حكاية الفصل

صار تدريب مصنف الصور سريعًا على GPU، لكن الجهاز ينتظر البيانات معظم الوقت. كانت الصور تُقرأ وتُفكّ واحدةً تلو الأخرى في Python. كيف نجعل خط الإدخال يواكب المسرّع دون تحميل البيانات كلها إلى الذاكرة؟

9.1 أهداف الفصل

- ☐ بناء Dataset من موترات أو مولّد.
- ☐ ترتيب shuffle mapg batchg cacheg prefetchg.
- ☐ فهم TFRecord والتوازي.
- ☐ قياس خط البيانات وتحديد اختناقه.

9.2 Dataset سلسلة تحويلات

يمكن النظر إلى إنتاجية الخط على أنها أقل إنتاجية بين مراحله المتوازية:



شكل 9-1 خط تدفق البيانات بواسطة tf.data

$$Throughput_{pipeline} \approx \min(r_{read}, r_{parse}, r_{augment}, r_{device})$$

`r` معدل العناصر في الثانية لكل مرحلة. لذلك لا يفيد تسريع القراءة إذا بقي فك الصور أبطأ مرحلة.

تخيّل خطًا في مصنع: لا تستطيع محطة التعبئة إنتاج أكثر مما تسمح به أبطأ محطة قبلها. خط البيانات كذلك؛ فـ `map` يصف ما تفعله بكل مثال، و `num_parallel_calls` يتيح تنفيذ أمثلة مستقلة في الوقت نفسه، و `batch` يجمعها لتقليل كلفة استدعاء النموذج، و `prefetch` يحضّر الدفعة التالية بينما تعمل البطاقة على الدفعة الحالية. إذا بقي GPU خاملًا، فالسؤال المفيد ليس «كيف أزيد حجم النموذج؟» بل «أي مرحلة تجعل المسرّع ينتظر؟».

تبدأ `tf.data.Dataset` بمصدر، ثم تحول العناصر على دفعات أو بالتوازي. `from_tensor_slices` يناسب بيانات موجودة في الذاكرة؛ `from_generator` يربط مولّد Python حين لا يناسب المصدر ذلك، لكنه قد يحد من قابلية التسلسل والأداء. للملفات الكبيرة، اقرأها كتدفق بدل تجميعها أولًا في مصفوفة عملاقة.

```python
import tensorflow as tf

x = tf.range(1000, dtype=tf.float32)
y = 3.0 * x + 1.0
ds = tf.data.Dataset.from_tensor_slices((x, y))
ds = ds.shuffle(1000, seed=42, reshuffle_each_iteration=True)
ds = ds.batch(32, drop_remainder=False).prefetch(tf.data.AUTOTUNE)
for xb, yb in ds.take(1):
    print(xb.shape, yb.shape)
# المتوقع: دفعة حتى 32 قيمة ودفعة أهداف بالشكل نفسه.
```

9.3 ترتيب التحويلات قرار ذاكرة وزمن

نفّذ shuffle على أمثلة التدريب قبل batch كي تختلط الأمثلة، واجعل prefetch قرب نهاية السلسلة ليتداخل تجهيز الدفعة مع التدريب. `cache` يخزن ناتج التحويل؛ إن وضعته قبل augmentation العشوائي حافظت على القراءة المكلفة وأبقيت augmentation متغيرًا في كل مرور. إن وضعت cache بعده، قد تثبت التحويل العشوائي لأول مرور. افهم ما الذي تخزنه قبل استخدامه.

التحويل	الأثر	ملاحظة
`map(..., AUTOTUNE)`	تحويل العناصر	دالة نقية أسهل للتوازي
`shuffle(buffer)`	يخلط ضمن المخزن	المخزن الأصغر خلطه أقل
`batch(size)`	يجمع الأمثلة	آخر دفعة قد تكون أصغر
`cache()`	يقلل القراءة اللاحقة	يحتاج مساحة ذاكرة/قرص
`prefetch(AUTOTUNE)`	يتداخل الإدخال والتدريب	غالبًا يوضع أخيرًا

هذا مثال صغير يربط أسماء التحويلات بوظيفتها. `AUTOTUNE` يسمح لـTensorFlow باختيار مستوى من التوازي، ثم تقيس زمن الحقبة واستهلاك الذاكرة على جهازك. لا تضع `cache()` بلا تقدير للحجم: صور مفكوكة بدقة كاملة قد تستهلك أضعاف مساحة الملفات المضغوطة.

```python
import tensorflow as tf

AUTOTUNE = tf.data.AUTOTUNE
ds = tf.data.Dataset.from_tensor_slices((features, labels))
ds = ds.shuffle(10_000, seed=42, reshuffle_each_iteration=True)
ds = ds.map(parse_and_augment, num_parallel_calls=AUTOTUNE)
ds = ds.batch(64, drop_remainder=False)
ds = ds.prefetch(AUTOTUNE)

for batch_x, batch_y in ds.take(1):
    print(batch_x.shape, batch_y.shape)
```

عقد التحويل هنا واضح: تبدأ كل عينة بمدخل ووسم، ثم تعيد `parse_and_augment` زوجًا من الشكل نفسه، ثم يضيف `batch` محور الدفعة. وإذا كان التحويل عشوائيًا، فاجعله بعد التخزين المؤقت كي لا تعيد التدريب على نسخة augmentation ثابتة كل حقبة. أما خط الاختبار فيبقى حتميًا عادةً؛ فالخلط فيه يصعّب مطابقة التنبؤات مع أمثلتها.

9.4 الملفات الكبيرة TFRecord

TFRecord صيغة سجلات متسلسلة مفيدة للتدفق والتوزيع، وليست ضغطًا سحريًا للصور. خزّن مخطط الحقول وإصدار المعالجة مع البيانات. استعمل `interleave` لقراءة عدة ملفات بالتوازي عندما يسمح التخزين بذلك. قارن زمن الحقبة واستهلاك الذاكرة، لأن التوازي المفرط قد يزاحم التدريب بدل تسريعه.

9.5 حالة واقعية: صور خط إنتاج

يقرأ الخط أسماء الملفات والملصقات، يخلط مسارات التدريب فقط، يفك الصور ويغيّر حجمها ضمن `map` متوازٍ، ثم يجمعها ويطبق augmentation أثناء التدريب. يبقى الاختبار حتميًا ومن دون تحويلات عشوائية. سجّل عدد الصور في الثانية على مراحل: القراءة، وفك الترميز، ثم التدريب؛ عندها تعرف موضع الاختناق.

9.6 من ملف صورة إلى دفعة مدربة

المسار القابل للصيانة يفصل تعداد الملفات عن فك الترميز وعن التحويل. خزّن قائمة المسارات والملصقات بإصدار معلوم، واخلط مسارات التدريب ببذرة محددة قبل القراءة، ثم طبّق augmentation بعد فك الصورة. عند تشغيل تقييم، لا تخلط ولا تستخدم تحولات عشوائية. يجب أن تكون كل خطوة محددة من اسم ملف أو

سجل إلى Tensor بمعنى معلوم.

```python
import tensorflow as tf

def read_image(path, label):
    encoded = tf.io.read_file(path)
    image = tf.io.decode_jpeg(encoded, channels=3)
    image = tf.image.resize(image, (224, 224))
    image = tf.cast(image, tf.float32) / 255.0
    return image, label

train_ds = tf.data.Dataset.from_tensor_slices((paths, labels))
train_ds = train_ds.shuffle(len(paths), seed=42, reshuffle_each_iteration=True)
train_ds = train_ds.map(read_image, num_parallel_calls=tf.data.AUTOTUNE)
train_ds = train_ds.batch(32).prefetch(tf.data.AUTOTUNE)
```

المثال يفترض أن الملفات JPEG وأن `paths` و`labels` متطابقان؛ أضف فحصًا للعناصر المفقودة والأبعاد والملصقات قبل تدريب مكلف. إذا كانت الصورة ملونة أو القناة اللونية مختلفة، ثبّت سياسة اللون في التحضير. لا تخلط التحويل إلى `[0,1]` مع `preprocess_input` الخاص بنموذج جاهز دون مراجعة ما يتوقعه النموذج.

9.7 متى تختار TFRecord؟

TFRecord حاوية لسجلات متسلسلة، و`tf.train.Example` يصف ميزات عددية أو بايتية أو قوائم أعداد. يفيد في تدفق ملفات مجزأة وتوزيعها على العمال، لكنه ليس قاعدة بيانات ولا يحسن الأداء تلقائيًا. ينبغي توثيق schema ونسخة المعالج، وفحص مجموعة عينة من السجلات، وقياس القراءة من التخزين المستهدف. تغيير صيغة التسجيل يتطلب قابلية ترحيل أو إعادة إنتاج المصدر.

9.8 الحتمية والتوزيع

لإعادة التجربة، ثبت البذور في مولد الخلط وفي مكتبات المعالجة، وسجل ما إذا كانت العمليات الحتمية مفعلة. قد تختلف بعض عمليات GPU أو الأداء بين العتاد والإصدار حتى مع بذرة واحدة؛ اذكر حدود إعادة الإنتاج بدقة. في التدريب الموزع، وزع shards بحيث لا يقرأ عاملان الأمثلة نفسها بالخطأ، واضبط batch size ليكون عالميًا أو لكل replica بوضوح. راقب عدد الأمثلة التي رأتها كل حقبة إذا كانت بياناتها غير قابلة للقسمة بالتساوي.

9.9 تشخيص اختناق البيانات

قِس إنتاجية النموذج على Tensor موجود في الذاكرة أولًا، ثم قِس الخط الكامل. إن كان النموذج سريعًا في الاختبار الأول وبطيئًا مع الملفات، فافصل القراءة، وفك الضغط، والتحويل، والخلط، والنسخ. `AUTOTUNE` يختار التوازي بناءً على الملاحظات لكنه لا يخلق مساحة تخزين أو CPU. كما أن `cache()` بعد تحويل مكلف قد يستهلك ذاكرة أكثر من المتاحة؛ يمكن التخزين إلى ملف مؤقت معلوم إذا كان مناسبًا.

9.10 الثبات وإعادة التشغيل

`Dataset.shuffle` يحتاج buffer بحجم مناسب للتوزيع؛ buffer صغير لا يخلط كامل المصدر، وإن كان يحسن التنوع عن الترتيب الأصلي. ثبّت seed وسجل `reshuffle_each_iteration` . الحتمية الكاملة قد تتطلب تفعيل عمليات deterministic وقد تقلل السرعة، كما تختلف بين إصدارات GPU. في تجربة علمية، سجل ما الذي ثُبّت بالفعل وما الذي بقي احتماليًا.

`repeat()` يجعل المصدر لا نهائيًا؛ عند استخدامه يجب تحديد `steps_per_epoch` وإلا لن تعرف نهاية epoch. لا تستخدم `repeat` بلا داعٍ مع `fit` على Dataset منتهٍ. `drop_remainder=True` يثبت حجم الدفعة ويساعد بعض المجمعات، لكنه يرمي أمثلة أخيرة كل حقبة؛ احسب عددها وقرر هل ذلك مقبول للفئات النادرة.

9.11 schemag TFRecord

سجل TFRecord يحتوي بايتات، و `tf.train.Example` رسالة features بأشكال bytes أو float أو int64. خزّن في وصف خارجي نسخة schema وكيفية فك كل ميزة. ابدأ بعدد ملفات يناسب التوازي؛ ملف ضخم واحد قد يقيد القراءة، وملفات صغيرة جدًا تضيف كلفة فتح metadata. استخدم checksum أو إحصاء عدد السجلات لتكشف ملفًا مبتورًا قبل التدريب. لا تخزن مسارات محلية مطلقة إذا ستنقل dataset إلى فريق آخر.

9.12 زيادة البيانات على مستوى batch

هناك augmentation في طبقات Keras مثل RandomFlip تعمل داخل النموذج أثناء التدريب، وهذا يسهل تضمين التحويل ضمن artifact ويجعل سلوك الاستدلال identity عند `training=False` . وهناك augmentation داخل `Dataset.map` ، وهذا قد يوازن الكلفة مع الإدخال. حدد هل عشوائية التحويل per-example أو per-batch. إذا نفذت تعديلًا على label كصندوق كشف أو قناع، طبّق الهندسة نفسها على الصورة والهدف؛ تحويل الصورة وحدها يفسد الإشراف.

9.13 مراجعة قبل التدريب

ارسم 16 عينة بعد كل خطوة: بعد القراءة، وبعد resize، وبعد normalization، وبعد augmentation. اطبع shape وdtype وmin وmax ومدى القيم. افحص عدة labels وclass counts، ومرر batch واحدة في النموذج. هذه المراجعة الصغيرة تلتقط خلط المسار بالملصق، القناة اللونية، وقناعًا فقد المحاذاة قبل صرف وقت GPU.

9.14 نمط reproducible لـ Dataset

إذا كان التحويل يستخدم عشوائية، فمرر seed صريحًا أو استخدم طبقات Keras ذات seed، وسجل ترتيب الخلط. لا تفترض أن `shuffle(seed=42)` يجعل كامل التدريب مطابقًا بين إصدارات النظام أو عدد العمال. عند الحاجة إلى تدقيق علمي، فعّل deterministic operations، واختبر أن dataset نفسها تعطي تسلسلًا ثابتًا، ثم قس تكلفة الحتمية.

في التوزيع، استفد من `Dataset.shard` أو استراتيجية التوزيع المعتمدة كي لا يعالج كل عامل كامل البيانات. احسب عدد الأمثلة الفعلي لكل عامل بعد drop remainder. إذا كانت الفئات مصنفة في مجلدات مرتبة، فاختلاط المسارات يزيل تحيز ترتيب الملفات؛ لكنه لا يصلح تقسيمًا مسربًا بين مصادر متشابهة.

9.15 مسألة cache عمليًا

`cache()` بلا مسار يحتفظ بالبيانات في ذاكرة العملية، ويكتمل التخزين بعد استهلاك dataset إلى النهاية. إذا أوقفت epoch قبل إكمال المرور، قد يعاد بناء cache في الحقبة التالية. إذا كان التحويل بعد cache عشوائيًا، يتكرر augmentation مع كل epoch؛ إذا قبله، قد تتجمد العينة. اختر موقع cache بناءً على ما هو مكلف وما ينبغي أن يبقى عشوائيًا، وراقب مساحة القرص عند استخدام اسم ملف.

9.16 معايير قبول خط البيانات

عند تسليم pipeline، قس وقت تحميل batch، إنتاجية الحقبة، نسبة تلف الملفات، عدد الأمثلة، توازن الفئات، ذاكرة cache، وثبات المعالجة بين train وinference. اكتب عقدًا مختصرًا لكل عنصر Dataset. إذا غيّر الفريق image size أو feature schema، زد data version وأعد التقييم؛ لا تحدث نفس الملف في مكانه بصمت.

9.17 هندسة CSV والنصوص

لـ CSV، ثبت header وأنواع الأعمدة وencoding، وتعامل مع missing values قبل batch. لا تستخدم `from_tensor_slices` على DataFrame ضخم إذا كان التحويل ينسخ البيانات كلها إلى RAM؛ استخدم reader متدفقًا أو صيغًا موزعة. للنص، حول bytes إلى UTF-8 ثم tokenize مع vocabulary محدد. وثق هل التقطيع يتضمن علامات ترقيم وتشكيلًا، لأن هذه المعالجة جزء من نموذجك.

`Dataset.from_generator` مفيد لتكامل مصدر Python خاص، لكنه يضع قيودًا على graph serialization والتوزيع، وقد يعيد التنفيذ في Python لكل عنصر. اكتب `output_signature` واضحًا، وقارن سرعة generator مع مصدر أصلي TensorFlow. إذا كان المصدر قابلًا للتحويل إلى TensorFlow I/O أو TFRecord، قد تحصل على أداء أكثر استقرارًا.

9.18 البيانات التي تتغير عند القراءة

إذا كان المصدر live أو متغيرًا، سجّل snapshot أو query boundary قبل التدريب. قراءة قاعدة بيانات عبر عدة عمال قد تنتج مجموعات مختلفة بين التجارب. استخدم pagination أو shards ثابتة ومعرّف dataset version. عند فشل ملف، لا تسقطه بصمت عبر `ignore_errors` إذا كانت نسبته مهمة؛ سجل العدد والمصدر، وضع حدًا للفشل يسمح بإيقاف التدريب بدل تقييم بيانات ناقصة.

9.19 أمثلة تنفيذية إضافية

تحويل متواز

```python
ds = ds.map(parse_example, num_parallel_calls=tf.data.AUTOTUNE)
```

فحص الدفعة

```python
for features, labels in ds.take(1):
    print(features.shape, labels.shape)
```

9.20 فحوص قبول قبل حساب مكلف

احتفظ بإحصاءات الخط قبل وبعد إعادة بناء المصدر. إذا اختلف العدد أو التوازن أو متوسط السمة، أوقف المقارنة حتى تفسر الفرق؛ قد تكون البيانات تغيرت لا النموذج. احسب تكلفة epoch كاملة مبكرًا، ثم قرر إن كانت cache أو صيغة تسجيل أخرى تستحق وقت التحويل.

9.21 حجم الملفات والمشاركة

جزّئ سجلات TFRecord حسب مصدر تخزين وتوازي القراءة، واستخدم أسماء ملفات تشمل نسخة schema. لا تضع الملفات المؤقتة في مسار cache غير مضمون ثم تعتبرها مصدرًا دائمًا. احسب checksums عند نقل dataset، وسجل وقت اكتمال النسخ.

9.22 فحص label mapping

استخرج خريطة الفئات مرة واحفظها مع نسخة النموذج. إذا ولّد loader معرفات بفئات مرتبة أبجديًا بينما API تستخدم ترتيبًا يدويًا، تكون التوقعات صحيحة رقميًا ومسمّاة خطأ. أضف اختبارًا على اسم معروف وفئة نادرة.

9.23 أخطاء شائعة

> ⚠ **تحذير**
>
> `shuffle` بعد `batch` يخلط الدفعات لا الأمثلة داخلها. [DANGER!] لا تستخدم `cache` في الذاكرة لبيانات تفوقها دون قصد؛ قد يتحول التسريع إلى نفاد ذاكرة.

9.24 تمارين

1. أنشئ Dataset من أزواج سمات وأهداف. (المتوقع: كل عنصر زوج متوافق.)
2. رتب التحويلات للتدريب والتقييم. (المتوقع: خلط للتدريب فقط.)
3. قارن prefetch مع بدونه. (المتوقع: قياس زمن الحقبة.)
4. فسّر أين تضع cache عند وجود augmentation عشوائي. (المتوقع: قبل التغيير العشوائي.)
5. اختر مصدرًا لبيانات أكبر من الذاكرة. (المتوقع: تدفق ملفات/TFRecord.)

6. حدّد اختناق خط صور. (المتوقع: قياس القراءة والفك والتدريب منفصلة.)

9.25 ملخص

- صمّم خطًا متدفقًا ومتوازيًا بدل قراءة عنصر في كل خطوة Python.
- ترتيب التحويلات يغير دلالة الخلط والعشوائية واستهلاك الذاكرة.
- قِس الإنتاجية ولا تفترض أن زيادة التوازي تفيد دائمًا.

الجزء الثاني · بناء الشبكات وتدريبها

الفصل العاشر

10

الرؤية الحاسوبية: CNN من الالتفاف إلى الإنتاج

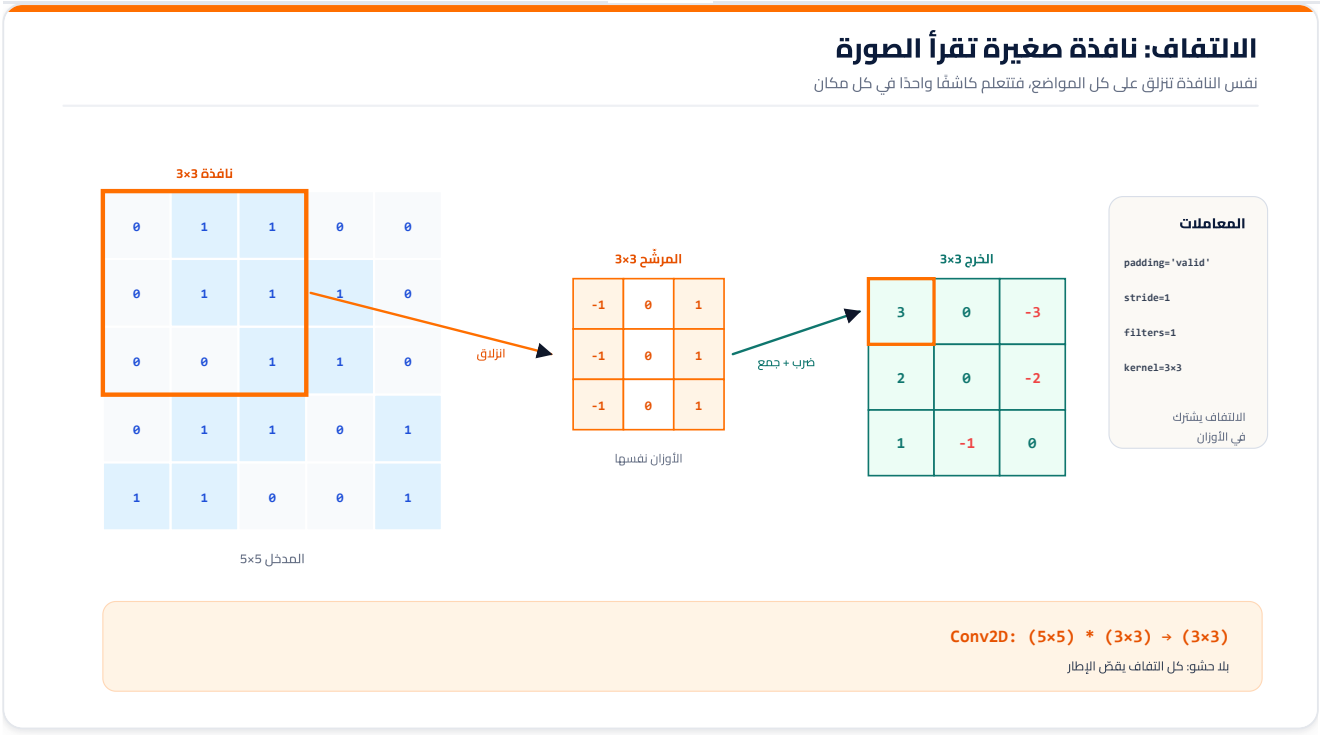
> **حكاية الفصل**
>
> طلبت ورشة صيانة فرز خدوش دقيقة على ألواح كبيرة. أعطى المصنف الكثيف نتائج ضعيفة، لأن تسطيح الصورة أزال الجوار المكاني ورفع عدد الأوزان. كيف تستفيد الشبكة من حقيقة أن الحافة القريبة تشبه الحافة نفسها في موضع آخر؟

10.1 أهداف الفصل

- ☐ حساب أبعاد الالتفاف وفهم القنوات والمرشحات.
- ☐ بناء CNN صغيرة وشرح pooling والتطبيع.
- ☐ مقارنة عائلات معمارية معاصرة.
- ☐ تفسير تنبؤ بصورة عبر Grad-CAM.

10.2 مرشح محلي يعاد استخدامه

تطبق طبقة الالتفاف مرشحات صغيرة على رقع متجاورة، فتتعلم حوافًا وأنماطًا محلية وتعيد استخدام الأوزان عبر المواضع. يحدد `stride` مقدار الحركة، ويضيف `padding="same"` حوافًا للحفاظ تقريبًا على الحجم المكاني. عدد المرشحات هو عدد قنوات الخرج.



شكل 10-1 حركة مرشح الالتفاف على الصورة



شكل 10-2 مكونات شبكة تلافيف للصور

$$H_{out} = \left\lfloor \frac{H + 2P - D(K-1) - 1}{S} \right\rfloor + 1$$

`H` ارتفاع الإدخال، و`K` حجم المرشح، و`P` الحشو، و`D` التمدد، و`S` الخطوة. تنطبق الصيغة على محور واحد، وتكرر للعرض.

```python
import keras
from keras import layers

model = keras.Sequential([
    layers.Input((64, 64, 3)),
    layers.Conv2D(32, 3, padding="same", use_bias=False),
    layers.BatchNormalization(), layers.Activation("relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, padding="same", activation="relu"),
    layers.GlobalAveragePooling2D(),
    layers.Dense(5, activation="softmax"),
])
model.summary()
```

يقلل pooling الدقة المكانية ويزيد مجال الرؤية النسبي، بينما `GlobalAveragePooling2D` يجمع كل خريطة سمات في قيمة واحدة ويحد من تضخم المعاملات مقارنة بـ Flatten.

10.3 من LeNet إلى ConvNeXt

العائلة	الفكرة التعليمية	ما الذي تراقبه؟
LeNet	التلافيف والتجميع للمشاهد الصغيرة	البساطة وعدد الطبقات
VGG	تتابع مرشحات صغيرة	كثرة الأوزان والذاكرة
ResNet	وصلات تخطٍ	تدريب شبكات أعمق
EfficientNet	موازنة العمق والعرض والدقة	كلفة الاستدلال
ConvNeXt	تحديث تصميم CNN الحديثة	السياق والموارد المطلوبة

لا تختَر النموذج باسم عائلته فقط. قارن الدقة والزمن والذاكرة على صورك، وافصل بيانات التحقق بحسب مصدر الصور كي لا يتسرب المشهد نفسه إلى التدريب والاختبار.

10.4 تفسير بصري لا يثبت السببية

Grad-CAM يجمع تدرجات فئة مستهدفة مع خرائط السمات في طبقة التفاف، فينتج خريطة تقريبية للمناطق المؤثرة في الدرجة. استخدمه لاكتشاف اعتماد النموذج على علامة مائية أو خلفية، لا لإثبات أن النموذج «فهم» الكائن. اختبر التفسير على أخطاء وصور مضادة وعلى طبقات مختلفة.

10.5 حالة واقعية: كشف خدش

تُقص صور الألواح وفق إجراء ثابت، وتُطبّع بالطريقة نفسها في التدريب والخدمة. يبدأ الفريق بخط أساس مدرب مسبقًا ثم يوازن الحساسية مع زمن الفحص. يسجل عدد الحالات التي فاتت كل وردية، ويعيد التقييم عند تغيير الكاميرا أو الإضاءة؛ فالتغير الفيزيائي قد يبدل التوزيع دون تغيير الشفرة.

10.6 مجال الرؤية وحساب الكلفة

ترى المرشحة الأولى رقعة محلية، ثم تزيد الطبقات والتجميع مجال الرؤية الفعلي. هذا يفسر كيف تنتقل الشبكة من الحواف إلى الأشكال، لكنه يفسر أيضًا لماذا لا تكفي طبقة صغيرة لعيب واسع. يمكن حساب مجال الرؤية نظريًا من تتابع أحجام المرشحات والخطوات؛ اكتب الحجم المكاني بعد كل طبقة على عينة صغيرة قبل تدريب المجموعة الكاملة.

معاملات التفاف 2D تحسب تقريبًا `K_h × K_w × C_in × C_out`، ويضاف انحياز لكل قناة إذا لم يُلغَ. تغيير عرض الشبكة يضاعف الحساب والذاكرة؛ مضاعفة عدد القنوات في الداخل والخارج قد ترفع عدد المعاملات بنحو أربعة أضعاف. قِس أزمنة inference على جهاز التنفيذ نفسه: بطاقات الخادم والهواتف تتفاوت في دعم المرشحات والدفعات.

10.7 زيادة البيانات يحافظ على معنى الملصق

التقليب الأفقي صالح لبعض صور المنتجات، لكنه قد يقلب معنى النص أو اتجاه قطعة غير متناظرة. الدوران الشديد قد يحول علامة اتجاه إلى عيب، والتباين المبالغ فيه قد يمحو خدشًا منخفض التباين. اربط كل زيادة بفرضية عن تغير العالم الحقيقي، واعرض لوحة عينات ناتجة قبل التدريب. طبّق التغيير على مجموعة التدريب فقط، واحتفظ بنسخة غير معدلة للاختبار.

10.8 النقل من نموذج مدرب مسبقًا

الأوزان المدربة على صور عامة تعلم سمات مفيدة غالبًا، لكن بيانات المجال تختلف في اللون والدقة وتوزيع الفئات. استخدم preprocessing الذي يطابق قاعدة النموذج، وجرب خط أساس مجمدًا ثم فك الطبقات العليا بمعدل أصغر. لا تخلط مصطلح `include_top=False` مع تطبيع الإدخال؛ الأول يزيل رأس التصنيف الأصلي، والثاني يحول الصورة إلى المقياس المتوقع.

10.9 تفسير Grad-CAM ومسؤولية التحقق

احسب تدرج logit الفئة بالنسبة إلى خرائط آخر طبقة تلافيفية، واجمع الخرائط بأوزان مشتقة من متوسط التدرجات مكانياً. طبّع الخريطة للعرض وارسمها فوق الصورة. اختبر حالات صحيحة وخاطئة، وصورة لا تحتوي الهدف، وصورة شُوّهت خلفيتها؛ إن بقيت الحرارة على إطار الصورة، فقد تعلم النموذج أثرًا جانبيًا. Grad-CAM لا يثبت سبب القرار ولا يكفي وحده للتصديق على نظام طبي أو صناعي.

10.10 من التدريب إلى القياس التشغيلي

لا تحسب الدقة على قصاصات متجاورة موزعة عشوائيًا إذا كانت كلها من اللوح أو الدفعة نفسها. اجمع التقييم على مستوى الوحدة التي سيواجهها النموذج: دفعة إنتاج، أو كاميرا، أو يوم عمل. اعرض confusion matrix وأداء كل نوع عيب والكمون عند حجم الدفعة المتوقع. إن كان على المفتش معالجة الصور في زمن محدد، فكمون p95 وحجم الذاكرة شرطان في تعريف النجاح.

10.11 وصفة تدريب قابلة للتكرار

قبل اختيار معمارية عميقة، أنشئ baseline: نموذج خطي أو CNN صغيرة، تطبيع ثابت، وتقسيم مجموعة بحسب مصدر الصور. ثم تحقق أن النموذج يمكنه overfit على دفعة واحدة؛ إذا لم يستطع، راجع labels وloss وshapes. استخدم `ModelCheckpoint` لمراقبة `val_loss` أو مقياس مرتبط بالقرار، و `EarlyStopping` مع استعادة أفضل أوزان. خزّن تاريخ التدريب مع seed، وسجل الفئات وأي mapping.

في CIFAR أو صور عامة، augmentation معقول قد يحسن التعميم، لكن لا تقارن نسخة augmentation على مجموعة تحقق مختلفة. لا تستخدم augmentation على validation/test؛ يجب أن تظل هذه العينات ممثلة للقياس. عند عدم توازن الفئات، اعرض per-class recall وbalanced accuracy أو macro-F1 بجانب accuracy. إن كان هناك threshold، اختَره على validation ثم ثبّته قبل الاختبار.

10.12 لماذا يختلف أداء CNN بين المجالين؟

مرشحات ImageNet تتعلم قوامًا وأشكالًا مفيدة، لكن الصور الطبية قد تكون grayscale وأنماطها أدق من تصنيفات عامة. قص الصورة إلى 224×224 قد يمحو عيبًا صغيرًا، والضغط قد يغير texture. جرّب resolution وتنسيق الصورة والمعالجة كفرضيات، وافصل أثرها عن أثر العمق. في البيانات عالية الدقة، tiled inference قد يساعد لكنه يكرر الحواف ويتطلب دمج الصناديق أو الأقنعة.

10.13 ضوابط BatchNorm ودفعة صغيرة

BatchNorm يحدّث moving statistics من الدفعة. إذا كانت الدفعة صغيرة بسبب صور عالية الدقة، قد تصبح الإحصاءات noisy. يمكن إبقاء base المدرب مسبقًا في `training=False` خلال fine-tuning في بعض السيناريوهات، أو اختيار GroupNorm/LayerNorm إن كان التصميم يدعمها. لا تغير normalization في نموذج مدرب مسبقًا دون فحص؛ وزن الطبقات التالية يرتبط بتوزيع التفعيلات المتوقع.

10.14 تهيئة صور الخدمة

يجب أن ينفذ المعالج الصورّي نفس خطوة التدريب: decode، ترتيب القنوات، interpolation، resize/crop، نوع الدقة، normalization. اختبر صورة محفوظة من الإنتاج على pipeline التدريب وpipelineg الخدمة وقارن الموتر الناتج عنصرًا بعنصر. اختلاف BGR/RGB لا يسبب استثناء، لكنه يفسد التنبؤ بصمت. سجّل مكتبة decode وإصدارها عندما يؤثر على النتائج.

10.15 Grad-CAM: تطبيق وتفسير حذر

اختر طبقة التفاف أخيرة لها خريطة مكانية، واحسب gradient للـ class logit، ثم خذ متوسط gradient على المحاور المكانية كوزن لكل قناة. اجمع خرائط القنوات بعد قص الأوزان السالبة، ثم طبّع وأعد الحجم على الصورة الأصلية. يمكن لخريطة مرتفعة على الخلفية أن تكشف dataset shortcut، لكن خريطة تبدو معقولة ليست اختبارًا سببيًا. قارن على أمثلة خاطئة، وبدّل الخلفية عندما يمكن ذلك.

10.16 تصميم مشروع تصنيف صور كامل

ابدأ بجمع بيانات يوضح من التقطها وتحت أي ظروف، واكتب taxonomy فئات لا تتداخل. افحص أمثلة ملتبسة مع خبير واتفق على labeling guidelines. افصل train/validation/test على مستوى المصدر الذي تتوقع التعميم عليه. حدد baseline بسيطًا، ثم CNN صغيرة. احفظ confusions بحسب الفئة، واعرض أمثلة false positive

false negativeg، لأن صورة بعيدة عن التوزيع قد تشبه فئة أخرى.

في التدريب، استخدم augmentation مضبوطًا، learning rate schedule، checkpointg لأفضل metric تحقق. لا تغيّر validation مع كل تجربة. بعد اختيار القرار، افحص set خارجية من كاميرا أو دفعة إنتاج لم تستخدم للضبط. قد تكون الدقة الإجمالية عالية لكن الفئة التي تتطلب الصيانة هي الأضعف؛ استخدم decision threshold أو abstention للصور غير الواثقة بمراجعة بشرية.

10.17 الموازنة بين الدقة والكمون

ResNet أعمق لا يضمن أنه مناسب للكميرا. قس زمن preprocess + forward + postprocess على CPU وGPU الهدف. خفض resolution يقلل الحساب تقريبًا بحسب مساحة الصورة لكنه قد يمحو الأجسام الصغيرة. quantization أو pruning قد يصغر النموذج مع تغير الدقة. اختبر كل ضغط على benchmark ثابت، وسجل model size وكمون p95 وذاكرة الوصول العشوائي.

10.18 أخطاء من الواقع

من الأخطاء الشائعة تسميات غير متسقة بين annotators، قنوات لونية مقلوبة، صور مكررة، أو crop يقطع الكائن. كشف duplicate بصمة الصور يساعد على منع leak. عند استخدام إعادة التحجيم، حدد interpolation؛ nearest للأقنعة، bilinear أو bicubic للصورة وفق العملية. لا تتخلص من صور «غريبة» تلقائيًا؛ قد تكون الحالات الأصعب والأكثر أهمية للتعميم.

10.19 المراجعة العلمية

أبلغ البنية والوزن المدرب مسبقًا ومصدره، طريقة تقسيم البيانات وعدد الصور، سياسة التهيئة، optimizer وجدول التعلم، بذور، epochs، وعتبة القرار. قدم class-wise metrics وفواصل ثقة إن أمكن. اذكر نسخة TensorFlow/Keras والعتاد، وشارك كود preprocessing وmodel configg حتى يتمكن المختبر من تكرار القياس.

10.20 استكشاف feature maps

اعرض تفعيلات طبقة مبكرة وأخرى عميقة للصورة نفسها. الأولى قد تستجيب لحواف واتجاهات، والأخيرة لقوام أو أجزاء ذات صلة بالفئة. استخدام feature map تفسير وصفي لا حكم على فهم النموذج. إذا كانت كل القنوات صفرًا أو ثابتة، افحص تهيئة الطبقة ومقياس الإدخال وحالة training/inference.

10.21 التهيئة المسبقة والملصقات الصاخبة

أوزان pretrained تقلل احتياج البيانات لكنها قد تحمل تحيزًا من مجموعة المصدر. قارن evaluation على فئات أو خلفيات متباينة. في الملصقات غير المؤكدة، لا ترفع capacity قبل تدقيق annotation؛ قد يحفظ النموذج أخطاء الوسم. اجمع أمثلة خلاف المراجعين وحدد سياسة adjudication.

10.22 متطلبات اكتساب البيانات

ضع معيار تصوير يعالج مسافة الكاميرا والإضاءة والfocus والخصوصية. إذا تطورت كاميرات الخط، اجمع بيانات انتقال قبل الطرح الواسع. سجّل metadata مثل الجهاز والوردية في مكان مضبوط الوصول، واستخدمها للتقسيم والتحليل دون تمريرها للنموذج إن لم تكن متاحة وقت الاستخدام.

10.23 أمثلة تنفيذية إضافية

زيادة البيانات

```python
augment = keras.Sequential([
    layers.RandomFlip("horizontal"), layers.RandomRotation(0.05),
])
augmented = augment(images, training=True)
```

تطبيع قيم البكسل

```python
images = tf.cast(images, tf.float32) / 255.0
print(tf.reduce_min(images).numpy(), tf.reduce_max(images).numpy())
```

10.24 أخطاء شائعة

⚠ تحذير

`Flatten` المبكر قد يحول صورة صغيرة إلى طبقة كثيفة ضخمة ويهدر البنية المكانية. [DANGER!] لا تخلط صورًا متجاورة أو مشتقة من الصورة الأصلية بين التدريب والاختبار؛ هذا يرفع القياس دون تعميم حقيقي.

10.25 تمارين

1. احسب حجم خرج مرشح 3×3 بخطوة 2. (المتوقع: تطبيق الصيغة لكل محور.)
2. ابنِ CNN بأربع فئات. (المتوقع: مخرج احتمالات طوله 4.)
3. قارن Flatten وGlobalAveragePooling. (المتوقع: اختلاف عدد المعاملات.)
4. حدد تقسيمًا يمنع تسرب صور اللوح نفسه. (المتوقع: فصل بحسب اللوح أو الوردية.)
5. فسّر خريطة Grad-CAM مضللة. (المتوقع: فحص الخلفية وعلامات المصدر.)
6. اختر عائلة نموذج تحت قيد ذاكرة. (المتوقع: قياس النماذج على الجهاز المستهدف.)

10.26 ملخص

- الالتفاف يستفيد من المحلية ومشاركة الأوزان.
- الخطوة والحشو والمرشح تحدد الشكل والكلفة، وعمق الشبكة ليس هدفًا بذاته.
- Grad-CAM وسيلة فحص مفيدة وليست برهانًا على السببية.